

Thinking in Recursion

A Complete, Intuition-First Guide to Recognizing and Writing Recursive Solutions

How to read this book

This book has one goal: by the end, when you see a problem, you should *feel* whether it is recursive, know instantly which pattern it fits, write the code almost automatically, and state its complexity without hesitation. That is exactly what data-structures exams, GATE-style questions, and coding interviews demand — so this book is built to make you exam-ready, not just conceptually comfortable.

Read Parts I-III in order — they build the mental model. Parts IV-VII catalog every family of recursive code. Parts VIII-X are the exam engine: how to analyze complexity, how to recognize the pattern from the problem statement, and fully worked exam problems. Part XI is quick reference. Every code example is in Python for readability, but the ideas are language-agnostic; a chapter covers language-specific concerns.

Contents:

- **Part I — The Mental Model.** What recursion really is, and how to see it.
 - **Part II — Writing the Logic.** A repeatable recipe for turning intuition into code.
 - **Part III — How Recursion Runs.** The call stack, tracing, and debugging.
 - **Part IV — Linear & Structural Recursion.** Lists, numbers, trees, the natural shapes.
 - **Part V — Branching & Search.** Multiple recursion, backtracking, combinatorics.
 - **Part VI — Divide, Conquer & Optimize.** Divide-and-conquer, memoization, DP.
 - **Part VII — Advanced & Exotic Forms.** Tail, mutual, nested, continuations, and more.
 - **Part VIII — Analyzing Recursion (The Exam Core).** Recurrences, master theorem, recursion trees, space, hidden-cost traps.
 - **Part IX — Pattern Recognition for Exams.** Signal words, the pick/don't-pick master template, and the recursion→memoization→tabulation pipeline.
 - **Part X — Worked Exam Problems.** Full solutions, MCQ traps, and a timed exam protocol.
 - **Part XI — Practice & Reference.** A graded problem gym with solutions, decision guide, and cheat sheets.
-

Table of contents

Part I — The Mental Model ... p.3 1. The one idea behind all recursion ... p.3 2. Your first recursion, felt from the inside ... p.3 3. The two-part anatomy of every recursive function ... p.4 4. How to recognize that a problem is recursive ... p.4 4b. Recursion is induction (why the leap of faith is valid) ... p.5 5. Recursion vs. iteration — the same coin ... p.5 5b. The four forces behind every recursive algorithm ... p.6

Part II — Writing the Logic ... p.9 6. A repeatable recipe ... p.9 6b. One example, ten views: `sum_list` from start to finish ... p.9 7. Choosing the right "smaller" ... p.11 8. Passing state: accumulators and helper functions ... p.12 9. The three most common beginner bugs ... p.13

Part III — How Recursion Runs ... p.15 10. The call stack, without the mystery ... p.15 10b. Work going down vs. work coming back up ... p.15 10c. Three styles of recursion ... p.16 11. Tracing recursion on paper ... p.18 12. When recursion goes too deep ... p.18

Part IV — Linear & Structural Recursion ... p.19 13. Recursion over numbers ... p.19 14. Recursion over lists and strings ... p.20 14b. Linked lists: recursion without slicing ... p.22 15. Trees: recursion's natural home ... p.24 15b. Trees vs. graphs: why one needs a visited set ... p.26 16. Nested and self-similar data ... p.27 16b. What should the recursive call return? (designing the contract) ... p.28

Part V — Branching & Search ... p.32 17. Multiple recursion (p.32) · 18. Generating all possibilities (p.32) · 19. Backtracking (p.34) · 20. Recursive descent parsing (p.36)

Part VI — Divide, Conquer & Optimize ... p.38 21. Divide and conquer (p.38) · 22. Memoization (p.39) · 23. From

recursion to dynamic programming (p.40)

Part VII — Advanced & Exotic Forms (*optional on a first read*) ... p.43 24. Tail recursion (p.43) · 25. Mutual recursion (p.43) · 26. Nested recursion (p.44) · 27. Structural vs. generative (p.44) · 28. Higher-order & corecursion (p.44) · 29. Continuation-passing style (p.45) · 30. Recursion → iteration with an explicit stack (p.45)

Part VIII — Analyzing Recursion (The Exam Core) ... p.47 31. From code to recurrence (p.47) · 32. Three ways to solve a recurrence (p.48) · 33. Space complexity (p.50) · 34. Hidden-cost traps (p.50) · 35. Reading complexity in 30 seconds (p.51)

Part IX — Pattern Recognition for Exams ... p.52 36. Signal words (p.52) · 37. The pick/don't-pick template (p.52) · 38. Other reusable templates (p.54) · 39. Recursion → memoization → tabulation (p.56) · 40. Defining the state (p.57)

Part X — Worked Exam Problems ... p.59 41. Write-the-recursion (p.59) · 42. Optimize-with-DP (p.60) · 43. Analyze-this-recursion (p.61) · 44. MCQ traps (p.62) · 45. A ten-minute exam protocol (p.62)

Part XI — Practice & Reference ... p.64 46. Language notes (p.64) · 47. Decision guide (p.64) · 48. The problem gym (with solutions) (p.64) · 49. Cheat sheet (p.77) · 49b. The exercise bank (practice without scaffolding) (p.79) · 50. Glossary (p.82)

Three ways through this book

Not everyone needs every chapter on the first pass. Pick the path that matches your goal, then come back for the rest.

The beginner path (build rock-solid intuition first). Read Parts I, II, and III slowly — especially Chapters 6b, 10b, and 10c, which are the heart of the intuition. Then Part IV. Do the trace-and-fill exercises in Chapter 48 as you go. *Skip Part VII entirely* until the core feels natural; tail recursion, Ackermann, CPS and corecursion add completeness but can make recursion look bigger than it is before you've practiced the basics. When Parts I-IV feel easy, continue to V and VI.

The interview path (fastest route to solving problems). Skim Parts I-III to calibrate the mental model, then focus on Parts IV, V, VI, IX, and X. Chapter 37 (the pick/don't-pick template) and Chapter 39 (recursion → memoization → tabulation) carry most of the weight. Drill the Chapter 48 gym with a timer.

The exam path (recursion on a DSA/GATE-style test). Do the interview path, then add Part VIII in full — recurrences, the Master Theorem, space analysis, and the hidden-cost traps — plus Chapters 43-45 on analyzing given code and defeating MCQ traps. The cheat sheet (Chapter 49) is your night-before reference.

Each of the teaching parts (I-X) opens with **learning objectives** and closes with a **"Checkpoint — you should now be able to..." list** so you can tell when you're ready to move on. Part XI is reference material — a problem gym, glossary, and cheat sheet — so it's organized for lookup rather than sequential reading.

Part I — The Mental Model

Learning objectives. Understand what a recursive function promises to return; learn to trust the recursive call without tracing it; identify base case, smaller problem, and combine step; and recognize when a problem is recursive at all.

Chapter 1. The one idea behind all recursion

Recursion is a single idea wearing many costumes:

Solve a problem by assuming you can already solve a smaller version of the same problem, then combining that with a little work of your own.

That is the whole thing. Everything else is detail.

Most people trip because they try to trace the entire chain of calls in their head — imagining the function calling itself calling itself, ten levels deep, and getting dizzy. That is the *wrong* way to think about recursion while writing it. It is like trying to drive by simulating every piston in the engine.

Instead, you use what is often called the **recursive leap of faith**. You write the function as if a *perfect, already-working copy of it* is available to you for smaller inputs. You don't trace inward. You trust.

Here is the shift in one sentence. Iteration says: "*Do this step, then the next, then the next, keeping track of where I am.*" Recursion says: "*This problem is just a small piece plus a smaller version of the same problem — and I already trust the smaller version.*"

Chapter 2. Your first recursion, felt from the inside

Suppose you're standing in a long line and want to know your position from the front, but you can only talk to the person directly ahead of you. You can't see the front.

You reason: "*If I knew the position of the person in front of me, mine is just theirs plus one.*" So you tap them and ask, "What's your position?" They don't know either, so they ask the person in front of *them*. This continues until the question reaches the very first person, who can answer immediately: "I'm number 1." That answer flows back: 1, then 2, then 3... until it reaches you.

Notice the three things that made this work, because they will appear in *every* recursion you ever write:

1. **A smaller version of the same question.** "What's your position?" is the same question, asked of someone closer to the front.
2. **A base case — a point where no more asking is needed.** The first person knows their answer directly.
3. **A way to combine.** Your answer is "their answer plus one."

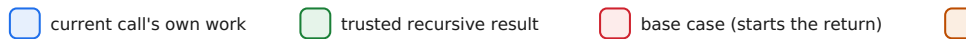
Let's write it:

```
def position(person):
    if person.is_first():           # 1. base case
        return 1
    return position(person.ahead) + 1 # 2. smaller problem 3. combine
```

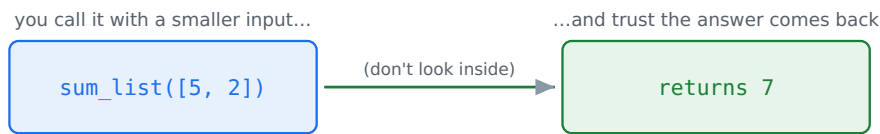
JavaScript:

```
function position(person) {
    if (person.isFirst()) return 1; // 1. base case
    return position(person.ahead) + 1; // 2. smaller problem 3. combine
}
```

You did not simulate the whole line. You described *one person's* reasoning and trusted the rest. That trust is the skill this book trains.



Colour key used in the diagrams throughout this book.



The leap of faith: treat the recursive call as a box that just works.

The recursive call as a black box you trust without opening.

Chapter 3. The two-part anatomy of every recursive function

Every recursive function is built from exactly two kinds of cases. If either is missing or wrong, the function breaks.

The base case(s). The input is small or simple enough that you can answer immediately, with no further recursion. This is what stops the process. A recursion without a reachable base case is like a staircase with no bottom step — you fall forever (in code: a stack overflow).

The recursive case(s). The input is not yet trivial, so you (a) reduce it to one or more smaller inputs, (b) call yourself on those, and (c) combine the results with a bit of your own work.

A useful template to keep in your head:

```

def solve(problem):
    if is_base_case(problem):
        return base_answer(problem)
    smaller = reduce(problem)      # make it smaller
    sub = solve(smaller)           # the leap of faith
    return combine(problem, sub)   # your own bit of work
  
```

JavaScript:

```

function solve(problem) {
    if (isBaseCase(problem)) return baseAnswer(problem);
    const smaller = reduce(problem);      // make it smaller
    const sub = solve(smaller);           // the leap of faith
    return combine(problem, sub);         // your own bit of work
}
  
```

Two rules that prevent 90% of all recursion bugs:

- **The recursive call must operate on a *strictly smaller input*** — smaller in a way that provably marches toward a base case. "Smaller" can mean a smaller number, a shorter list, a smaller range, a subtree, fewer remaining choices — but it must genuinely shrink.
- **Every path must eventually hit a base case.** If some branch of your logic can recurse without ever qualifying as a base case, you have an infinite recursion.

Chapter 4. How to *recognize* that a problem is recursive

This is the chapter most books skip, and it's the one that matters most. Recognition beats memorization. Ask yourself these questions about any problem:

Question 1 — "Can I define this problem in terms of a smaller version of itself?" If the answer is yes, it is recursive. "The factorial of n is n times the factorial of $n-1$." "A directory's total size is the size of its files plus the total size of each subdirectory." "The number of ways to climb n stairs is the ways to climb $n-1$ plus the ways to climb $n-2$." All recursive.

Question 2 — "Does the data itself nest or repeat its own shape?" Trees, filesystems, nested lists, JSON,

HTML/DOM, family trees, organizational charts, comment threads with replies — these are **self-similar** structures. A tree's children are themselves trees. Whenever the data is self-similar, recursion fits like a glove, because the code can mirror the shape of the data.

Question 3 — "Can I make one choice now and leave a smaller version of the same decision for later?"

Puzzles where you place one queen, pick one item, or try one path — then face the same puzzle with one fewer thing to decide — are recursive (this is backtracking, Part V).

Question 4 — "Can I split this into independent halves (or parts) that I solve the same way, then merge?"

Sorting, searching a sorted range, many geometry and array problems split cleanly. That's divide-and-conquer (Part VI).

If a problem answers *yes* to any of these, reach for recursion. With practice, this check takes under a second.

Chapter 4b. Recursion is induction (why the leap of faith is valid)

The "leap of faith" can feel like cheating — how is it legitimate to *assume* the recursive call works while you're still writing it? The answer is that recursion is the computational twin of **mathematical induction**, a proof technique that has been rigorous for centuries. Understanding the correspondence makes the leap of faith feel earned rather than magical.

Induction proves a statement $P(n)$ holds for all n by two steps:

1. **Base step:** prove P holds for the smallest case, e.g. $P(0)$.
2. **Inductive step:** assuming $P(k)$ holds (the "inductive hypothesis"), prove $P(k+1)$ follows.

If both steps hold, $P(n)$ is true for every n — because $P(0)$ gives $P(1)$, which gives $P(2)$, and so on up to any n .

A recursive function has exactly the same two parts, and they line up one-to-one:

Induction	Recursion
Base step: prove $P(0)$ directly	Base case: return the answer for the smallest input directly
Inductive hypothesis: assume $P(k)$ holds	The leap of faith: assume the recursive call on the smaller input is correct
Inductive step: derive $P(k+1)$ from $P(k)$	Combine step: build this call's answer from the smaller call's result
Conclusion: $P(n)$ holds for all n	The function is correct for all inputs in its domain

So the leap of faith **is** the inductive hypothesis. When you write `return xs[0] + sum_list(xs[1:])` and trust the recursive call, you are doing precisely what a mathematician does when they write "assume the claim holds for k ." The trust is licensed by the same logic that makes induction valid: you've handled the base case, and you've shown how a correct smaller answer produces a correct larger one, so correctness propagates from the base up to every input.

This correspondence also tells you *how to prove a recursion correct*, which is occasionally asked on exams:

1. State what the function should return (its contract — the $P(n)$ you're proving).
2. Show the base case returns the right answer.
3. Assume the recursive call is correct for the smaller input (hypothesis).
4. Show the combine step then produces the correct answer for the current input (inductive step).
5. Confirm the input strictly shrinks toward the base case (this guarantees the induction actually reaches every case — the analogue of "n decreases to 0").

Step 5 is the recursion-specific addition: induction over the natural numbers automatically covers all n , but a recursion only terminates if its input provably marches to a base case. That is why "every recursive call must be on strictly smaller input" (Chapter 3) is not just a bug-avoidance tip — it's the condition that makes the underlying induction sound.

The practical payoff: when you're unsure whether a recursion is correct, don't trace it. Instead, check the four induction pieces — base case right? hypothesis clearly stated as the contract? combine step correct given the hypothesis? input shrinking? If all four hold, the recursion is correct, and you never had to unfold the call stack in your head.

Chapter 5. Recursion vs. iteration — the same coin

Any recursive computation can be simulated iteratively — sometimes only by maintaining an *explicit* stack that mimics the call stack (Chapter 30) — and any iterative computation can be re-expressed recursively. In that sense they're equally powerful. The choice is about *fit*, not capability.

Recursion shines when the problem or data is self-similar, when there are multiple branches to explore, or when an explicit "remember where I was" bookkeeping would be painful. Iteration shines when the process is a simple march forward and you care about raw speed or memory, because loops don't pay the cost of stacking up function calls.

A helpful reframe, though one to state carefully: **the general way to turn recursion into iteration is to manage the "remembering" by hand with an explicit stack.** When a `while` loop uses a counter and an accumulator, it's doing that bookkeeping manually for a *simple* (linear) recursion; for branching recursions like tree walks, the manual version needs an actual stack data structure. Not every loop is literally "recursion in disguise" — but every recursion can be *rewritten* as a loop this way, and seeing that makes both feel less mysterious. We'll make the connection concrete when we cover tail recursion in Part VII and explicit-stack conversion in Chapter 30.

Checkpoint — end of Part I. You should now be able to: state in one sentence what a recursive function promises to return; explain why the leap of faith is valid by analogy to induction; identify the base case, the smaller subproblem, and the combine step in any recursion; and decide whether a given problem is recursive at all.

Chapter 5b. The four forces behind every recursive algorithm

You now have the intuition (trust the smaller call) and, in the next part, a step-by-step recipe. But intuition and recipe are not yet a *design system* — a small, fixed set of questions that, answered in order, hand you the algorithm. This chapter gives you that system. It is the intellectual spine of the whole book, and from here on every worked example is organized around it.

Every recursive algorithm is determined by **four forces**. Get these four right and the code writes itself; get any one wrong and the algorithm is wrong.

The four forces 1. **The shape of the problem determines the recursive calls.** Is the input a sequence, a tree, a graph, a numeric range, a decision set? The shape tells you what "smaller" means and therefore which recursive calls you make. 2. **The goal determines how results are combined.** Are you deciding (yes/no), counting, optimizing, enumerating, or constructing? The goal tells you the combine operator: `or`, `+`, `min/max`, or concatenation/mutation. 3. **The state determines whether answers may be remembered.** What information completely defines the remaining problem? That is your function's parameters and, if subproblems repeat, your memo key. 4. **The progress measure guarantees termination.** What strictly decreases on every recursive call? That is what drives you to a base case and proves the recursion ends.

Two supporting questions turn the four forces into code:

- **What does each smaller call promise?** — the *contract* (Chapter 6).
- **What does the current call contribute?** — the *combine step*.

Here is the whole system as a table you can run down in under a minute for any problem:

Question	What it determines
What is the shape? Sequence, tree, graph, decision set, range?	Which smaller calls are made
What is the goal? Decide, count, optimize, enumerate, construct?	The combine operator: <code>or</code> , <code>+</code> , <code>min/max</code> , <code>concat/mutate</code>
What is the state? What information completely defines the remaining problem?	Function parameters and memo key
What strictly decreases?	Base cases and termination
What does each smaller call promise?	The contract
What does the current call contribute?	The combine step

Notice how the four forces absorb ideas you have already met. The "leap of faith" is the contract (force-adjacent).

"Choosing the right smaller" (Chapter 7) is force 1. The three styles (Chapter 10c) are downstream of force 2. The whole of complexity analysis (Part VIII) falls out of forces 3 and 4: the state count times the work per state gives the time, and the progress measure gives the depth. You are not learning a dozen unrelated tricks; you are learning to answer four questions.

The Recursive Design Canvas

To make the four forces a *habit*, fill in this canvas — on paper — *before* writing any code. Every worked algorithm in this book from here on is preceded by its canvas, and you should demand the same of yourself.

```
Problem:
State:
Contract — solve(state) means:
Base states:
Current call's own responsibility:
Smaller state(s):
What I trust each recursive call to return:
Combine rule:
Progress measure:
Repeated states?
Time:
Stack space:
```

The canvas is not bureaucracy. Each line is one of the four forces made concrete, and filling it turns code from something to memorize into a *translation of thought*. When you get stuck writing a recursion, you are almost always stuck on one specific canvas line — usually the contract or the state — and knowing which line is stuck tells you exactly what to think about.

A worked canvas: count nodes with exactly one child

Before any code, fill the canvas for "count the nodes of a binary tree that have exactly one (non-null) child."

```
Problem:      Count nodes with exactly one child in a binary tree.
State:        The current node.
Contract:     countOneChild(node) returns the number of nodes with
              exactly one child in the subtree rooted at `node`.
Base states:  node is None -> 0.
Current call's own responsibility:
              1 if exactly one of node.left, node.right is non-null, else 0.
Smaller states:
              node.left and node.right (each a proper subtree).
What I trust each recursive call to return:
              the count for that subtree (by the contract).
Combine rule:  current_contribution + answer(left) + answer(right).
Progress:     each call moves to a proper subtree; depth is finite.
Repeated states? No (each node visited once) -> no memoization.
Time:         O(n)   Stack space: O(height).
```

Only *now*, with every force pinned down, does the code appear — and it is a direct transcription of the canvas:

```
def count_one_child(node):
    if node is None:                # base state
        return 0
    exactly_one = (node.left is None) != (node.right is None) # current contribution
    here = 1 if exactly_one else 0
    return here + count_one_child(node.left) + count_one_child(node.right) # combine
```

JavaScript:

```
function countOneChild(node) {
    if (node === null) return 0; // base state
    const exactlyOne = (node.left === null) !== (node.right === null); // current contribution
    const here = exactlyOne ? 1 : 0;
    return here + countOneChild(node.left) + countOneChild(node.right); // combine
}
```

Read the code against the canvas line by line. The base state is the `if node is None` guard; the current contribution is `here`; the smaller states are the two child calls; the combine rule is the `+`. The canvas is the algorithm; the code is

just its spelling in a language.

From now on, whenever you meet a new problem, reach for the canvas first. Fill the four forces, and the recursion will already be written in your head before your fingers touch the keyboard.

Part II — Writing the Logic

Learning objectives. Turn intuition into correct code with a repeatable five-step recipe; develop one example (`sum_list`) through every stage; choose how to shrink the input; carry state with accumulators and helpers; and avoid the three classic beginner bugs.

Chapter 6. A repeatable recipe

When you sit down to write a recursive function, don't stare at the blank screen. Follow this five-step recipe every single time. It works for the simplest and the hairiest problems alike.

Step 1 — Define the function precisely. Write one sentence stating exactly what the function returns for a *general* input. Not how it works — *what it returns*. This "contract" is the anchor for everything else. Example: "`sum_list(xs)` returns the sum of all numbers in the list `xs`."

Step 2 — Find the base case. Ask: "What is the smallest, most trivial input, and what's the answer for it?" For a list, that's usually the empty list. For a number, usually 0 or 1. For a tree, usually a null/empty node. Return the answer directly, no recursion.

Step 3 — Assume the recursive call works (the leap of faith). Pretend the function is already finished and correct for any *smaller* input. Do not think about how. Just say, "I trust `sum_list(rest)` gives me the sum of the rest."

Step 4 — Do one step's worth of work and combine. Take the general input, peel off one piece, apply the trusted recursive result to the remainder, and combine. "The sum is the first element plus `sum_list(the rest)`."

Step 5 — Check that you shrink toward the base case. Confirm every recursive call is on strictly smaller input and that repeated shrinking always reaches the base case. If yes, you're done.

Applied to `sum_list`:

```
def sum_list(xs):
    # Step 1 contract: returns the sum of all numbers in xs
    if not xs:           # Step 2: base case, empty list
        return 0
    first = xs[0]
    rest = xs[1:]        # Step 4a: peel one piece; rest is smaller (Step 5 ✓)
    return first + sum_list(rest)  # Step 3 + 4b: trust + combine
```

JavaScript:

```
function sumList(xs) {
    // Step 1 contract: returns the sum of all numbers in xs
    if (xs.length === 0) return 0;           // Step 2: base case, empty list
    const first = xs[0];
    const rest = xs.slice(1);                // Step 4a: peel one piece; rest is smaller
    return first + sumList(rest);            // Step 3 + 4b: trust + combine
}
```

Read that function against the recipe until each line maps to a step. Once this mapping feels automatic, you can write most recursions without conscious effort.

Chapter 6b. One example, ten views: `sum_list` from start to finish

Before collecting more patterns, let's slow down and look at a *single* problem from every angle. We'll use the simplest possible recursion — summing a list of numbers — and develop it through ten stages. The problem never changes; only what we notice about it does. If you understand this one example completely, most of the rest of the book is variation on it.

Stage 1 — State the contract. Write one sentence about what the function *returns*, for any input, before thinking about how.

`sum_list(xs)` returns the sum of all the numbers in the list `xs`. For the empty list, that sum is 0.

Stage 2 — Find the smaller problem. A list is a first element followed by a *shorter* list. So "the sum of `xs`" contains, inside it, "the sum of the rest of `xs`" — a smaller instance of the exact same problem. That is the recursive structure, spotted.

Stage 3 — Write the base case. The smallest input is the empty list, and its sum is 0. No recursion needed:

```
if not xs:
    return 0
```

JavaScript:

```
if (xs.length === 0) {
    return 0;
}
```

Stage 4 — Trust the recursive answer (the leap of faith). Assume a perfect `sum_list` already exists for the *shorter* list `xs[1:]`. Don't ask how. Just believe that `sum_list(xs[1:])` hands you the sum of everything after the first element.

Stage 5 — Combine. The sum of the whole list is the first element plus the sum of the rest:

```
def sum_list(xs):
    if not xs:                # base
        return 0
    return xs[0] + sum_list(xs[1:])  # first + (trusted) sum of the rest
```

JavaScript:

```
function sumList(xs) {
    if (xs.length === 0) return 0;
    return xs[0] + sumList(xs.slice(1));
}
```

That's a correct, complete recursion. Everything below is about *understanding* it, not fixing it.

Stage 6 — Draw the stack (the downward phase). Call `sum_list([3, 5, 2])`. Each call, unable to finish, parks itself and calls a smaller version. Frames pile up:

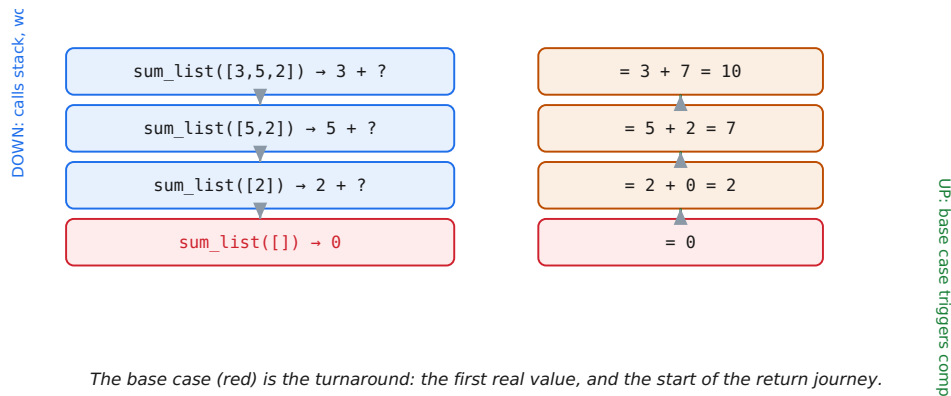
```
sum_list([3,5,2])  waits on  3 + sum_list([5,2])
  sum_list([5,2])  waits on  5 + sum_list([2])
    sum_list([2])  waits on  2 + sum_list([])
      sum_list([]) returns 0  ← base case: the FIRST real value
```

Notice: until the base case, *nothing* has an actual number. Every frame is holding an unfinished expression like "3 + (whatever comes back)". The base case produces the first concrete value.

Stage 7 — Watch the return (the upward phase). Now the values flow back up, each frame completing its parked addition:

```
sum_list([]) = 0
sum_list([2]) = 2 + 0 = 2
sum_list([5,2]) = 5 + 2 = 7
sum_list([3,5,2]) = 3 + 7 = 10
```

The recursion has two phases: a **downward** phase where calls stack up and defer their work, and an **upward** phase where the base case kicks off a chain of completions. Chapter 10b makes this down/up distinction its whole subject, because it is one of the most useful intuitions in all of recursion.



The base case (red) is the turnaround: the first real value, and the start of the return journey.

Downward phase (calls stack, work deferred) and upward phase (base case triggers completions) for `sum_list([3,5,2])`.

Stage 8 — Analyze time and space. There are n calls, each doing $O(1)$ real work (one addition), so **time is $O(n)$** — if we ignore the slicing. The stack reaches depth n before the base case returns, so **stack space is $O(n)$** . Its recurrence is $T(n) = T(n-1) + O(1)$, which unrolls to $O(n)$ (Chapter 32).

Stage 9 — Remove the hidden cost. The catch: `xs[1:]` copies the rest of the list every call, so the innocent version above is secretly $O(n^2)$. Fix it by passing an **index** instead of slicing — same idea, no copying:

```
def sum_list(xs):
    """Precondition: xs is a list of numbers. Returns their sum.  $O(n)$  time."""
    def helper(i):
        if i == len(xs):          # base: past the end
            return 0
        return xs[i] + helper(i + 1)  # first-from-i + sum of the rest
    return helper(0)
```

JavaScript:

```
function sumList(xs) {
    // Precondition: xs is an array of numbers.  $O(n)$  time — index, no slicing.
    function helper(i) {
        if (i === xs.length) return 0;          // base: past the end
        return xs[i] + helper(i + 1);          // first-from-i + sum of the rest
    }
    return helper(0);
}
```

Stage 10 — Convert to a loop. The downward phase carried no information except position, and the upward phase just added things up. That means we can do the same work with an accumulator and no stack at all:

```
def sum_list(xs):
    total = 0
    for x in xs:          # this loop IS the upward phase, done by hand
        total += x
    return total
```

JavaScript:

```
function sumList(xs) {
    let total = 0;
    for (const x of xs) { // this loop IS the upward phase, done by hand
        total += x;
    }
    return total;
}
```

The loop and the recursion compute the identical thing; the loop simply keeps its "running total" in a variable instead of in parked stack frames. Seeing one problem wear all ten of these costumes — contract, smaller problem, base case, trust, combine, stack, unwinding, complexity, de-slicing, and de-recursing — is worth more than seeing ten different problems once each. Keep `sum_list` in mind as the reference; we'll point back to it repeatedly.

Chapter 7. Choosing the right "smaller"

The art of recursion is choosing *how* to make the problem smaller. There is often more than one way, and the choice shapes the whole solution. Common ways to shrink:

- **Peel one from the front (or back).** `xs[1:]` or `xs[:-1]`. Turns a size- n problem into size $n-1$. This is *linear recursion*.
- **Decrement a number.** `n → n-1`, or `n → n-2`, etc. Classic for counting and math.
- **Halve the range.** `[lo, hi] → [lo, mid]` or `[mid+1, hi]`. Turns n into $n/2$. This is *divide-and-conquer* and gives fast (logarithmic or $n \log n$) algorithms.
- **Descend into substructure.** For a tree, recurse on `node.left` and `node.right`; for a filesystem, recurse into each subdirectory. The structure defines "smaller."
- **Remove one choice from the decision set.** "I've decided where queen 1 goes; now solve for queens 2..n." Smaller = fewer decisions left.

When a problem feels stuck, try a *different* notion of smaller. Many problems that look hard with "peel one" become easy with "halve," and vice versa.

Chapter 8. Passing state: accumulators and helper functions

Sometimes the natural contract from Step 1 doesn't carry enough information to combine results, or you want to build the answer as you descend rather than on the way back up. The tool for this is an **accumulator**: an extra parameter that carries partial results down the recursion.

Compare two ways to reverse a list. First, the pure "combine on the way up" style:

```
def reverse(xs):
    if not xs:
        return []
    return reverse(xs[1:]) + [xs[0]]    # put first element last
```

JavaScript:

```
function reverse(xs) {
    if (xs.length === 0) return [];
    return [...reverse(xs.slice(1)), xs[0]];    // put first element last
}
```

Now the accumulator style, which builds the answer going *down*:

```
def reverse(xs, acc=None):
    if acc is None:
        acc = []
    if not xs:
        return acc    # base case returns the built-up answer
    return reverse(xs[1:], [xs[0]] + acc) # carry partial result downward
```

JavaScript:

```
function reverse(xs, acc = []) {
    if (xs.length === 0) return acc;    // base case returns the built-up answer
    return reverse(xs.slice(1), [xs[0], ...acc]); // carry partial result downward
}
```

A caution before you copy these. Both versions above are written for clarity, not efficiency. In Python, `xs[1:]` copies the rest of the list ($O(n)$ work) and `[xs[0]] + acc` copies the accumulator — done at every one of the n levels, that quietly makes these $O(n^2)$. Chapter 34 covers this trap in full. The point here is the idea of an accumulator, so read the code as a diagram. When you want the same logic without the hidden cost, carry an **index** instead of slicing:

```
def reverse(xs):
    """Precondition: xs is a list. Returns a new reversed list. O(n) time."""
    def helper(i, acc):
        if i < 0:
            return acc
        acc.append(xs[i])          # append is O(1); no copying
        return helper(i - 1, acc)
    return helper(len(xs) - 1, [])
```

JavaScript:

```
function reverse(xs) {
    // Precondition: xs is an array. Returns a new reversed array. O(n) time.
    function helper(i, acc) {
        if (i < 0) return acc;
        acc.push(xs[i]);          // push is O(1); no copying
        return helper(i - 1, acc);
    }
    return helper(xs.length - 1, []);
}
```

This index-based shape — pass a position, never slice — is the efficient template you should reach for in real Python. We'll use it repeatedly from here on, and every time you see `xs[1:]` in this book, read it as "conceptually the rest of the list" and remember the index version exists.

The accumulator pattern has two big virtues. It often turns the recursion **tail-recursive** (Part VII), and it lets you thread along running totals, paths taken so far, depth, visited-sets, and other bookkeeping.

A closely related tool is the **helper function**. Your public function has a clean contract, but the recursion needs extra parameters (an accumulator, an index, bounds). So you define an inner helper that does the real recursion, and the outer function just kicks it off:

```
def count_down_to_target(n, target):
    """Precondition: target <= n (both integers). Returns the count of steps."""
    def helper(current, steps):
        if current == target:
            return steps
        return helper(current - 1, steps + 1)
    return helper(n, 0)
```

JavaScript:

```
function countDownToTarget(n, target) {
    // Precondition: target <= n (both integers). Returns the count of steps.
    function helper(current, steps) {
        if (current === target) return steps;
        return helper(current - 1, steps + 1);
    }
    return helper(n, 0);
}
```

This keeps the interface simple for callers while giving the recursion the room it needs. You will use helpers constantly in real code, especially for trees, graphs, and backtracking. (Note the precondition: if `target > n`, counting *down* never reaches it — a reminder that defining a recursion starts with stating precisely what inputs it's meant for.)

Chapter 9. The three most common beginner bugs (and their fixes)

Bug 1 — Missing or unreachable base case → infinite recursion. The function never stops and you get a `RecursionError` (CPython commonly starts with a recursion limit around 1000 — check yours with `sys.getrecursionlimit()`; you can raise it with `sys.setrecursionlimit`, but that treats the symptom, not the cause). Fix: make sure at least one base case exists and that every recursive branch moves toward it.

Bug 2 — Recursing on input that isn't actually smaller. For example, calling `f(n)` instead of `f(n-1)`, or `f(xs)` instead of `f(xs[1:])`. The input never shrinks, so the base case is never reached. Fix: verify with your own eyes that each recursive call has strictly smaller input.

Bug 3 — Forgetting to *return* the recursive result. In many languages you must return (or otherwise use) what the recursive call produces. Writing `sum_list(rest)` on its own line and then returning something else silently throws the recursion's work away. Fix: the recursive call's result must flow into your combine step and out through `return`.

A fourth, subtler issue: **doing the same subproblem many times.** Naive `fib(n) = fib(n-1) + fib(n-2)` recomputes the same values exponentially often. That's not a correctness bug but a performance disaster, and Part VI shows the fix (memoization).

Checkpoint — end of Part II. You should now be able to: apply the five-step recipe from a blank page; take one problem (`sum_list`) through contract, base case, trust, and combine; pick a sensible way to shrink the input; carry state with an accumulator or helper; and name the three classic beginner bugs.

Part III — How Recursion Runs

Learning objectives. Picture the call stack pushing and popping frames; distinguish work done on the way down from work done on the way up; tell the three recursion styles apart; trace recursion on paper; and know what to do when it goes too deep.

Chapter 10. The call stack, without the mystery

When you write recursion using the leap of faith, you deliberately *don't* trace it. But to debug it, and to understand its cost, you need a clear picture of what the machine actually does. That picture is the **call stack**.

Every time a function is called, the computer creates a small workspace called a **stack frame**. It holds that call's parameters, its local variables, and a bookmark for where to resume. Frames pile up like a stack of plates: the most recent call sits on top. When a call finishes, its frame is removed (popped) and control returns to the frame beneath it, resuming at the bookmark.

Recursion just means many frames of the *same* function stacked up, each with its own copy of the parameters. Consider `factorial(3)`:

```
def factorial(n):
    if n == 0:
        return 1
    return n * factorial(n - 1)
```

JavaScript:

```
function factorial(n) {
    if (n === 0) return 1;
    return n * factorial(n - 1);
}
```

The stack grows as calls are made, then shrinks as they return:

factorial(3)	push frame n=3
→ 3 * factorial(2)	push frame n=2
→ 2 * factorial(1)	push frame n=1
→ 1 * factorial(0)	push frame n=0
→ returns 1	pop n=0
→ 1 * 1 = 1	pop n=1
→ 2 * 1 = 2	pop n=2
→ 3 * 2 = 6	pop n=3
returns 6	

Two lasting insights come from this picture. First, **the base case is what starts the returning** — until you hit it, nothing has an actual value; everything is "waiting" for the deeper call. Second, **the maximum stack depth is your recursion's memory cost**. A recursion that goes n deep uses memory proportional to n , even if it returns a single number. This is a trade-off against iteration: a *simple linear* recursion can often be rewritten as a constant-space loop, which is the win. But not every recursion converts to constant space — branching or structural recursion (tree and graph walks) still needs an explicit stack whose size is proportional to the traversal depth, so the iterative version saves the function-call overhead but not the fundamental $\Theta(\text{depth})$ memory. We'll see this concretely in Chapter 30.

Chapter 10b. Work going down vs. work coming back up

Here is a distinction that quietly explains preorder vs. postorder traversal, why reversing works, and why statement order inside a recursive function matters enormously. Consider two nearly identical functions:

```
def print_down(n):
    if n == 0:
        return
    print(n)          # (A) work happens BEFORE the recursive call
    print_down(n - 1)

def print_up(n):
    if n == 0:
        return
    print_up(n - 1)
    print(n)          # (B) work happens AFTER the recursive call returns
```

JavaScript:

```
function printDown(n) {
    if (n === 0) return;
    console.log(n);      // (A) work happens BEFORE the recursive call
    printDown(n - 1);
}

function printUp(n) {
    if (n === 0) return;
    printUp(n - 1);
    console.log(n);      // (B) work happens AFTER the recursive call returns
}
```

`print_down(3)` prints `3 2 1` — each call does its work *on the way down*, before handing off. `print_up(3)` prints `1 2 3` — each call defers its work until the deeper calls have finished, so the work happens *on the way back up*, in reverse order of the calls.

Picture the two phases on a single timeline for `f(3)`:

DOWNWARD (calls stacking)	UPWARD (calls returning)
f(3): [work here = "on the way down"]	[work here = "on the way up"]
f(2): [down]	[up]
f(1): [down]	[up]
f(0): base case	turnaround point

Everything placed *before* the recursive call executes in call order (3, 2, 1). Everything placed *after* the recursive call executes in the reverse order the calls were made (1, 2, 3), because those statements were waiting, parked in their frames, for the deeper calls to come back.

This one idea unlocks a surprising amount:

- **Preorder vs. postorder traversal** (Chapter 15) is *exactly* this: "visit the node" placed before the child calls gives preorder; placed after gives postorder. Same code, one line moved.
- **Reversing** anything — a list, a string, a linked list — is naturally "work on the way up," because the last element reached is the first one processed on return.
- **Why some recursions must return a value:** if a call needs a result *from* its deeper call to do its own work (like `sum_list`), that work is inherently upward-phase and the deeper call must hand something back. If the work is purely downward (like `print_down`), the calls can be `void`/return nothing.
- **Why the call stack exists at all:** it's the mechanism that *remembers* the parked upward-phase work of every unfinished call.

Whenever you write a recursive function, ask deliberately: *does this line belong on the way down or on the way up?* Placing it on the wrong side is one of the most common recursion bugs, and it never triggers an error — it just silently produces the wrong order.

Chapter 10c. Three styles of recursion (and not mixing them up)

Recursive functions come in three distinct styles. Beginners get confused because textbooks (and earlier chapters here) slide between them without saying so. Naming them prevents that confusion.

Style 1 — Return-value recursion. The recursive call *returns* an answer, and the caller *combines* it into its own answer. This is the purest style: no shared state, no printing, just values flowing back up. `sum_list`, `factorial`,

`height`, `lcs` are all this style.

```
def sum_list(xs, i=0):
    if i == len(xs): return 0
    return xs[i] + sum_list(xs, i + 1) # combine the returned value
```

JavaScript:

```
function sumList(xs, i = 0) {
    if (i === xs.length) return 0;
    return xs[i] + sumList(xs, i + 1); // combine the returned value
}
```

Tell: the recursive call appears inside an expression whose value is returned.

Style 2 — Accumulator / helper recursion. State is carried *downward* as an extra parameter, built up as you descend, and read out at the base case. Nothing is combined on the way up; the answer is already assembled by the time you hit bottom.

```
def sum_list(xs):
    def helper(i, running_total): # running_total flows DOWN
        if i == len(xs):
            return running_total # base just hands back what was built
        return helper(i + 1, running_total + xs[i])
    return helper(0, 0)
```

JavaScript:

```
function sumList(xs) {
    function helper(i, runningTotal) { // runningTotal flows DOWN
        if (i === xs.length) return runningTotal; // base just hands back what was built
        return helper(i + 1, runningTotal + xs[i]);
    }
    return helper(0, 0);
}
```

Tell: an extra parameter accumulates the answer; the base case returns that parameter, not a fresh value.

Style 3 — Side-effect / backtracking recursion. The function doesn't return the answer at all; it *produces* results by printing, appending to a shared list, or mutating a structure — often choosing, exploring, then undoing. The return value is usually ignored.

```
def subsets(xs, i=0, current=None, out=None):
    if current is None: current, out = [], []
    if i == len(xs):
        out.append(current[:]); return out # record via side effect
    current.append(xs[i]); subsets(xs, i+1, current, out) # choose, explore
    current.pop(); subsets(xs, i+1, current, out) # undo, explore
    return out
```

JavaScript:

```
function subsets(xs, i = 0, current = [], out = []) {
    if (i === xs.length) {
        out.push([...current]); return out; // record via side effect (a COPY)
    }
    current.push(xs[i]); subsets(xs, i + 1, current, out); // choose, explore
    current.pop(); subsets(xs, i + 1, current, out); // undo, explore
    return out;
}
```

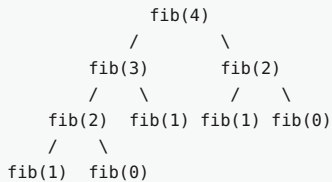
Tell: results accumulate in a mutable structure or via output; you see `.append()`, `print()`, `make/undo` pairs, and the recursive calls are made for their *effects*, not their returned values.

Why this matters: the three styles fail in different ways and are debugged differently. Return-value bugs are usually a wrong combine step. Accumulator bugs are usually a wrong initial value or reading the accumulator at the wrong moment. Side-effect bugs are usually a forgotten *undo* (in backtracking) or appending a reference instead of a copy (`out.append(current)` vs `out.append(current[:])` — the classic aliasing bug). When your recursion misbehaves, first

ask *which style is this?* — the answer tells you where to look. Much of Parts IV–VII is return-value and accumulator style; the backtracking of Chapter 19 is squarely Style 3.

Chapter 11. Tracing recursion on paper (a skill worth practicing)

When a recursion misbehaves, trace it by hand using the "call tree." Write the top call, draw an arrow to each call it makes, and so on, then fill in returned values from the leaves upward. For branching recursion like Fibonacci, this tree makes the repeated work visually obvious:



Count the leaves and you feel, viscerally, why naive Fibonacci is slow: `fib(2)` is computed twice, and the duplication explodes as n grows. Drawing three or four of these trees builds an instinct for which recursions are cheap and which need memoization.

A lighter-weight tracing trick during development is to print with indentation by depth:

```
def fib(n, depth=0):
    print(" " * depth + f"fib({n})")
    if n < 2:
        return n
    return fib(n - 1, depth + 1) + fib(n - 2, depth + 1)
```

JavaScript:

```
function fib(n, depth = 0) {
    console.log(" ".repeat(depth) + `fib(${n})`);
    if (n < 2) return n;
    return fib(n - 1, depth + 1) + fib(n - 2, depth + 1);
}
```

The indented output is the call tree, printed live. This is often the fastest way to see what your recursion is actually doing versus what you thought it was doing.

Chapter 12. When recursion goes too deep

Because each call consumes a stack frame, deep recursion can exhaust the stack — a **stack overflow**, surfacing in Python as `RecursionError: maximum recursion depth exceeded`. This bites in three situations: a genuine bug (no reachable base case), a correct recursion on very large input (walking a linked list of a million nodes), or naturally deep structures (a deeply unbalanced tree).

Your options, in order of preference: (1) fix the base case if it's a bug; (2) rewrite as iteration if the recursion is simply linear and deep; (3) convert to an explicit-stack iterative version for tree/graph walks (Chapter 30); or (4), as a last resort, raise the limit with `sys.setrecursionlimit(bigger)` — knowing that pushing it too high risks a hard crash rather than a catchable error, because you can outrun the underlying C stack. Prefer restructuring over raising the limit.

Checkpoint — end of Part III. You should now be able to: describe how frames are pushed and popped on the call stack; tell whether a line runs on the way down or the way up and place it correctly; distinguish return-value, accumulator, and side-effect styles; trace a small recursion on paper; and know your options when recursion goes too deep.

Part IV — Linear & Structural Recursion

Learning objectives. Apply the standard recursion templates to numbers, lists, strings, trees, and arbitrarily nested data — the shapes you'll meet most often.

This part covers the workhorse patterns you'll use most: recursion over numbers, over lists/strings, and over the self-similar structures (trees, nested data, filesystems) where recursion truly shines.

Chapter 13. Recursion over numbers

Numbers are the gentlest place to start, because "smaller" is obvious: subtract, or divide.

Factorial. $n! = n \times (n-1)!$, with $0! = 1$.

```
def factorial(n):
    """Precondition: n is a non-negative integer. (For n < 0 the recursion
    would skip past the n == 0 base case and never terminate.)"""
    if n == 0:          # base
        return 1
    return n * factorial(n - 1)
```

JavaScript:

```
function factorial(n) {
    // Precondition: n is a non-negative integer. (For n < 0 the recursion
    // would skip past the n === 0 base case and never terminate.)
    if (n === 0) return 1;           // base
    return n * factorial(n - 1);
}
```

Sum of 1..n. The sum up to n is n plus the sum up to $n-1$.

```
def sum_to(n):
    """Precondition: n is a non-negative integer."""
    if n == 0:
        return 0
    return n + sum_to(n - 1)
```

JavaScript:

```
function sumTo(n) {
    // Precondition: n is a non-negative integer.
    if (n === 0) return 0;
    return n + sumTo(n - 1);
}
```

Power (fast exponentiation). Here "smaller" via *halving* the exponent gives a logarithmic-time algorithm — a first taste of divide-and-conquer applied to a number.

```
def power(base, exp):
    """Precondition: exp is a non-negative integer.
    (Negative exponents need a 1/result step and a different base case.)"""
    if exp == 0:
        return 1
    half = power(base, exp // 2)
    if exp % 2 == 0:
        return half * half
    return half * half * base
```

JavaScript:

```
function power(base, exp) {
  // Precondition: exp is a non-negative integer.
  // (Negative exponents need a 1/result step and a different base case.)
  if (exp === 0) return 1;
  const half = power(base, Math.floor(exp / 2));
  if (exp % 2 === 0) return half * half;
  return half * half * base;
}
```

Greatest common divisor (Euclid). The classic: $\text{gcd}(a, b) = \text{gcd}(b, a \bmod b)$. "Smaller" is the shrinking remainder.

```
def gcd(a, b):
    """Precondition: a, b are non-negative integers, not both zero."""
    if b == 0:
        return a
    return gcd(b, a % b)
```

JavaScript:

```
function gcd(a, b) {
  // Precondition: a, b are non-negative integers, not both zero.
  if (b === 0) return a;
  return gcd(b, a % b);
}
```

Digit problems. Many number puzzles recurse by stripping the last digit with `n % 10` and `n // 10`. For example, summing a number's digits:

```
def digit_sum(n):
    """Precondition: n is a non-negative integer.
    (Call digit_sum(abs(n)) if a negative input is possible.)"""
    if n < 10:
        return n
    return n % 10 + digit_sum(n // 10)
```

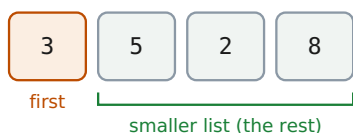
JavaScript:

```
function digitSum(n) {
  // Precondition: n is a non-negative integer.
  // (Call digitSum(Math.abs(n)) if a negative input is possible.)
  if (n < 10) return n;
  return (n % 10) + digitSum(Math.floor(n / 10));
}
```

Stating the **precondition** — the set of inputs the function is defined for — is not busywork. Every one of these functions misbehaves outside its assumed domain (negative `n` in `factorial` recurses forever; a negative `exp` in `power` never hits `exp == 0`). Designing a recursion *begins* with pinning down exactly what inputs it means to handle, because that domain is what guarantees your "smaller" input actually marches to the base case.

Chapter 14. Recursion over lists and strings

A list is secretly a recursive structure: it's either empty, or it's a *first element* followed by a *smaller list*.



Every list is one element in front of a smaller list — the recursive shape.

A list is a first element in front of a smaller list.

That single observation generates a huge family of functions. The universal template:

```
def process(xs):
    if not xs:                # base: empty
        return base_value
    first, rest = xs[0], xs[1:] # head + smaller tail
    return combine(first, process(rest))
```

JavaScript:

```
function process(xs) {
    if (xs.length === 0) return baseValue; // base: empty
    const first = xs[0], rest = xs.slice(1); // head + smaller tail
    return combine(first, process(rest));
}
```

The `xs[1:]` here is the *conceptual* form — clear to read but $O(n^2)$ in Python because slicing copies the tail at every level (Chapter 34). Every list template below is written this way for readability; in real code, pass an index instead (`process(xs, i+1)`) as shown in Chapter 6b, Stage 9. Read the slicing versions as diagrams of the idea, and reach for the index form when you actually run the code.

Fill in `base_value` and `combine`, and you get:

```
def length(xs):
    if not xs: return 0
    return 1 + length(xs[1:]) # combine = +1

def maximum(xs):
    # Precondition: xs is non-empty (there is no maximum of nothing).
    if len(xs) == 1: return xs[0] # base: single element
    return max(xs[0], maximum(xs[1:])) # combine = max

def contains(xs, target):
    if not xs: return False
    return xs[0] == target or contains(xs[1:], target)

def count(xs, target):
    if not xs: return 0
    return (1 if xs[0] == target else 0) + count(xs[1:], target)
```

JavaScript:

```
function length(xs) {
    if (xs.length === 0) return 0;
    return 1 + length(xs.slice(1)); // combine = +1
}

function maximum(xs) {
    // Precondition: xs is non-empty (there is no maximum of nothing).
    if (xs.length === 1) return xs[0]; // base: single element
    return Math.max(xs[0], maximum(xs.slice(1))); // combine = max
}

function contains(xs, target) {
    if (xs.length === 0) return false;
    return xs[0] === target || contains(xs.slice(1), target);
}

function count(xs, target) {
    if (xs.length === 0) return 0;
    return (xs[0] === target ? 1 : 0) + count(xs.slice(1), target);
}
```

Strings behave identically — they're sequences of characters. Checking a **palindrome** is a clean example of shrinking from *both* ends:

```
def is_palindrome(s):
    if len(s) <= 1:           # empty or single char is a palindrome
        return True
    if s[0] != s[-1]:         # ends must match
        return False
    return is_palindrome(s[1:-1]) # shrink from both sides
```

JavaScript:

```
function isPalindrome(s) {
    if (s.length <= 1) return true;           // empty or single char is a palindrome
    if (s[0] !== s[s.length - 1]) return false; // ends must match
    return isPalindrome(s.slice(1, -1));       // shrink from both sides
}
```

Note the two base cases and the "peel from both ends" shrink — a shape you'll reuse for two-pointer problems.

Active practice (do before reading on; answers in Chapter 48's answer key, §14). 1. Predict. For `count(xs, target)`, what does the call `count([2,2,3], 2)` trust `count([2,3], 2)` to return, and what does it then return? 2. Complete. Fill the blank so this returns the number of elements greater than `t`: `def gt(xs, t): return 0 if not xs else ( ____ ) + gt(xs[1:], t)`. 3. Diagnose. `def prod(xs): return xs[0] * prod(xs[1:])` gives an error on some input — which, and what one line fixes it? 4. Derive (blank page). Write a recursion returning the **second-largest distinct** value of a list. Then, two chapters from now, try the transfer task: recursively **remove adjacent duplicates** from a list.

Chapter 14b. Linked lists: recursion without slicing

A linked list is recursion's cleanest real-world data structure, and it fixes a bad habit the slicing examples can create. A linked list is *either* empty (`None`) or a node holding a value and a reference to a smaller linked list — its `next`. That is a recursive definition with no copying: "the rest of the list" is just `node.next`, obtained in $O(1)$, not `xs[1:]`, which copies in $O(n)$.

```
class Node:
    def __init__(self, value, next=None):
        self.value = value
        self.next = next
```

JavaScript:

```
class Node {
    constructor(value, next = null) {
        this.value = value;
        this.next = next;
    }
}
```

Length — the empty list has length 0; otherwise it's 1 plus the length of the rest:

```
def length(node):
    if node is None:           # base: empty list
        return 0
    return 1 + length(node.next) # 1 + length of the rest (no copy!)
```

JavaScript:

```
function length(node) {
    if (node === null) return 0;           // base: empty list
    return 1 + length(node.next);         // 1 + length of the rest (no copy!)
}
```

Search — is a value present?

```
def contains(node, target):
    if node is None:
        return False
    return node.value == target or contains(node.next, target)
```

JavaScript:

```
function contains(node, target) {
    if (node === null) return false;
    return node.value === target || contains(node.next, target);
}
```

Recursive reversal — this one repays careful thought, because it's the purest example of *work on the way up* (Chapter 10b). You recurse to the end first, then rewire pointers as the calls return:

```
def reverse(node):
    """Returns the new head of the reversed list."""
    if node is None or node.next is None:
        return node          # base: empty or single node is its own reverse
    new_head = reverse(node.next) # reverse the rest FIRST (go all the way down)
    node.next.next = node        # on the way UP: make the next node point back to me
    node.next = None            # and sever my old forward link
    return new_head            # the deepest node bubbles up as the new head
```

JavaScript:

```
function reverse(node) {
    // Returns the new head of the reversed list.
    if (node === null || node.next === null) {
        return node;          // base: empty or single node
    }
    const newHead = reverse(node.next); // reverse the rest FIRST
    node.next.next = node;        // on the way UP: point the next node back to me
    node.next = null;            // sever my old forward link
    return newHead;            // the deepest node bubbles up as the new head
}
```

Trace it on **1→2→3**: you descend to 3 (the new head), then on the way back up **3→2** is wired, then **2→1**, yielding **3→2→1**. The rewiring is *upward-phase* work — it can only happen after the deeper call has returned, which is exactly why the recursive-call line comes before the rewiring lines.

Merging two sorted lists — pick the smaller head, then merge the rest:

```
def merge(a, b):
    if a is None: return b          # base: one list exhausted
    if b is None: return a
    if a.value <= b.value:
        a.next = merge(a.next, b)   # a's head is smaller; merge the rest after it
        return a
    else:
        b.next = merge(a, b.next)
        return b
```

JavaScript:

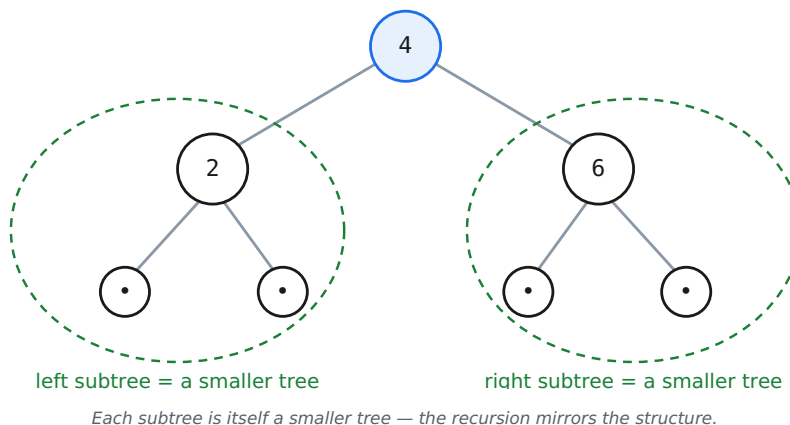
```
function merge(a, b) {
    if (a === null) return b;          // base: one list exhausted
    if (b === null) return a;
    if (a.value <= b.value) {
        a.next = merge(a.next, b);     // a's head is smaller; merge the rest after it
        return a;
    } else {
        b.next = merge(a, b.next);
        return b;
    }
}
```

Two things to take away. First, **linked lists show recursion at its most natural** — the structure is recursive, and the code mirrors it with no artificial copying. Second, reversal cleanly separates *returning a value* (the new head

flows up) from *rewiring on the upward phase* (the pointer surgery happens as calls return). If the slicing examples earlier made recursion feel expensive, linked lists are the antidote: here "the rest of the structure" is free.

Chapter 15. Trees: recursion's natural home

A tree is *the* self-similar structure: each child is itself the root of a smaller tree.



This is why tree code and recursion fit so perfectly — the code's shape mirrors the data's shape. Almost every tree function follows one template:

```
def process(node):
    if node is None:                # base: empty tree
        return base_value
    left_result = process(node.left) # trust: solves left subtree
    right_result = process(node.right) # trust: solves right subtree
    return combine(node.value, left_result, right_result)
```

JavaScript:

```
function process(node) {
    if (node === null) return baseValue; // base: empty tree
    const leftResult = process(node.left); // trust: solves left subtree
    const rightResult = process(node.right); // trust: solves right subtree
    return combine(node.value, leftResult, rightResult);
}
```

Concrete instances:

```
def height(node):
    if node is None: return 0
    return 1 + max(height(node.left), height(node.right))

def size(node):
    # count nodes
    if node is None: return 0
    return 1 + size(node.left) + size(node.right)

def tree_sum(node):
    if node is None: return 0
    return node.value + tree_sum(node.left) + tree_sum(node.right)

def contains(node, target):
    if node is None: return False
    return (node.value == target
            or contains(node.left, target)
            or contains(node.right, target))
```

JavaScript:


```

function height(node) {
  if (node === null) return 0;
  return 1 + Math.max(height(node.left), height(node.right));
}

function size(node) { // count nodes
  if (node === null) return 0;
  return 1 + size(node.left) + size(node.right);
}

function treeSum(node) {
  if (node === null) return 0;
  return node.value + treeSum(node.left) + treeSum(node.right);
}

function contains(node, target) {
  if (node === null) return false;
  return node.value === target
    || contains(node.left, target)
    || contains(node.right, target);
}

```

Traversals are just different choices of *when* you combine the current node relative to its children:

```

def inorder(node, out):
    if node is None: return
    inorder(node.left, out)      # left
    out.append(node.value)       # then self -> sorted, for a BST
    inorder(node.right, out)     # then right

def preorder(node, out):
    if node is None: return
    out.append(node.value)       # self first -> good for copying a tree
    preorder(node.left, out)
    preorder(node.right, out)

def postorder(node, out):
    if node is None: return
    postorder(node.left, out)
    postorder(node.right, out)
    out.append(node.value)       # self last -> good for deleting/sizing

```

JavaScript:

```

function inorder(node, out) {
  if (node === null) return;
  inorder(node.left, out); // left
  out.push(node.value);    // then self -> sorted, for a BST
  inorder(node.right, out); // then right
}

function preorder(node, out) {
  if (node === null) return;
  out.push(node.value);    // self first -> good for copying a tree
  preorder(node.left, out);
  preorder(node.right, out);
}

function postorder(node, out) {
  if (node === null) return;
  postorder(node.left, out);
  postorder(node.right, out);
  out.push(node.value);    // self last -> good for deleting/sizing
}

```

Once you see that *pre/in/post-order* differ only in the placement of one line, traversals stop being three things to memorize and become one idea with a knob.

Active practice (answers in Chapter 48's answer key, §15). 1. Predict. For `tree_sum`, what does the root call trust each child call to return? 2. Complete. Fill the base case so this counts leaves: `def leaves(nd): if nd is`

None: return ____; if nd.left is None and nd.right is None: return 1; return leaves(nd.left)+leaves(nd.right) .

3. Diagnose. A *height* function returns a value one larger than expected everywhere. Where is the off-by-one most likely to be? 4. Derive (blank page). Write *max_depth_value(node)* returning the value stored at the deepest node (leftmost if tied). What must the recursive call return for the parent to choose correctly? (Hint: revisit Chapter 16b.)

Chapter 15b. Trees vs. graphs: why one needs a visited set

Chapter 15 walked trees without ever worrying about revisiting a node. That safety is not automatic — it comes from a structural fact worth stating plainly:

A tree has no cycles. Every node has exactly one parent (the root has none), so following children can never lead you back to a node you've already seen. The recursion terminates naturally because each step moves strictly downward toward the leaves.

A graph may have cycles. A node can be reachable by multiple paths, and edges can loop back. If you recurse over a graph the way you recurse over a tree — just visit every neighbor — a cycle sends you around forever. The fix is a **visited set**: mark a node before exploring its neighbors, and skip any neighbor already marked.

```
def dfs(graph, node, visited=None):
    if visited is None: visited = set()
    if node in visited:      # the crucial guard trees don't need
        return
    visited.add(node)        # mark BEFORE exploring
    process(node)
    for neighbor in graph[node]:
        dfs(graph, neighbor, visited)
```

JavaScript:

```
function dfs(graph, node, visited = new Set()) {
    if (visited.has(node)) return;           // the crucial guard trees don't need
    visited.add(node);                       // mark BEFORE exploring
    process(node);
    for (const neighbor of graph[node]) {
        dfs(graph, neighbor, visited);
    }
}
```

So the rule is: **tree recursion follows children directly; graph recursion follows neighbors but must prevent revisiting states.** Whenever your "tree" can actually reach the same node two ways — a grid where you can move in all four directions, a state space where different move orders reach the same configuration — it's really a graph, and you need the visited set.

A subtlety in maze/backtracking: when to unmark. This trips people up. It depends on what you're computing:

- **For simple reachability** ("can I get from A to B at all?"), once you've visited a cell you never need to enter it again, so you leave it marked. Marked-forever is correct and faster.
- **For enumerating all distinct paths** (or finding a longest/shortest simple path), you must **unmark on the way back up** — *visited.remove(node)* after exploring — because a cell that's off-limits on *this* path might be essential on a *different* one. Leaving it marked would wrongly forbid other paths from using it.

```
def count_paths(graph, node, target, visited):
    if node == target: return 1
    visited.add(node)           # choose
    total = 0
    for nb in graph[node]:
        if nb not in visited:
            total += count_paths(graph, nb, target, visited) # explore
    visited.remove(node)        # UNDO — so other paths may reuse this node
    return total
```

JavaScript:

```
function countPaths(graph, node, target, visited) {
  if (node === target) return 1;
  visited.add(node); // choose
  let total = 0;
  for (const nb of graph[node]) {
    if (!visited.has(nb)) {
      total += countPaths(graph, nb, target, visited); // explore
    }
  }
  visited.delete(node); // UNDO – so other paths may reuse this node
  return total;
}
```

The mark-and-keep vs. mark-and-undo choice is the graph-world version of the choose/undo distinction from backtracking (Chapter 19). Ask: "could a node that's blocked on this path be needed on another valid answer?" If yes, unmark on return; if no, leave it marked.

Chapter 16. Nested and self-similar data

Real data nests: JSON objects contain arrays containing objects, HTML/DOM elements contain children that contain children, comment threads have replies that have replies. The recursion mirrors the nesting.

Flattening an arbitrarily nested list:

```
def flatten(xs):
    result = []
    for item in xs:
        if isinstance(item, list):
            result.extend(flatten(item)) # recurse into sub-list
        else:
            result.append(item) # base: a plain value
    return result

# flatten([1, [2, [3, 4], 5], [6]]) -> [1, 2, 3, 4, 5, 6]
```

JavaScript:

```
function flatten(xs) {
  const result = [];
  for (const item of xs) {
    if (Array.isArray(item)) {
      result.push(...flatten(item)); // recurse into sub-array
    } else {
      result.push(item); // base: a plain value
    }
  }
  return result;
}

// flatten([1, [2, [3, 4], 5], [6]]) -> [1, 2, 3, 4, 5, 6]
```

Summing all numbers in nested JSON-like data:

```
def deep_sum(data):
    if isinstance(data, (int, float)):
        return data # base
    if isinstance(data, dict):
        return sum(deep_sum(v) for v in data.values())
    if isinstance(data, list):
        return sum(deep_sum(v) for v in data)
    return 0 # strings, None, etc.
```

JavaScript:

```
function deepSum(data) {
  if (typeof data === "number") return data;    // base
  if (Array.isArray(data)) {
    return data.reduce((s, v) => s + deepSum(v), 0);
  }
  if (data && typeof data === "object") {        // plain object (dict-like)
    return Object.values(data).reduce((s, v) => s + deepSum(v), 0);
  }
  return 0;                                     // strings, null, etc.
}
```

Walking a filesystem (the everyday recursion): a directory's total size is the size of its own files plus the total size of each subdirectory — the definition is literally recursive.

```
import os
def total_size(path):
  if os.path.isfile(path):
    return os.path.getsize(path)                # base
  return sum(total_size(os.path.join(path, name))
             for name in os.listdir(path))      # recurse into entries
```

JavaScript:

```
const fs = require("fs");
const path = require("path");
function totalSize(p) {
  if (fs.statSync(p).isFile()) {
    return fs.statSync(p).size;                  // base
  }
  return fs.readdirSync(p)
    .reduce((sum, name) => sum + totalSize(path.join(p, name)), 0);
}
```

Watch out — a filesystem can secretly be a graph. The code above assumes the directory tree is really a tree. Real filesystems allow **symbolic links**, which can point back to an ancestor directory and create a cycle — turning the "tree" into a graph and sending this recursion into an infinite loop (or double-counting). Production code should either not follow symlinks (`os.walk(..., followlinks=False)` , the default) or track already-visited filesystem identities (device + inode via `os.stat`) in a visited set, exactly as Chapter 15b describes for graphs. This is the clearest everyday example of why the tree-vs-graph distinction matters.

The lesson of this chapter: **when the data nests, let the recursion nest with it.** You rarely need to think hard; you translate "a thing contains smaller things of the same kind" directly into "recurse on each smaller thing" — as long as the nesting is genuinely acyclic.

Chapter 16b. What should the recursive call return? (designing the contract)

You have learned to *trust* the recursive call. But trust presupposes you have decided *what* the call returns — and for any non-obvious algorithm, **choosing the right return value is the hard part, and it is where the algorithm is actually invented.** This is the skill that separates "I can follow a recursion" from "I can design one." The governing principle:

Make the recursive result carry exactly the information the parent needs — no less, and no more.

Too little information forces the parent to recompute (often turning an $O(n)$ algorithm into $O(n^2)$). Too much bloats the state and confuses the combine step. The craft is finding the *minimal sufficient* return value. Three classic problems show the idea, each solved twice: once with a weak contract, once with a strong one.

Example 1 — Is a binary tree height-balanced?

A tree is height-balanced if, at every node, the heights of the left and right subtrees differ by at most 1.

The weak contract: `is_balanced(node)` returns a boolean. It sounds right, but to *check* the balance condition at a node you need the two subtree *heights* — which this contract doesn't return. So you call a separate `height()` at every node:

```
def height(node):
    if node is None: return 0
    return 1 + max(height(node.left), height(node.right))

def is_balanced_slow(node):
    # WEAK contract -> O(n^2)
    if node is None: return True
    if abs(height(node.left) - height(node.right)) > 1:
        return False
    return is_balanced_slow(node.left) and is_balanced_slow(node.right)
```

JavaScript:

```
function height(node) {
    if (node === null) return 0;
    return 1 + Math.max(height(node.left), height(node.right));
}

function isBalancedSlow(node) {
    // WEAK contract -> O(n^2)
    if (node === null) return true;
    if (Math.abs(height(node.left) - height(node.right)) > 1) return false;
    return isBalancedSlow(node.left) && isBalancedSlow(node.right);
}
```

Why $O(n^2)$? `height` walks the whole subtree, and it's called at every node — so the top node's `height` scans all n nodes, its children scan $n/2$ each, and so on: the classic $T(n)=2T(n/2)+O(n)$ if balanced, but $\Theta(n^2)$ on a skewed tree because `height` and `is_balanced` both re-descend.

The strong contract: return the subtree's *height* if it is balanced, and a sentinel -1 if it is not. Now **one call delivers both facts the parent needs** — whether the subtree is balanced *and* its height — so the parent never re-descends:

```
def height_if_balanced(node):
    # STRONG contract -> O(n)
    """Returns the subtree's height if balanced, else -1 (a 'not balanced' sentinel)."""
    if node is None:
        return 0
    lh = height_if_balanced(node.left)
    if lh == -1: return -1
    rh = height_if_balanced(node.right)
    if rh == -1: return -1
    if abs(lh - rh) > 1: return -1
    return 1 + max(lh, rh)

def is_balanced(root):
    return height_if_balanced(root) != -1
```

JavaScript:

```
function heightIfBalanced(node) {
    // STRONG contract -> O(n)
    // Returns the subtree height if balanced, else -1 (a "not balanced" sentinel).
    if (node === null) return 0;
    const lh = heightIfBalanced(node.left);
    if (lh === -1) return -1;
    const rh = heightIfBalanced(node.right);
    if (rh === -1) return -1;
    if (Math.abs(lh - rh) > 1) return -1;
    return 1 + Math.max(lh, rh);
}

function isBalanced(root) {
    return heightIfBalanced(root) !== -1;
}
```

Each node is now visited exactly once — $O(n)$. The entire speedup came from *redesigning what the call returns*: bundling "am I balanced?" and "how tall am I?" into a single result so the parent gets both in one visit. The code barely changed; the *contract* did all the work.

Example 2 — Tree diameter (longest path between any two nodes)

The diameter is the number of edges on the longest path between any two nodes; that path may or may not pass through the root. The trap: a node needs to report *two different things*.

- To its **parent**, a node must report the longest *downward* path starting at itself (so the parent can extend it). Call this the node's **height/reach**.
- But the **answer** — the longest path anywhere — might be a path that *bends* at this node, going down-left then down-right, and never reaches the parent at all.

So the recursive call returns the *reach* (for the parent to use), while a separate running maximum captures the *best complete path found anywhere*. Recognizing that these are **two distinct quantities** is the whole insight:

```
def diameter(root):
    best = 0                                # longest complete path seen anywhere
    def reach(node):                        # returns longest DOWNWARD path length (edges)
        nonlocal best
        if node is None:
            return 0
        left = reach(node.left)             # trust: longest reach down the left
        right = reach(node.right)           # trust: longest reach down the right
        best = max(best, left + right)      # a path that BENDS here uses both sides
        return 1 + max(left, right)         # to the parent: extend the longer side
    reach(root)
    return best
```

JavaScript:

```
function diameter(root) {
    let best = 0;                          // longest complete path seen anywhere
    function reach(node) {                 // returns longest DOWNWARD path (edges)
        if (node === null) return 0;
        const left = reach(node.left);    // trust: longest reach down the left
        const right = reach(node.right);  // trust: longest reach down the right
        best = Math.max(best, left + right); // a path that BENDS here uses both sides
        return 1 + Math.max(left, right);  // to the parent: extend the longer side
    }
    reach(root);
    return best;
}
```

The return value (`reach`) is deliberately *not* the answer — it's the piece the parent needs. The answer accumulates on the side. Designing that split is the algorithm.

Example 3 — Lowest common ancestor (LCA)

Given two target nodes `p` and `q`, find their lowest common ancestor. What must a call return to its parent? Just "did I find `p` or `q` below me?" is *almost* enough — combined with the structural fact that the LCA is the deepest node where `p` and `q` appear in *different* subtrees (or the node itself is one of them).

```
def lca(node, p, q):
    """Returns the LCA of p and q if both are in this subtree, else whichever
    of p/q is present, else None. The parent uses the presence info."""
    if node is None or node is p or node is q:
        return node                    # found a target (or dead end)
    left = lca(node.left, p, q)        # trust: LCA/presence on the left
    right = lca(node.right, p, q)      # trust: LCA/presence on the right
    if left and right:
        return node                    # p and q split here -> this node is the LCA
    return left or right               # both on one side: pass up whichever we found
```

JavaScript:

```
function lca(node, p, q) {
  // Returns the LCA if both targets are in this subtree, else whichever of
  // p/q is present, else null. The parent uses the presence information.
  if (node === null || node === p || node === q) return node; // found a target / dead end
  const left = lca(node.left, p, q); // trust: LCA/presence on the left
  const right = lca(node.right, p, q); // trust: LCA/presence on the right
  if (left && right) return node; // targets split here -> this node is the LCA
  return left || right; // both on one side: pass up what we found
}
```

The clever part is that a *single* return value does double duty: partway up it means "here's the one target I found," and the moment both sides come back non-null it means "I am the LCA." Designing a return value that carries just enough for the parent to make that distinction — that is the art.

The lesson

In all three examples the recursion *structure* was routine (visit children, combine). The difficulty — and the algorithm — lived entirely in **deciding what each call returns**:

- Balanced tree: return *height-or-sentinel*, not just a boolean → $O(n)$ instead of $O(n^2)$.
- Diameter: return *downward reach*, accumulate the *bent path* separately → two quantities, cleanly separated.
- LCA: return a *found-target/LCA* value that means different things at different depths → one value, double duty.

When a recursion feels impossible, the question to ask is almost never "how do I recurse?" It is **"what would the recursive call have to hand me for the combine step to be easy?"** Design that return value — put exactly the parent's needed information in it, nothing more — and the algorithm falls out. This is the single most transferable algorithm-design skill in this book.

Checkpoint — end of Part IV. You should now be able to: write recursions over numbers, lists, strings, linked lists, trees, and nested data using the standard templates; reverse and merge linked lists by rewiring on the upward phase; add a visited set when a structure is a graph rather than a tree; and — the key design skill — decide what a recursive call should return so the parent gets exactly the information it needs (height-or-sentinel for balance, downward-reach plus a running best for diameter).

Part V — Branching & Search

Learning objectives. Handle recursions that branch into many possibilities: enumerate subsets/permutations/combinations, solve constraint puzzles by backtracking with pruning, and parse nested grammars.

The recursions in Part IV mostly made *one* recursive call per step (or a fixed number, like a tree's two children). Now we look at recursions that branch into *many* possibilities and explore them — the engine behind puzzles, combinatorics, and search.

Chapter 17. Multiple recursion and the subproblem explosion

Multiple recursion is any function that calls itself more than once in a way that branches. Fibonacci is the canonical example:

```
def fib(n):
    if n < 2:
        return n
    return fib(n - 1) + fib(n - 2)
```

JavaScript:

```
function fib(n) {
    if (n < 2) return n;
    return fib(n - 1) + fib(n - 2);
}
```

Two calls per level means the call tree roughly doubles each level, giving exponential work. This is fine for small n and disastrous for large n — which is exactly the motivation for memoization in Part VI. But branching itself is not bad; it's *necessary* whenever a problem has genuinely many independent sub-possibilities to combine. The **Tower of Hanoi** is a beautiful case where branching is the whole solution:

```
def hanoi(n, source, target, spare):
    if n == 0:
        return
    hanoi(n - 1, source, spare, target)    # move top n-1 out of the way
    print(f"Move disk {n}: {source} -> {target}")    # move the big disk
    hanoi(n - 1, spare, target, source)    # bring the n-1 back on top
```

JavaScript:

```
function hanoi(n, source, target, spare, moves = []) {
    if (n === 0) return moves;
    hanoi(n - 1, source, spare, target, moves);    // move top n-1 out of the way
    moves.push(`Move disk ${n}: ${source} -> ${target}`);    // move the big disk
    hanoi(n - 1, spare, target, source, moves);    // bring the n-1 back on top
    return moves;
}
```

Read the three lines as a plan: to move n disks, first move the top $n-1$ aside (trusting the recursion), move the largest, then move the $n-1$ back on top (trusting again). You never trace the whole thing; you trust the two smaller moves. That is the leap of faith doing heavy lifting.

Chapter 18. Generating all possibilities (combinatorial recursion)

A whole class of problems asks you to *enumerate* things: all subsets, all permutations, all combinations, all binary strings of length n . The recursive idea is always the same: **make one decision, then recurse on the rest**.

All subsets (the power set). For each element, you either include it or you don't — two branches:


```
def subsets(xs):
    if not xs:
        return [[]] # base: the only subset of nothing is the empty set
    first, rest = xs[0], xs[1:]
    without = subsets(rest) # subsets that exclude first
    with_first = [[first] + s for s in without] # same subsets, plus first
    return without + with_first
```

JavaScript:

```
function subsets(xs) {
    if (xs.length === 0) return [[]]; // base: only subset of nothing is the empty set
    const first = xs[0], rest = xs.slice(1);
    const without = subsets(rest); // subsets that exclude first
    const withFirst = without.map(s => [first, ...s]); // same subsets, plus first
    return [...without, ...withFirst];
}
```

All permutations. Pick each element to go first, then permute the remainder:

```
def permutations(xs):
    if len(xs) <= 1:
        return [xs]
    result = []
    for i in range(len(xs)):
        chosen = xs[i]
        rest = xs[:i] + xs[i+1:]
        for perm in permutations(rest): # trust: all perms of the rest
            result.append([chosen] + perm)
    return result
```

JavaScript:

```
function permutations(xs) {
    if (xs.length <= 1) return [xs];
    const result = [];
    for (let i = 0; i < xs.length; i++) {
        const chosen = xs[i];
        const rest = [...xs.slice(0, i), ...xs.slice(i + 1)];
        for (const perm of permutations(rest)) { // trust: all perms of the rest
            result.push([chosen, ...perm]);
        }
    }
    return result;
}
```

All binary strings of length n. Append a 0 or a 1, then recurse for the remaining length:

```
def binary_strings(n, prefix=""):
    if n == 0:
        print(prefix) # base: prefix is complete
        return
    binary_strings(n - 1, prefix + "0")
    binary_strings(n - 1, prefix + "1")
```

JavaScript:

```
function binaryStrings(n, prefix = "") {
    if (n === 0) {
        console.log(prefix); // base: prefix is complete
        return;
    }
    binaryStrings(n - 1, prefix + "0");
    binaryStrings(n - 1, prefix + "1");
}
```

Feel the common shape: **a base case that emits/returns a completed item, and a recursive case that branches over the choices for the next position.** Master this shape and most "generate all X" problems become routine.

Chapter 19. Backtracking: search with take-backs

Backtracking is combinatorial recursion plus one crucial move: when a partial solution can't possibly work, you *abandon* it (backtrack) instead of continuing. You explore a decision tree depth-first, undoing choices as you retreat. The template:

```
def backtrack(state, choices):
    if is_complete(state):
        record(state)                # found a full solution
        return
    for choice in choices:
        if is_valid(state, choice):  # prune dead branches early
            make(state, choice)      # choose
            backtrack(state, next_choices(state, choice)) # explore
            undo(state, choice)      # un-choose (the "back" in backtracking)
```

JavaScript:

```
function backtrack(state, choices) {
    if (isComplete(state)) {
        record(state);                // found a full solution
        return;
    }
    for (const choice of choices) {
        if (isValid(state, choice)) { // prune dead branches early
            make(state, choice);      // choose
            backtrack(state, nextChoices(state, choice)); // explore
            undo(state, choice);      // un-choose (the "back" in backtracking)
        }
    }
}
```

N-Queens — place n queens on an $n \times n$ board so none attack another. Decide one row at a time; try each column; abandon columns that conflict:

```
def solve_n_queens(n):
    solutions = []
    def place(row, cols, diag1, diag2, board):
        if row == n:
            solutions.append(board[:]) # a complete valid placement
            return
        for col in range(n):
            d1, d2 = row - col, row + col
            if col in cols or d1 in diag1 or d2 in diag2:
                continue # prune: this column is attacked
            cols.add(col); diag1.add(d1); diag2.add(d2) # choose
            board.append(col)
            place(row + 1, cols, diag1, diag2, board) # explore
            board.pop(); cols.discard(col)           # undo
            diag1.discard(d1); diag2.discard(d2)
    place(0, set(), set(), set(), [])
    return solutions
```

JavaScript:

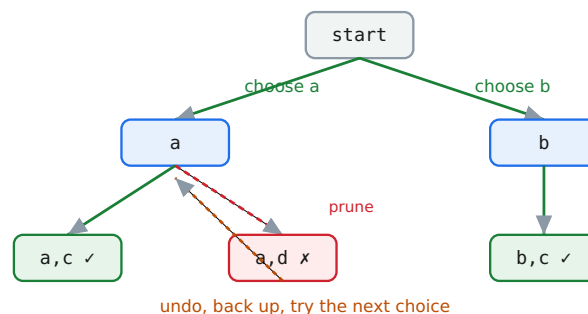
```
function solveNQueens(n) {
  const solutions = [];
  const cols = new Set(), diag1 = new Set(), diag2 = new Set(), board = [];
  function place(row) {
    if (row === n) { solutions.push([...board]); return; } // complete valid placement
    for (let col = 0; col < n; col++) {
      const d1 = row - col, d2 = row + col;
      if (cols.has(col) || diag1.has(d1) || diag2.has(d2)) continue; // prune
      cols.add(col); diag1.add(d1); diag2.add(d2); board.push(col); // choose
      place(row + 1); // explore
      board.pop(); cols.delete(col); diag1.delete(d1); diag2.delete(d2); // undo
    }
  }
  place(0);
  return solutions;
}
```

Maze / path finding — from a cell, try each direction; if you reach the goal, success; otherwise backtrack. Mark cells visited so you don't loop:

```
def solve_maze(grid, r, c, path, visited):
  if not (0 <= r < len(grid) and 0 <= c < len(grid[0])):
    return False # off the board
  if grid[r][c] == 'wall' or (r, c) in visited:
    return False
  path.append((r, c))
  visited.add((r, c))
  if grid[r][c] == 'goal':
    return True # base: arrived
  for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
    if solve_maze(grid, r+dr, c+dc, path, visited):
      return True # a neighbor found the goal
  path.pop() # undo: dead end, back out
  return False
```

JavaScript:

```
function solveMaze(grid, r, c, path, visited) {
  if (r < 0 || c < 0 || r >= grid.length || c >= grid[0].length) return false; // off board
  if (grid[r][c] === "wall" || visited.has(r + "," + c)) return false;
  path.push([r, c]);
  visited.add(r + "," + c);
  if (grid[r][c] === "goal") return true; // base: arrived
  for (const [dr, dc] of [[-1,0],[1,0],[0,-1],[0,1]]) {
    if (solveMaze(grid, r + dr, c + dc, path, visited)) return true; // a neighbor found the goal
  }
  path.pop(); // undo: dead end, back out
  return false;
}
```



Backtracking walks a decision tree: choose (green) → explore deeper → hit a dead end (red), prune, then undo (orange) and try the next branch.

A backtracking decision tree: choose, explore, prune dead ends, undo, try the next branch.

The mental model for backtracking: **you are walking a tree of decisions. Going deeper is "choose"; returning**

is "un-choose." Pruning invalid branches early is what makes it fast enough to be practical. Sudoku solvers, word-search grids, subset-sum, combination-sum, and constraint puzzles are all this same skeleton with different `is_valid` rules.

Active practice (answers in Chapter 48's answer key, §19). 1. Predict. In `subsets`, after the "include `xs[i]`" branch fully returns, what must be true about `current` before the "exclude" branch runs? 2. Complete. Fill the pruning guard so `gen_parens` never produces an invalid string; add `)` only when _____. 3. Diagnose. A backtracking solver returns the same solution many times, each identical. Which line is almost certainly wrong? 4. Derive (blank page). Write a backtracker that prints all ways to place `k` non-attacking rooks on an `n×n` board. State the invariant that must hold after each `undo`.

Chapter 20. Recursive descent: parsing with recursion

Language and expression grammars are self-similar — an expression can contain sub-expressions — so parsing them is naturally recursive. A **recursive descent parser** has one function per grammar rule, and rules that refer to each other become functions that call each other. Here's a compact evaluator for arithmetic with `+` `-` `*` `/` and parentheses, honoring precedence:

```
def evaluate(expr):
    """Precondition: expr is a syntactically valid arithmetic expression
    containing only digits, + - * /, and matched parentheses, with NO
    spaces and NO unary minus (e.g. "-3" or "2 + 3" are not handled).
    Handling those needs a tokenizer step and a unary-operator rule."""
    pos = 0
    def peek(): return expr[pos] if pos < len(expr) else None
    def advance():
        nonlocal pos; ch = expr[pos]; pos += 1; return ch

    def parse_expression():          # handles + and -
        value = parse_term()
        while peek() in ('+', '-'):
            op = advance()
            rhs = parse_term()
            value = value + rhs if op == '+' else value - rhs
        return value

    def parse_term():                # handles * and / (higher precedence)
        value = parse_factor()
        while peek() in ('*', '/'):
            op = advance()
            rhs = parse_factor()
            value = value * rhs if op == '*' else value / rhs
        return value

    def parse_factor():              # numbers and parenthesized sub-expressions
        if peek() == '(':
            advance()                # consume '('
            value = parse_expression() # recurse into the sub-expression
            advance()                # consume ')'
            return value
        num = ''
        while peek() is not None and peek().isdigit():
            num += advance()
        return int(num)

    return parse_expression()

# evaluate("2+3*4")    -> 14
# evaluate("(2+3)*4")  -> 20
```

JavaScript:

```

function evaluate(expr) {
  // Precondition: expr is a syntactically valid arithmetic expression using only
  // digits, + - * /, and matched parentheses, with NO spaces and NO unary minus.
  let pos = 0;
  const peek = () => (pos < expr.length ? expr[pos] : null);
  const advance = () => expr[pos++];

  function parseExpression() { // handles + and -
    let value = parseTerm();
    while (peek() === "+" || peek() === "-") {
      const op = advance(), rhs = parseTerm();
      value = op === "+" ? value + rhs : value - rhs;
    }
    return value;
  }

  function parseTerm() { // handles * and / (higher precedence)
    let value = parseFactor();
    while (peek() === "*" || peek() === "/") {
      const op = advance(), rhs = parseFactor();
      value = op === "*" ? value * rhs : value / rhs;
    }
    return value;
  }

  function parseFactor() { // numbers and (sub-expressions)
    if (peek() === "(") {
      advance(); // consume '('
      const value = parseExpression(); // recurse into the sub-expression
      advance(); // consume ')'
      return value;
    }
    let num = "";
    while (peek() !== null && /[0-9]/.test(peek())) num += advance();
    return parseInt(num, 10);
  }

  return parseExpression();
}

```

The key insight: `parse_factor` calls `parse_expression` for parenthesized groups, which calls `parse_term`, which calls `parse_factor` again — the mutual calls mirror how the grammar nests. This is the same machinery behind real compilers, calculators, and config/JSON parsers.

Checkpoint — end of Part V. You should now be able to: enumerate subsets, permutations, and combinations; solve constraint puzzles by backtracking with choose/explore/undo and pruning; decide when to unmark visited nodes; and parse a nested grammar with recursive descent.

Part VI — Divide, Conquer & Optimize

Learning objectives. Split problems into halves and combine; recognize overlapping subproblems; and speed up recursion with memoization, taking the first step toward dynamic programming.

Chapter 21. Divide and conquer

Divide and conquer is a recursion strategy in three beats: **divide** the problem into smaller independent parts (usually halves), **conquer** each part by recursion, and **combine** the sub-results. Because each step halves the input, these algorithms are famously fast.

Binary search — find a target in a *sorted* array by repeatedly halving the search range:

```
def binary_search(xs, target, lo=0, hi=None):
    if hi is None:
        hi = len(xs) - 1
    if lo > hi:
        return -1                # base: empty range, not found
    mid = (lo + hi) // 2
    if xs[mid] == target:
        return mid
    if target < xs[mid]:
        return binary_search(xs, target, lo, mid - 1)  # left half
    return binary_search(xs, target, mid + 1, hi)      # right half
```

JavaScript:

```
function binarySearch(xs, target, lo = 0, hi = null) {
    if (hi === null) hi = xs.length - 1;
    if (lo > hi) return -1;                // base: empty range, not found
    const mid = Math.floor((lo + hi) / 2);
    if (xs[mid] === target) return mid;
    if (target < xs[mid]) return binarySearch(xs, target, lo, mid - 1); // left half
    return binarySearch(xs, target, mid + 1, hi); // right half
}
```

Merge sort — split in half, sort each half (trust!), then merge two sorted halves:

```
def merge_sort(xs):
    if len(xs) <= 1:
        return xs                # base: already sorted
    mid = len(xs) // 2
    left = merge_sort(xs[:mid])  # conquer left
    right = merge_sort(xs[mid:]) # conquer right
    return merge(left, right)    # combine

def merge(a, b):
    result, i, j = [], 0, 0
    while i < len(a) and j < len(b):
        if a[i] <= b[j]:
            result.append(a[i]); i += 1
        else:
            result.append(b[j]); j += 1
    result.extend(a[i:]); result.extend(b[j:])
    return result
```

JavaScript:

```
function mergeSort(xs) {
  if (xs.length <= 1) return xs;           // base: already sorted
  const mid = Math.floor(xs.length / 2);
  const left = mergeSort(xs.slice(0, mid)); // conquer left
  const right = mergeSort(xs.slice(mid));   // conquer right
  return merge(left, right);               // combine
}

function merge(a, b) {
  const result = []; let i = 0, j = 0;
  while (i < a.length && j < b.length) {
    if (a[i] <= b[j]) result.push(a[i++]);
    else result.push(b[j++]);
  }
  return result.concat(a.slice(i)).concat(b.slice(j));
}
```

Quicksort — pick a pivot, partition into smaller/equal/greater, recurse on the outer parts:

```
def quicksort(xs):
  if len(xs) <= 1:
    return xs
  pivot = xs[len(xs) // 2]
  less = [x for x in xs if x < pivot]
  equal = [x for x in xs if x == pivot]
  greater = [x for x in xs if x > pivot]
  return quicksort(less) + equal + quicksort(greater)
```

JavaScript:

```
function quicksort(xs) {
  if (xs.length <= 1) return xs;
  const pivot = xs[Math.floor(xs.length / 2)];
  const less = xs.filter(x => x < pivot);
  const equal = xs.filter(x => x === pivot);
  const greater = xs.filter(x => x > pivot);
  return [...quicksort(less), ...equal, ...quicksort(greater)];
}
```

The signature of divide-and-conquer: **two (or more) recursive calls on roughly half the data, with real work in the combine step (merge) or the split step (partition)**. When you see "sorted," "halve," or "split and merge," think divide and conquer.

Active practice (answers in Chapter 48's answer key, §21). 1. Predict. In `binary_search([1,4,6,9,11], 9, 0, 4)`, which half does the first recursive call explore, and why? 2. Complete. Fill the recurrence so `merge_sort` splits correctly: `mid = ____`; `left = merge_sort(xs[:mid])`; `right = merge_sort(xs[mid:])`. 3. Diagnose. A binary search loops forever on a missing target. Which of `mid`, `mid-1`, `mid+1` is misused? 4. Derive (blank page). Write a divide-and-conquer function returning the **maximum element** of an array by splitting in half. State its recurrence $T(n)=\dots$ and solve it (Chapter 32).

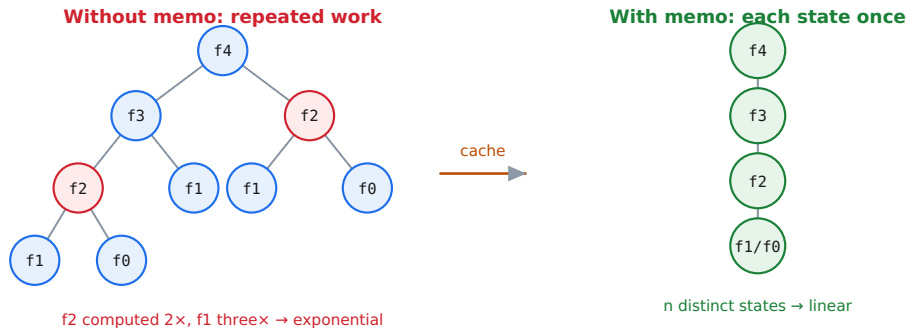
Chapter 22. Memoization: remembering to go fast

Recall that naive Fibonacci recomputes the same subproblems exponentially many times. **Memoization** fixes this by caching each result the first time it's computed and reusing it thereafter. The recursion's *shape* is unchanged; you just add a cache.

```
def fib(n, memo=None):
  if memo is None:
    memo = {}
  if n < 2:
    return n
  if n in memo:
    # already computed? return cached
    return memo[n]
  memo[n] = fib(n - 1, memo) + fib(n - 2, memo) # compute once, store
  return memo[n]
```

JavaScript:

```
function fib(n, memo = {}) {
  if (n < 2) return n;
  if (n in memo) return memo[n]; // already computed? return cached
  memo[n] = fib(n - 1, memo) + fib(n - 2, memo); // compute once, store
  return memo[n];
}
```



Repeated subproblems (left) collapse into one-per-state work once memoized (right).

In Python, the standard library gives you this for free:

```
from functools import lru_cache

@lru_cache(maxsize=None)
def fib(n):
  if n < 2:
    return n
  return fib(n - 1) + fib(n - 2)
```

JavaScript:

```
// JavaScript has no built-in memo decorator; wrap the function in a cache.
const fib = (() => {
  const memo = new Map();
  return function fib(n) {
    if (n < 2) return n;
    if (memo.has(n)) return memo.get(n);
    const v = fib(n - 1) + fib(n - 2);
    memo.set(n, v);
    return v;
  };
})();
```

Memoization turns the exponential Fibonacci into linear time, because each distinct n is computed exactly once. The rule of thumb: **if your recursion revisits the same subproblem more than once, memoize it**. This condition — called *overlapping subproblems* — is precisely what separates problems that merely branch (Hanoi, permutations, where every subproblem is distinct) from problems that benefit from caching.

Chapter 23. From recursion to dynamic programming

Dynamic programming (DP) is, at heart, memoized recursion — often rewritten as a bottom-up loop. The recursion tells you *what* to compute (the recurrence and base cases); DP is *how* to compute it efficiently. The recommended path for almost any DP problem is: **first write the plain recursion, then add memoization, then (if you want) convert to a loop**. Writing the recursion first is the part beginners skip and then struggle; don't skip it.

Coin change (fewest coins to make an amount). Recurrence: the best for `amount` is 1 plus the best for `amount - coin`, minimized over coins.


```

from functools import lru_cache
def min_coins(coins, amount):
    """Precondition: every coin is a POSITIVE integer and amount >= 0.
    A zero or negative coin would let `rem` stall or grow, so `rem - c`
    might never reach the rem == 0 base case → infinite recursion."""
    @lru_cache(maxsize=None)
    def best(rem):
        if rem == 0:
            return 0 # base: nothing left to make
        if rem < 0:
            return float('inf') # base: overshoot, impossible
        return min((1 + best(rem - c) for c in coins), default=float('inf'))
    result = best(amount)
    return result if result != float('inf') else -1

```

JavaScript:

```

function minCoins(coins, amount) {
    // Precondition: every coin is a POSITIVE integer and amount >= 0.
    // A zero or negative coin would let `rem` stall or grow → infinite recursion.
    const memo = new Map();
    function best(rem) {
        if (rem === 0) return 0; // base: nothing left to make
        if (rem < 0) return Infinity; // base: overshoot, impossible
        if (memo.has(rem)) return memo.get(rem);
        let m = Infinity;
        for (const c of coins) m = Math.min(m, 1 + best(rem - c));
        memo.set(rem, m);
        return m;
    }
    const result = best(amount);
    return result !== Infinity ? result : -1;
}

```

Longest common subsequence of two strings — compare the last characters; if equal, they contribute one and we recurse on both shortened; else we try dropping one character from either:

```

from functools import lru_cache
def lcs(a, b):
    @lru_cache(maxsize=None)
    def solve(i, j):
        if i == 0 or j == 0:
            return 0 # base: one string exhausted
        if a[i-1] == b[j-1]:
            return 1 + solve(i-1, j-1) # match: take it
        return max(solve(i-1, j), solve(i, j-1)) # try dropping from each
    return solve(len(a), len(b))

```

JavaScript:

```

function lcs(X, Y) {
    const memo = new Map();
    function solve(i, j) {
        if (i === 0 || j === 0) return 0; // base: one string exhausted
        const key = i + "," + j;
        if (memo.has(key)) return memo.get(key);
        let val;
        if (X[i-1] === Y[j-1]) val = 1 + solve(i-1, j-1); // match: take it
        else val = Math.max(solve(i-1, j), solve(i, j-1)); // try dropping from each
        memo.set(key, val);
        return val;
    }
    return solve(X.length, Y.length);
}

```

Grid paths (count ways to reach bottom-right moving only right/down):

```

from functools import lru_cache
def count_paths(rows, cols):
    """Precondition: rows >= 1 and cols >= 1 (a grid needs at least one cell)."""
    @lru_cache(maxsize=None)
    def ways(r, c):
        if r == 0 or c == 0:
            return 1 # base: only one way along an edge
        return ways(r-1, c) + ways(r, c-1) # from above + from the left
    return ways(rows-1, cols-1)

```

JavaScript:

```

function countPaths(rows, cols) {
    // Precondition: rows >= 1 and cols >= 1 (a grid needs at least one cell).
    const memo = new Map();
    function ways(r, c) {
        if (r === 0 || c === 0) return 1; // base: only one way along an edge
        const key = r + "," + c;
        if (memo.has(key)) return memo.get(key);
        const v = ways(r-1, c) + ways(r, c-1); // from above + from the left
        memo.set(key, v);
        return v;
    }
    return ways(rows - 1, cols - 1);
}

```

Notice how each of these is just the Part II recipe — contract, base case, leap of faith, combine — plus a cache. That's the reassuring truth about DP: **it isn't a new skill, it's recursion you already know, made efficient**. Once you can write the recurrence, `@lru_cache` does the rest, and converting to bottom-up tables is a mechanical follow-up when you need the extra speed or want to avoid deep call stacks.

Active practice (answers in Chapter 48's answer key, §23). 1. Predict. In memoized `climb(n)`, how many distinct states are ever computed for `climb(20)`? 2. Complete. Add the one line that turns the plain recursion into $O(n)$: ____ above the function. 3. Diagnose. A memoized grid-path counter gives wrong answers when obstacles are added. Which base case was probably omitted? 4. Derive (blank page). Write `min_path_sum(grid)` (top-left to bottom-right, right/down only) with memoization. Define its state, then give its time and space by the states $\times$ work rule (Chapter 35).

Checkpoint — end of Part VI. You should now be able to: split a problem into halves and combine; recognize overlapping subproblems; and convert a slow recursion into memoized (top-down) form.

Part VII — Advanced & Exotic Forms

Learning objectives. Round out your fluency with the forms you'll meet less often: tail recursion and why some languages optimize it, mutual and nested recursion, the structural-vs-generative distinction, higher-order recursion and corecursion, continuation-passing style, and the general recipe for converting any recursion into an explicit-stack iteration. (Optional on a first read — come back after the core patterns feel natural.)

This part catalogs the remaining forms of recursion so your coverage is complete. You'll meet each less often than the patterns above, but recognizing them rounds out your fluency.

Chapter 24. Tail recursion

A recursive call is in **tail position** if it is the *very last* action the function performs — nothing is done to its result before returning. Contrast:

```
def factorial(n):                # NOT tail recursive
    if n == 0: return 1
    return n * factorial(n - 1)  # after the call returns, we still multiply

def factorial_tail(n, acc=1):    # tail recursive (uses an accumulator)
    if n == 0: return acc
    return factorial_tail(n - 1, acc * n)  # the call IS the last action
```

JavaScript:

```
function factorial(n) {          // NOT tail recursive
    if (n === 0) return 1;
    return n * factorial(n - 1); // after the call returns, we still multiply
}

function factorialTail(n, acc = 1) { // tail recursive (uses an accumulator)
    if (n === 0) return acc;
    return factorialTail(n - 1, acc * n); // the call IS the last action
}
```

Why it matters: in tail position, the current frame has no more work to do, so a language *can* reuse the frame instead of stacking a new one — **tail-call optimization (TCO)** — turning the recursion into a loop under the hood, with constant stack usage. Languages like Scheme and many functional languages guarantee this; C and JavaScript engines may do it. Importantly, **Python does not perform TCO**, by an intentional design decision, so tail recursion in Python still consumes stack and can still overflow. The accumulator style from Chapter 8 is what makes a recursion tail-recursive; in Python, treat that mainly as a stepping stone to a plain loop when depth is a concern.

Chapter 25. Mutual (indirect) recursion

Recursion doesn't require a function to call *itself* directly. Two or more functions can call *each other*; this is **mutual recursion**. The classic tidy example is even/odd defined in terms of each other:

```
def is_even(n):
    # Precondition: n is a non-negative integer (negative n never reaches 0).
    if n == 0: return True
    return is_odd(n - 1)

def is_odd(n):
    if n == 0: return False
    return is_even(n - 1)
```

JavaScript:

```
function isEven(n) {
  // Precondition: n is a non-negative integer (negative n never reaches 0).
  if (n === 0) return true;
  return isOdd(n - 1);
}

function isOdd(n) {
  if (n === 0) return false;
  return isEven(n - 1);
}
```

Each call reduces n , so the pair marches to the base case together. You already saw a practical instance in Chapter 20: the parser's `parse_expression` $\rightarrow$ `parse_term` $\rightarrow$ `parse_factor` $\rightarrow$ `parse_expression` cycle is mutual recursion mirroring a grammar. Whenever two concepts are defined in terms of each other, mutual recursion is the natural expression.

Chapter 26. Nested recursion

In **nested recursion**, a recursive call appears *inside the argument* of another recursive call — the function's own result feeds back into itself. The textbook example is the Ackermann function, famous for growing astronomically fast and for being *total but not primitive recursive* — meaning it cannot be written using only bounded `for`-style loops whose iteration counts are fixed in advance:

```
def ackermann(m, n):
    """Precondition: m, n are non-negative integers. Grows astronomically."""
    if m == 0:
        return n + 1
    if n == 0:
        return ackermann(m - 1, 1)
    return ackermann(m - 1, ackermann(m, n - 1)) # recursion inside recursion
```

JavaScript:

```
function ackermann(m, n) {
  // Precondition: m, n are non-negative integers. Grows astronomically.
  if (m === 0) return n + 1;
  if (n === 0) return ackermann(m - 1, 1);
  return ackermann(m - 1, ackermann(m, n - 1)); // recursion inside recursion
}
```

A common misstatement is that Ackermann "can't be done with loops." It can — any computable function can be evaluated iteratively by maintaining an explicit stack (Chapter 30). What it *cannot* be expressed as is a *primitive recursive* function (roughly, one built only from bounded loops with no open-ended `while`). The precise fact is about that restricted class, not about looping in general. You'll rarely write nested recursion in practice, but knowing it exists explains why some recursions are far more powerful (and more expensive) than ordinary ones.

Chapter 27. Structural vs. generative recursion

A useful lens for classifying recursion by *where the smaller input comes from*. In **structural recursion**, the smaller input is a piece of the input's own structure — the tail of a list, a subtree, a substring. Termination is basically guaranteed because the structure is finite and you always move inward. Most of Part IV is structural.

In **generative recursion**, you *compute* a new, smaller problem that isn't literally a substructure of the input — quicksort generates partitions, Euclid's gcd generates `a mod b`, Newton's method generates a refined guess. Generative recursion is more powerful but requires you to *prove* termination yourself, since there's no finite structure automatically shrinking. Knowing which kind you're writing tells you how carefully to check that you'll actually reach the base case.

Chapter 28. Higher-order and corecursion (a peek)

Two ideas at the edges. **Higher-order recursion** is when recursion is expressed through functions that take functions — a `fold / reduce` is recursion over a list abstracted into a reusable combinator, and mapping a function over a tree is recursion parameterized by an operation. Recognizing this lets you replace many hand-written

recursions with a single fold.

Corecursion flips the usual direction: instead of breaking input *down* to a base case, it builds output *up*, potentially forever, producing one piece at a time on demand. Generators express this naturally — an infinite stream of naturals or an unfolding sequence. Where ordinary recursion is defined by how it *terminates*, corecursion is defined by how it *produces*. This is more of a functional-programming curiosity for most readers, but it completes the map.

Chapter 29. The Continuation-Passing Style (CPS), briefly

In **continuation-passing style**, instead of returning a result, each function receives an extra argument — a *continuation*, i.e. a function representing "what to do next" — and calls it with the result. Every call becomes a tail call, which is how compilers and some interpreters convert recursion into iteration internally, and how languages without native loops still compute.

```
def factorial_cps(n, k):
    if n == 0:
        return k(1) # hand the result to the continuation
    return factorial_cps(n - 1, lambda r: k(n * r)) # build a bigger continuation
# factorial_cps(5, lambda x: x) -> 120
```

JavaScript:

```
function factorialCPS(n, k) {
    if (n === 0) return k(1); // hand the result to the continuation
    return factorialCPS(n - 1, r => k(n * r)); // build a bigger continuation
}
// factorialCPS(5, x => x) -> 120
```

You will almost never write CPS by hand, but seeing it demystifies phrases like "callbacks are continuations" and explains how deeply recursive computations can be flattened.

Chapter 30. Converting recursion to iteration with an explicit stack

Any recursion can be made iterative by managing the call stack *yourself* with an explicit stack data structure — precisely the manual bookkeeping we described back in Chapter 5. This is the go-to fix when a correct tree/graph recursion is too deep for the language's call stack. Depth-first tree traversal, rewritten iteratively:

```
def inorder_iterative(root):
    out, stack, node = [], [], root
    while stack or node:
        while node: # go as far left as possible, remembering the way
            stack.append(node)
            node = node.left
        node = stack.pop() # backtrack to the last remembered node
        out.append(node.value) # visit it
        node = node.right # then head right
    return out
```

JavaScript:

```
function inorderIterative(root) {
    const out = [], stack = [];
    let node = root;
    while (stack.length || node) {
        while (node) { // go as far left as possible, remembering the way
            stack.push(node);
            node = node.left;
        }
        node = stack.pop(); // backtrack to the last remembered node
        out.push(node.value); // visit it
        node = node.right; // then head right
    }
    return out;
}
```

The explicit `stack` list is doing exactly what the call stack did automatically. Understanding this equivalence is the

capstone of "getting" recursion: **the machine was always using a stack; recursion just let you not write it by hand.**

Checkpoint — end of Part VII. *You should now be able to: recognize tail position and explain why CPython doesn't optimize it; spot mutual and nested recursion; tell structural from generative recursion and know which needs its own termination argument; and convert a recursive tree/graph walk into an iterative one using an explicit stack.*

Part VIII — Analyzing Recursion (The Exam Core)

Learning objectives. Write a recurrence relation from code; solve it by substitution, recursion tree, or Master Theorem; analyze stack-space as well as time; and catch the hidden-cost traps that exams plant.

Recognizing and writing recursion is half the battle. Exams — GATE, university finals, and coding interviews — spend at least as much time asking *how fast* and *how much memory* a recursion uses. This part turns that from a source of dread into a mechanical skill. The single most important idea: **every recursive function has a recurrence relation, and a recurrence relation is just the running time written in terms of itself.** Once you can write the recurrence, three tools solve it.

Chapter 31. From code to recurrence relation

A **recurrence relation** expresses the time $T(n)$ to solve a problem of size n in terms of the time to solve smaller versions, plus the non-recursive work done at this level. You build it by reading the code literally:

- Each recursive call contributes a $T(\text{smaller})$ term.
- All the non-recursive work in one call (loops, comparisons, arithmetic, the combine step) contributes an $f(n)$ term.
- The base case contributes $T(\text{small}) = O(1)$.

Read these examples until the translation feels automatic:

```
def factorial(n):          # one call on n-1, O(1) work here
    if n == 0: return 1
    return n * factorial(n - 1)
```

JavaScript:

```
function factorial(n) {    // one call on n-1, O(1) work here
    if (n === 0) return 1;
    return n * factorial(n - 1);
}
```

Recurrence: $T(n) = T(n-1) + O(1)$. One smaller call, constant work.

```
def sum_list(xs):          # one call on the tail, O(1) work
    if not xs: return 0
    return xs[0] + sum_list(xs[1:]) # NOTE: xs[1:] copies -> see the trap below
```

JavaScript:

```
function sumList(xs) {     // one call on the tail, O(1) work
    if (xs.length === 0) return 0;
    return xs[0] + sumList(xs.slice(1)); // NOTE: slice copies -> see the trap below
}
```

Recurrence (ignoring the slice copy): $T(n) = T(n-1) + O(1)$.

```
def binary_search(xs, t, lo, hi):    # ONE call on half, O(1) work
    ...
    return binary_search(xs, t, lo, mid-1)
```

JavaScript:

```
function binarySearch(xs, t, lo, hi) { // ONE call on half, O(1) work
  // ...
  return binarySearch(xs, t, lo, mid - 1);
}
```

Recurrence: $T(n) = T(n/2) + O(1)$.

```
def merge_sort(xs): # TWO calls on halves, O(n) merge
  ...
  return merge(merge_sort(left), merge_sort(right))
```

JavaScript:

```
function mergeSort(xs) { // TWO calls on halves, O(n) merge
  // ...
  return merge(mergeSort(left), mergeSort(right));
}
```

Recurrence: $T(n) = 2 \cdot T(n/2) + O(n)$.

```
def fib(n): # TWO calls, sizes n-1 and n-2, O(1) work
  return fib(n-1) + fib(n-2)
```

JavaScript:

```
function fib(n) { // TWO calls, sizes n-1 and n-2, O(1) work
  return fib(n - 1) + fib(n - 2);
}
```

Recurrence: $T(n) = T(n-1) + T(n-2) + O(1)$.

```
def hanoi(n): # TWO calls on n-1, O(1) work
  hanoi(n-1); move(); hanoi(n-1)
```

JavaScript:

```
function hanoi(n) { // TWO calls on n-1, O(1) work
  hanoi(n - 1); move(); hanoi(n - 1);
}
```

Recurrence: $T(n) = 2 \cdot T(n-1) + O(1)$.

The pattern to memorize: **count the recursive calls (that's the coefficient), note how each shrinks the input (subtract → "decreasing"; divide → "dividing"), and measure the work outside the calls (that's $f(n)$)**. Those three numbers determine everything.

Chapter 32. Three ways to solve a recurrence

There are three core tools used in this book for solving recurrences, and each fits a different situation. (Other methods exist — the Akra-Bazzi method for uneven splits, generating functions, the characteristic-equation method for linear recurrences like Fibonacci — but these three cover essentially all exam recurrences.)

32.1 The substitution (unrolling) method

Expand the recurrence into itself repeatedly, spot the pattern after k steps, then set k so the term reaches the base case. Best for "decreasing" recurrences like $T(n) = T(n-1) + f(n)$.

Take $T(n) = T(n-1) + n$ (this is selection-sort-style work, or `sum_to`):


```

T(n) = T(n-1) + n
      = [T(n-2) + (n-1)] + n
      = [T(n-3) + (n-2)] + (n-1) + n
      ...
      = T(0) + 1 + 2 + ... + n
      = O(1) + n(n+1)/2
      = O(n^2)

```

The habit: unroll two or three times, write the general k -th line, then substitute the k that hits the base case. Summation formulas you must have memorized: $1+2+\dots+n = n(n+1)/2 = \Theta(n^2)$, and $1+2+4+\dots+2^k = 2^{k+1}-1 = \Theta(2^k)$.

32.2 The recursion-tree method

Draw the recursion as a tree. Each node is labeled with the *non-recursive work* it does, f (its size). Sum the work **per level**, then sum across levels. Best for "dividing" recurrences and for building intuition. For $T(n) = 2 \cdot T(n/2) + n$ (merge sort):

```

Level 0:      n                      total = n
Level 1:    n/2  n/2                  total = n
Level 2:   n/4  n/4  n/4  n/4        total = n
...
Level log n: 1 1 1 ... 1  (n leaves) total = n

```

Every level does n work. The tree has $\log_2 n + 1$ levels (halving until size 1). So total = $n \cdot (\log n + 1) = \Theta(n \log n)$. This picture explains *why* merge sort is $n \log n$ far better than any formula: **n work per level $\times$ $\log n$ levels.**

Three questions unlock any recursion tree: (1) How much work per node? (2) How many nodes per level (does it grow, shrink, or stay flat down the tree)? (3) How many levels until the base case?

32.3 The Master Theorem (the exam workhorse)

For **dividing** recurrences of the standard form

$$T(n) = a \cdot T(n/b) + f(n), \text{ with } a \geq 1, b > 1,$$

where a is the number of subproblems, n/b is each subproblem's size, and $f(n)$ is the work to split and combine, the Master Theorem reads off the answer by comparing $f(n)$ against the "watershed" function $n^{(\log_b a)}$. Compute the critical exponent $c = \log_b a$, then:

- **Case 1 (leaves dominate):** if $f(n) = O(n^{(c-\epsilon)})$ for some $\epsilon > 0$ — i.e. f grows *slower* than n^c — then $T(n) = \Theta(n^c)$.
- **Case 2 (balanced):** if $f(n) = \Theta(n^c)$ — f matches the watershed — then $T(n) = \Theta(n^c \cdot \log n)$. (More generally, for $f(n) = \Theta(n^c \cdot \log^k n)$ with $k \geq 0$, we get $T(n) = \Theta(n^c \cdot \log^{k+1} n)$. This extension requires $k \geq 0$; a *negative* power such as $f(n) = n^c / \log n$ is a genuine gap case the Master Theorem does not cover — see the note after the table.)
- **Case 3 (root dominates):** if $f(n) = \Omega(n^{(c+\epsilon)})$ for some $\epsilon > 0$ — f grows *faster* — and the regularity condition $a \cdot f(n/b) \leq k \cdot f(n)$ holds for some $k < 1$, then $T(n) = \Theta(f(n))$.

Worked reads, do these until they're reflexive:

Recurrence	a	b	c = log _b a	f(n)	Case	Answer
Binary search: $T(n)=T(n/2)+O(1)$	1	2	0	$n^0=1$	2	$\Theta(\log n)$
Merge sort: $T(n)=2T(n/2)+O(n)$	2	2	1	n^1	2	$\Theta(n \log n)$
Balanced binary-tree traversal: $T(n)=2T(n/2)+O(1)$	2	2	1	n^0	1	$\Theta(n)$
$T(n)=2T(n/2)+O(n^2)$	2	2	1	n^2	3	$\Theta(n^2)$
Karatsuba: $T(n)=3T(n/2)+O(n)$	3	2	≈ 1.585	n^1	1	$\Theta(n^{1.585})$
Strassen: $T(n)=7T(n/2)+O(n^2)$	7	2	≈ 2.807	n^2	1	$\Theta(n^{2.807})$

The mental shortcut: compute $n^{(\log_b a)}$; **whichever of that or $f(n)$ is bigger wins; if they tie, multiply by $\log n$** . This shortcut applies *only* when the recurrence actually matches the $a \cdot T(n/b) + f(n)$ form and the polynomial-gap

condition (Cases 1 and 3 need f to differ from n^c by a polynomial factor n^ϵ) and the Case-3 regularity condition are satisfied. When $f(n)$ sits in the gap — for instance $f(n) = n^c / \log n$, which is smaller than n^c but not by a polynomial factor — the Master Theorem simply doesn't apply, and you fall back to the recursion-tree method.

One caveat about the tree-traversal row: $T(n)=2T(n/2)+O(1)$ models a *balanced* tree, where each call really does split the work into two equal halves. An *arbitrary* binary tree isn't divided into equal halves — a skewed tree behaves more like $T(n)=T(n-1)+O(1)$ — but a full traversal still visits every node exactly once, so it is $\Theta(n)$ either way. Use the Master Theorem for the balanced idealization; use "one visit per node" as the robust argument for any tree.

32.4 The Master Theorem for decreasing recurrences

The classic Master Theorem does **not** apply when the subproblem shrinks by *subtraction* rather than division. For $T(n) = a \cdot T(n-b) + f(n)$ with $a \geq 1$, $b > 0$, and $f(n) = \Theta(n^k)$, use the decreasing-form rule:

- If $a = 1$: $T(n) = \Theta(n^{k+1})$. Example: $T(n)=T(n-1)+O(n) \rightarrow \Theta(n^2)$; $T(n)=T(n-1)+O(1) \rightarrow \Theta(n)$.
- If $a > 1$: the exponential term dominates and $T(n) = \Theta(a^{n/b})$ — the polynomial n^k factor is swamped and drops out. Example: $T(n)=2T(n-1)+O(1) \rightarrow \Theta(2^n)$ (Tower of Hanoi).

That handles any recurrence with a *single* recursive term. Naive Fibonacci, $T(n) = T(n-1) + T(n-2) + O(1)$, has *two different* subproblem sizes, so it doesn't fit the single-term $a \cdot T(n-b)$ form at all. Its growth is found by the recursion-tree/characteristic-equation argument and comes out to $\Theta(\varphi^n)$ with $\varphi \approx 1.618$ (the golden ratio) — still exponential, just with a smaller base than 2^n because one branch shrinks faster.

The takeaway that survives all this: growth is set by the *combination* of the branching factor (how many recursive calls) and the shrink rate (how much smaller each call's input is). Both matter — one call on $n-1$ is linear, but *two* calls on $n-1$ is exponential. Holding the branching factor fixed, though, the shrink rate is usually what tips the balance: with the same number of calls, **subtractive** shrinking ($n-1$, $n-2$) tends to be exponential, while **divisive** shrinking ($n/2$) tends to be polynomial or logarithmic. That's why Hanoi and naive Fibonacci (subtract, branch twice) are exponential, while merge sort (divide, branch twice) is only $n \log n$.

Chapter 33. Space complexity: don't forget the stack

Time is only half the question; exams love asking about *space*. A recursion's space cost has two parts:

1. **The call-stack depth.** Every pending call holds a frame. The space is proportional to the **maximum depth of the recursion tree**, not the total number of calls. Naive `fib(n)` makes exponentially many calls but the stack is only ever $O(n)$ deep, because it's depth-first — one root-to-leaf path at a time.
2. **Any data allocated per frame.** If each call copies a list (`xs[1:]`) or builds a structure, multiply that by the depth.

Quick reference for common shapes:

Recursion	Time	Stack space
Linear (factorial, sum)	$\Theta(n)$	$\Theta(n)$
Binary search	$\Theta(\log n)$	$\Theta(\log n)$
Merge sort	$\Theta(n \log n)$	$\Theta(\log n)$ call stack + $\Theta(n)$ merge buffer = $\Theta(n)$ total
Naive Fibonacci	$\Theta(\varphi^n)$ ($\varphi \approx 1.618$)	$\Theta(n)$ — depth only!
Balanced tree traversal	$\Theta(n)$	$\Theta(\log n)$
Skewed/degenerate tree traversal	$\Theta(n)$	$\Theta(n)$ — worst-case depth
Memoized DP over 1 param	$\Theta(n)$	$\Theta(n)$ table + $\Theta(n)$ stack
2-D DP (LCS, edit distance)	$\Theta(m \cdot n)$	$\Theta(m \cdot n)$ table

The exam trap here: candidates quote the *number of calls* as the space. It's the *depth* that matters for the stack. Say it out loud when solving: "time counts all nodes; stack space counts the longest path."

Chapter 34. The four hidden-cost traps exams love

These are the details that separate a correct-looking answer from the right one.

Trap 1 — Slicing/copying is not free. In Python, `xs[1:]` builds a new list in $O(n)$. So the "clean" `sum_list(xs[1:])`

is actually $T(n) = T(n-1) + O(n) = O(n^2)$, not $O(n)$. Passing an index (`sum_list(xs, i+1)`) keeps it $O(n)$. Interviewers plant this deliberately; always ask whether the "smaller input" was *copied* or merely *referenced*.

Trap 2 — String concatenation in recursion. `prefix + char` at each of n levels copies the whole prefix, turning an $O(2^n)$ enumeration into $O(n \cdot 2^n)$. Use a list and `join`, or pass indices.

Trap 3 — Recomputation without memoization. Naive `fib` is $O(\varphi^n)$; memoized it's $O(n)$. The *code* looks nearly identical; the complexity differs exponentially. Any time the recursion tree has repeated node labels, the un-memoized cost is far higher than the number of *distinct* subproblems.

Trap 4 — The base case that does $O(n)$ work. If your base case itself scans the input (e.g. `if sorted(xs) == xs`), that cost multiplies through. Base cases are supposed to be cheap; verify they actually are.

Chapter 35. Reading complexity straight off the recursion — a 30-second method

When a problem hands you recursive code and asks for its complexity, run this checklist in your head:

1. **Count recursive calls per invocation → that's a .**
2. **See how the argument shrinks:** $n-1$ / $n-c$ (decreasing) or $n/2$ / n/b (dividing).
3. **Measure the non-recursive work $f(n)$** — watch for hidden slicing/concatenation (Trap 1, 2).
4. **Pick the tool:** dividing → Master Theorem; decreasing with one call → substitution; decreasing with ≥ 2 calls → decreasing-form (likely exponential); repeated subproblems → memoize, then count *distinct states* $\times$ *work per state*.
5. **State space separately:** maximum recursion depth (+ per-frame allocations, + any memo table).

For DP specifically, there's a reliable accounting rule: **time = (number of reachable states) $\times$ (work done per transition); space = (size of the memo table) + (maximum recursion depth)**. For LCS with states (i, j) , that's $m \cdot n$ states $\times$ $O(1)$ each = $O(mn)$ time and $O(mn)$ space. This one formula answers most DP complexity questions on sight — just remember that it gives a *polynomial* bound only when the number of reachable states is itself polynomial (Chapter 39).

Checkpoint — end of Part VIII. You should now be able to: write a recurrence relation from code; solve it by substitution, recursion tree, or the Master Theorem (and know when none of these apply); and state time and stack-space complexity, watching for the hidden-cost traps.

Part IX — Pattern Recognition for Exams

Learning objectives. Map problem-statement signal words to techniques; wield the pick/don't-pick master template across many problems; and convert recursion → memoization → tabulation mechanically, defining the state correctly.

Exam and interview problems are not infinite. They cluster into a dozen recurring shapes. Once you can *name the shape* from the problem statement, the code writes itself. This part teaches the naming skill, then drills the highest-value patterns with full worked solutions.

Chapter 36. The signal words that reveal the pattern

Train yourself to react to these phrases the way a doctor reacts to symptoms — but remember that symptoms *suggest* a diagnosis, they don't prove it. A signal word points you to a *candidate* pattern; before committing, verify the state, the recurrence, the constraints, and (for DP) whether subproblems actually overlap. "Minimum," "maximum," and "how many ways" often mean DP, but a greedy choice or a closed-form formula sometimes wins instead. Use the table to form a hypothesis, then check it.

If the problem says...	The pattern is...	Reach for...
"all subsets / subsequences", "each element in or out"	pick / don't-pick	binary-choice recursion (Ch. 37)
"all permutations / arrangements / orderings"	choose-one-then-rest	permutation recursion
"all combinations of k", "all ways to..."	choose with a start index	combination recursion
"place / arrange without conflict", "valid configuration"	constraint satisfaction	backtracking with pruning (Ch. 19)
"find a path", "can we reach", "maze / grid / islands"	explore neighbors	DFS / flood fill
"minimum / maximum ... to achieve", "fewest / most"	optimization over choices	DP (memoized recursion, Ch. 39)
"how many ways", "count the number of..."	counting over choices	DP (sum instead of min/max)
"longest / shortest sub-...", "edit / match two strings"	two-pointer DP	2-D DP
"sorted array", "search", "find in O(log n)"	halve the range	binary search / divide & conquer
"sort", "split and merge/partition"	divide & conquer	merge/quick sort
"nested / hierarchical / tree / JSON / folders"	self-similar structure	structural recursion (Ch. 15-16)
"nth term defined by previous terms"	linear recurrence	direct recursion, then memoize

The meta-skill: **before writing a single line, say aloud which row you're in.** Naming the pattern collapses a scary blank-page problem into "oh, this is just the pick/don't-pick template with a sum at the leaf."

Chapter 37. The master template: pick / don't-pick

An enormous fraction of recursion problems — subsets, subset-sum, knapsack, coin change, partition, target-sum, "count ways" — are the *same* recursion: at each element you either **take it** or **skip it**, and recurse on the rest. Learn this one template deeply and you've learned a dozen problems.

```
def solve(i, state):
    if i == len(items):           # base: decided every element
        return leaf_value(state) # 1 if state is a valid full choice, else 0/...
    take = solve(i + 1, use(state, items[i])) # branch A: include items[i]
    skip = solve(i + 1, state)               # branch B: exclude items[i]
    return combine(take, skip)               # combine = + (count), or max/min (optimize), or list-merge (enumerate)
```

JavaScript:

```
function solve(i, state) {
  if (i === items.length) return leafValue(state); // 1 if valid full choice, else 0/...
  const take = solve(i + 1, use(state, items[i])); // branch A: include items[i]
  const skip = solve(i + 1, state); // branch B: exclude items[i]
  return combine(take, skip); // combine = + (count), max/min (optimize), or merge (enumerate)
}
```

Only three things change between problems: what **state** tracks, what the leaf returns, and how you **combine**. Watch the *same skeleton* solve four classic problems:

Count subsets — enumerate, combine by concatenation:

```
def subsets(i, xs, cur, out):
  if i == len(xs):
    out.append(cur[:]); return
  cur.append(xs[i]); subsets(i+1, xs, cur, out) # take
  cur.pop(); subsets(i+1, xs, cur, out) # skip
```

JavaScript:

```
function subsets(i, xs, cur, out) {
  if (i === xs.length) {
    out.push([...cur]); return;
  }
  cur.push(xs[i]); subsets(i + 1, xs, cur, out); // take
  cur.pop(); subsets(i + 1, xs, cur, out); // skip
}
```

Subset-sum (does any subset total T?) — combine by OR:

```
def can_sum(i, xs, target):
  # Precondition: xs holds NON-NEGATIVE numbers. The `target < 0` prune is only
  # valid because adding an element can't overshoot and later recover; with
  # negative values present, drop that prune (correctness) and lose the speedup.
  if target == 0: return True
  if i == len(xs) or target < 0: return False
  return can_sum(i+1, xs, target - xs[i]) or can_sum(i+1, xs, target) # take OR skip
```

JavaScript:

```
function canSum(i, xs, target) {
  // Precondition: xs contains NON-NEGATIVE numbers, so the `target < 0`
  // prune is valid (once you overshoot you can never come back).
  if (target === 0) return true;
  if (i === xs.length || target < 0) return false;
  return canSum(i + 1, xs, target - xs[i]) || canSum(i + 1, xs, target); // take OR skip
}
```

Count ways to reach a target — combine by addition:

```
def count_ways(i, xs, target):
  # Precondition: xs contains NON-NEGATIVE numbers, so the `target < 0`
  # prune is valid (once you overshoot you can never come back).
  if target == 0: return 1
  if i == len(xs) or target < 0: return 0
  return count_ways(i+1, xs, target - xs[i]) + count_ways(i+1, xs, target) # take + skip
```

JavaScript:

```
function countWays(i, xs, target) {
  // Precondition: xs contains NON-NEGATIVE numbers (validates the target < 0 prune).
  if (target === 0) return 1;
  if (i === xs.length || target < 0) return 0;
  return countWays(i + 1, xs, target - xs[i]) + countWays(i + 1, xs, target); // take + skip
}
```

0/1 knapsack (maximize value under weight W) — combine by max:

```
def knapsack(i, wts, vals, W):
    # Precondition: weights are POSITIVE and W >= 0.
    if i == len(wts) or W == 0: return 0
    skip = knapsack(i+1, wts, vals, W)
    take = 0
    if wts[i] <= W:
        take = vals[i] + knapsack(i+1, wts, vals, W - wts[i])
    return max(take, skip) # optimize -> max
```

JavaScript:

```
function knapsack(i, wts, vals, W) {
    // Precondition: weights are POSITIVE and W >= 0.
    if (i === wts.length || W === 0) return 0;
    const skip = knapsack(i + 1, wts, vals, W);
    let take = 0;
    if (wts[i] <= W) {
        take = vals[i] + knapsack(i + 1, wts, vals, W - wts[i]);
    }
    return Math.max(take, skip); // optimize -> max
}
```

Read those four side by side. The recursion is *identical*; only the leaf value and the combine operator (`concat` / `or` / `+` / `max`) differ. This is the clearest possible demonstration of the second of the four forces (Chapter 5b): **the problem's shape determines the branches; the question being asked determines the combine operator.** Here the shape (a sequence of take-or-skip decisions) is fixed, so all four algorithms branch identically — and only the *goal* changes, swapping `or` for `+` for `max` for `concat`. **This is the single highest-leverage realization in exam recursion:** most "choose a subset" problems are one template with a swapped combine operator. The *decision* variants — feasibility (`or`), counting (`+`), and optimization (`max/min`) — become efficient the moment you add memoization on the state `(i, target)` or `(i, W)`, because many paths collapse onto the same state (see Chapter 39). The *enumeration* variant is different: listing all subsets must produce up to 2^n answers, so its output alone is exponential and no memoization can make it polynomial. Memoization speeds up problems that *summarize* the choices into a single value, not those that must emit every choice.

One trap worth its own box: the coin-change variants. "Coin change" is not a single problem, and the *transition after taking a coin* changes with the rules. Confusing these is one of the most common DP mistakes. The cleanest way to remember them is by what the "take" move does:

Variant	Transition after taking a coin
0/1 combinations (each coin usable once)	advance to <code>i + 1</code>
Unbounded combinations (order-independent)	stay at <code>i</code> (may reuse this coin), iterate coins with a <code>start</code> index
Unbounded ordered sequences (order matters)	try every coin again at the next position
Minimum coins (unbounded)	<code>min(1 + best(rem - c))</code> over every coin <code>c</code>

The tells: **reuse allowed** → "take" stays at `i`; **reuse forbidden** → "take" advances to `i+1`. **Order should not matter** → iterate coins with a `start` index so you never revisit an earlier coin (this is what makes `{1,2}` and `{2,1}` the same); **order matters** → consider all coins at each position (making them different). Before coding any coin/change problem, decide which row you're in first — it fully determines the state and the loop structure.

Chapter 38. The other reusable templates

Combinations (choose `k`, order doesn't matter) — use a `start` index so you never look back, which avoids duplicates:

```
def combine(start, n, k, cur, out):
    if len(cur) == k:
        out.append(cur[:]); return
    for x in range(start, n + 1):
        cur.append(x)
        combine(x + 1, n, k, cur, out) # x+1: don't reuse earlier numbers
    cur.pop()
```

JavaScript:

```
function combine(start, n, k, cur, out) {
  if (cur.length === k) {
    out.push([...cur]); return;
  }
  for (let x = start; x <= n; x++) {
    cur.push(x);
    combine(x + 1, n, k, cur, out); // x+1: don't reuse earlier numbers
    cur.pop();
  }
}
```

Permutations (order matters) — swap-in each unused element:

```
def permute(xs, start, out):
  if start == len(xs):
    out.append(xs[:]); return
  for i in range(start, len(xs)):
    xs[start], xs[i] = xs[i], xs[start] # choose
    permute(xs, start + 1, out)
    xs[start], xs[i] = xs[i], xs[start] # undo
```

JavaScript:

```
function permute(xs, start, out) {
  if (start === xs.length) {
    out.push([...xs]); return;
  }
  for (let i = start; i < xs.length; i++) {
    [xs[start], xs[i]] = [xs[i], xs[start]]; // choose
    permute(xs, start + 1, out);
    [xs[start], xs[i]] = [xs[i], xs[start]]; // undo
  }
}
```

Grid DFS / flood fill — recurse into four neighbors, mark visited:

```
def dfs(grid, r, c, seen):
  if not (0 <= r < len(grid) and 0 <= c < len(grid[0])): return
  if (r, c) in seen or grid[r][c] == 0: return
  seen.add((r, c))
  for dr, dc in ((1,0), (-1,0), (0,1), (0,-1)):
    dfs(grid, r + dr, c + dc, seen)
```

JavaScript:

```
function dfs(grid, r, c, seen) {
  if (r < 0 || c < 0 || r >= grid.length || c >= grid[0].length) return;
  if (seen.has(r + "," + c) || grid[r][c] === 0) return;
  seen.add(r + "," + c);
  for (const [dr, dc] of [[1,0], [-1,0], [0,1], [0,-1]]) {
    dfs(grid, r + dr, c + dc, seen);
  }
}
```

Two-string / two-pointer DP — compare current characters, branch on match:

```
def edit_distance(a, b, i, j):
  if i == 0: return j # insert remaining of b
  if j == 0: return i # delete remaining of a
  if a[i-1] == b[j-1]:
    return edit_distance(a, b, i-1, j-1) # free match
  return 1 + min(edit_distance(a, b, i-1, j), # delete
                  edit_distance(a, b, i, j-1), # insert
                  edit_distance(a, b, i-1, j-1)) # replace
```

JavaScript:

```
function editDistance(a, b, i, j) {
  if (i === 0) return j;      // insert remaining of b
  if (j === 0) return i;      // delete remaining of a
  if (a[i-1] === b[j-1]) {
    return editDistance(a, b, i-1, j-1);    // free match
  }
  return 1 + Math.min(editDistance(a, b, i-1, j), // delete
                      editDistance(a, b, i, j-1), // insert
                      editDistance(a, b, i-1, j-1)); // replace
}
```

Between the pick/don't-pick template of Chapter 37 and these five, you can express the large majority of exam recursion problems. Memorize their *shapes*, not their exact lines.

Chapter 39. The recursion → memoization → tabulation pipeline

This is the transformation exams test more than any other, and the single most reliable way to solve a DP problem: **never start with tabulation. Start with recursion, then mechanically upgrade.** Three stages, always in this order.

Stage 1 — Write the plain recursion. Identify the state (the minimal set of parameters that fully describes a subproblem), the choices, the base cases, and the recurrence. Don't worry about speed yet. This is where all the thinking happens; the rest is mechanical.

Stage 2 — Add memoization (top-down DP). If the recursion revisits states (draw two levels of the tree and check for repeats), cache results keyed by the state. In Python this is often a one-line change: `@lru_cache`. The recurrence is unchanged; you've just stopped recomputing.

Stage 3 — Convert to tabulation (bottom-up DP), if needed. Replace the recursion with a table filled in an order that guarantees every dependency is ready before it's used. This removes call-stack overhead and the risk of stack overflow, and makes space optimization obvious.

Watch all three stages on **climbing stairs** (ways to climb n steps taking 1 or 2 at a time):

```
# Stage 1 — plain recursion. State = n. Recurrence: ways(n) = ways(n-1) + ways(n-2).
def ways(n):
    if n <= 1: return 1
    return ways(n-1) + ways(n-2)    # O(φ^n) — exponential

# Stage 2 — memoized (top-down). Same recurrence, cached. O(n).
from functools import lru_cache
@lru_cache(maxsize=None)
def ways(n):
    if n <= 1: return 1
    return ways(n-1) + ways(n-2)

# Stage 3 — tabulated (bottom-up). O(n) time, O(1) space after optimization.
def ways(n):
    if n <= 1: return 1
    a, b = 1, 1    # ways(0), ways(1)
    for _ in range(2, n + 1):
        a, b = b, a + b    # slide the window forward
    return b
```

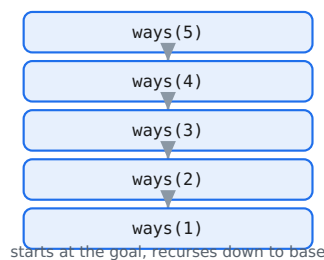
JavaScript:


```
// Stage 1 – plain recursion. State = n. ways(n) = ways(n-1) + ways(n-2).
function ways(n) {
  if (n <= 1) return 1;
  return ways(n - 1) + ways(n - 2);    // O(phi^n) – exponential
}

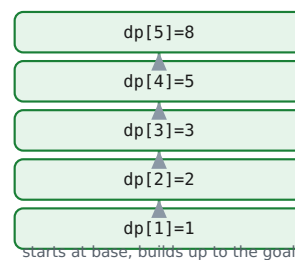
// Stage 2 – memoized (top-down). Same recurrence, cached. O(n).
const waysMemo = (() => {
  const memo = new Map();
  return function ways(n) {
    if (n <= 1) return 1;
    if (memo.has(n)) return memo.get(n);
    const v = ways(n - 1) + ways(n - 2);
    memo.set(n, v);
    return v;
  };
})();

// Stage 3 – tabulated (bottom-up). O(n) time, O(1) space.
function waysTab(n) {
  if (n <= 1) return 1;
  let a = 1, b = 1;    // ways(0), ways(1)
  for (let k = 2; k <= n; k++) [a, b] = [b, a + b]; // slide the window forward
  return b;
}
```

Top-down (memoized recursion)



Bottom-up (tabulation)



Same recurrence $ways(n) = ways(n-1) + ways(n-2)$; only the evaluation order differs.

The same recurrence evaluated top-down (memoized recursion) and bottom-up (tabulation).

The recurrence $ways(n) = ways(n-1) + ways(n-2)$ is *identical* across all three. That's the point: **tabulation is not a different idea, it's the same recurrence evaluated in a different order**. The direction of dependency in the recursion (n depends on $n-1$ and $n-2$) tells you the fill order of the table (small to large). This mapping is completely mechanical once you see it:

- Recursion parameter $\rightarrow$ table index/dimension.
- Base case $\rightarrow$ table initialization.
- Recursive formula $\rightarrow$ the table update rule.
- "Which smaller calls does this make?" $\rightarrow$ "which earlier cells must be filled first?"

The conditions that make this pipeline apply — and that exams ask you to state — are that the answer is **determined entirely by a well-defined state** and that **the same states are revisited** (overlapping subproblems); for *optimization* problems the recurrence must additionally have **optimal substructure** (the best answer is built from best answers to subproblems). When these hold, the recursion is a DP in disguise. But "DP" does not automatically mean "polynomial": the running time is (number of *reachable* states) $\times$ (transition work per state), which is polynomial *only when both of those quantities are polynomial*. Some correct DPs still have exponentially many states — bitmask DP over subsets has 2^n states, for instance — so memoization removes the *recomputation*, not necessarily the exponential. The pipeline turns exponential *recomputation* into work proportional to the state space; whether that is fast depends on how large the state space is.

Chapter 40. How to define the "state" (the make-or-break DP skill)

Every DP is only as good as its state. The state is the smallest description of a subproblem such that its answer doesn't depend on how you arrived there. Getting it right is 80% of DP. Ask:

- **What varies between recursive calls?** Those varying quantities are your state parameters. In `fib(n)` it's just `n`. In LCS it's the two positions `(i, j)`. In knapsack it's `(index, remaining_capacity)`.
- **Is anything redundant?** If two parameters always move together, collapse them. Fewer dimensions means a smaller table and faster code.
- **Does the answer for a state depend only on the state, not the path taken to reach it?** If arriving at `(i, w)` two different ways can give different valid answers, the state is *incomplete* — you've forgotten a parameter. This is the most common DP bug.

A reliable procedure: write the recursion first with whatever parameters feel natural; then examine the values that actually *vary* across the recursive calls. From those, choose the **smallest independent set that completely determines the answer**. Three refinements matter in practice: fixed inputs (the array being searched, the target string) are constant across all calls, so capture them *outside* the state rather than in the memo key; redundant parameters that always move in lockstep with another should be dropped; and a parameter that's derivable from the others (like a running index that equals `len - remaining`) is not part of the state. The memo key is the tuple of the *remaining* state parameters, and the table has one dimension per such parameter. Number of states × work per state gives you the time bound for free (Chapter 35).

Checkpoint — end of Part IX. You should now be able to: map a problem's signal words to a candidate technique and then verify it; wield the pick/don't-pick template across many problems; distinguish the four coin-change variants; and run the recursion → memoization → tabulation pipeline, defining the state correctly.

Part X — Worked Exam Problems

Learning objectives. See the full method applied end to end on exam-style problems, defeat common MCQ traps, and follow a fixed ten-minute protocol under time pressure.

Theory sticks only when you watch it applied end to end. Each problem below is solved the way you should solve it under exam pressure: **name the pattern, define the state, write the recurrence, code it, then state complexity.** Follow the reasoning, not just the code.

Chapter 41. Type A — "Write the recursion" problems

Problem 1. Print all subsequences of a string whose ASCII sum equals K. *Pattern:* pick / don't-pick with a running sum (Ch. 37). *State:* index + accumulated sum. *Reasoning:* at each character, include it (add its value) or skip it; at the end, emit if the running sum equals K.

```
def subseq_sum_k(s, i, cur, total, K):
    if i == len(s):
        if total == K and cur:
            print(cur)
        return
    subseq_sum_k(s, i+1, cur + s[i], total + ord(s[i]), K) # pick
    subseq_sum_k(s, i+1, cur, total, K) # skip
```

JavaScript:

```
function subseqSumK(s, i, cur, total, K) {
    if (i === s.length) {
        if (total === K && cur) console.log(cur);
        return;
    }
    subseqSumK(s, i+1, cur + s[i], total + s.charCodeAt(i), K); // pick
    subseqSumK(s, i+1, cur, total, K); // skip
}
```

Complexity: $O(2^n)$ subsequences, $O(n)$ to build each string $\rightarrow O(n \cdot 2^n)$ time, $O(n)$ stack depth.

Problem 2. Generate all balanced parentheses of n pairs. *Pattern:* constrained pick (backtracking). *State:* how many (and) used so far. *Key insight — the pruning rule:* you may add (while `open < n`, and) only while `close < open`. Those two constraints are what keep every result valid.

```
def gen_parens(n, cur="", open_=0, close=0, out=None):
    if out is None: out = []
    if len(cur) == 2*n:
        out.append(cur); return out
    if open_ < n: gen_parens(n, cur+"(", open_+1, close, out)
    if close < open_: gen_parens(n, cur+")", open_, close+1, out)
    return out
```

JavaScript:

```
function genParens(n, cur = "", open = 0, close = 0, out = []) {
    if (cur.length === 2 * n) {
        out.push(cur); return out;
    }
    if (open < n) genParens(n, cur + "(", open + 1, close, out);
    if (close < open) genParens(n, cur + ")", open, close + 1, out);
    return out;
}
```

Complexity: the count of results is the nth Catalan number $\approx 4^n/(n^{1.5})$; work is $O(n)$ per result.

Problem 3. Tower of Hanoi — count and print the moves. *Pattern:* two-calls-on-(n-1) (Ch. 17). *Recurrence:* $\text{moves}(n) = 2 \cdot \text{moves}(n-1) + 1$, so $\text{moves}(n) = 2^n - 1$. *Reasoning:* move n-1 disks aside, move the biggest, move n-1

back.

```
def hanoi(n, src, dst, aux):
    """Precondition: n >= 0. Returns the number of moves made and prints each."""
    def move(k, frm, to, via):
        if k == 0:
            return 0
        left = move(k - 1, frm, via, to) # move k-1 aside
        print(f"{frm}->{to}") # move the biggest disk
        right = move(k - 1, via, to, frm) # bring k-1 back on top
        return left + 1 + right
    return move(n, src, dst, aux)
```

JavaScript:

```
function hanoi(n, src, dst, aux) {
    // Precondition: n >= 0. Returns the number of moves made and prints each.
    function move(k, frm, to, via) {
        if (k === 0) return 0;
        const left = move(k - 1, frm, via, to); // move k-1 aside
        console.log(`${frm}->${to}`); // move the biggest disk
        const right = move(k - 1, via, to, frm); // bring k-1 back on top
        return left + 1 + right;
    }
    return move(n, src, dst, aux);
}
```

(Note: the move count is returned, not accumulated in a shared list. A mutable default argument like `count=[0]` would be a bug here — the *same* list persists across separate top-level calls to `hanoi`, so counts would leak between runs. Returning the value avoids that trap entirely.) Complexity: $\Theta(2^n)$ time (decreasing form, $a=2 \rightarrow$ exponential), $\Theta(n)$ stack depth.

Chapter 42. Type B — "Optimize with DP" problems

Problem 4. Coin change — fewest coins to make amount A (unlimited coins). *Pattern:* optimization over choices $\rightarrow$ DP (Ch. 39). *State:* remaining amount. *Recurrence:* $\text{best}(A) = 1 + \min \text{ over coins } c \text{ of } \text{best}(A-c)$. *Pipeline:* recursion $\rightarrow$ memoize on the amount.

```
from functools import lru_cache
def min_coins(coins, A):
    """Precondition: every coin is a POSITIVE integer and A >= 0.
    A zero coin makes best(rem - 0) recurse on the same state forever."""
    @lru_cache(maxsize=None)
    def best(rem):
        if rem == 0: return 0
        if rem < 0: return float('inf')
        return min((1 + best(rem - c) for c in coins), default=float('inf'))
    r = best(A)
    return r if r != float('inf') else -1
```

JavaScript:

```
// JavaScript: cache with a Map instead of a decorator.
const minCoins = (coins, A) => {
    const memo = new Map();
    function best(rem) {
        if (rem === 0) return 0;
        if (rem < 0) return Infinity;
        if (memo.has(rem)) return memo.get(rem);
        let m = Infinity;
        for (const c of coins) m = Math.min(m, 1 + best(rem - c));
        memo.set(rem, m);
        return m;
    }
    const r = best(A);
    return r !== Infinity ? r : -1;
};
```

Complexity: states = A, work per state = $O(\#coins) \rightarrow O(A \cdot \#coins)$ time, $O(A)$ space. (This is exactly the "states $\times$ work" rule from Ch. 35.)

Problem 5. Longest common subsequence of strings X, Y. Pattern: two-pointer DP (Ch. 38). State: the pair of positions (i, j). Recurrence: if $X[i-1] == Y[j-1]$ then $1 + \text{LCS}(i-1, j-1)$, else $\max(\text{LCS}(i-1, j), \text{LCS}(i, j-1))$.

```
from functools import lru_cache
def lcs(X, Y):
    @lru_cache(maxsize=None)
    def f(i, j):
        if i == 0 or j == 0: return 0
        if X[i-1] == Y[j-1]: return 1 + f(i-1, j-1)
        return max(f(i-1, j), f(i, j-1))
    return f(len(X), len(Y))
```

JavaScript:

```
const lcs = (a, b) => {
    const memo = new Map();
    function solve(i, j) {
        if (i === 0 || j === 0) return 0;
        const key = i + "," + j;
        if (memo.has(key)) return memo.get(key);
        let val;
        if (a[i-1] === b[j-1]) val = 1 + solve(i-1, j-1);
        else val = Math.max(solve(i-1, j), solve(i, j-1));
        memo.set(key, val);
        return val;
    }
    return solve(a.length, b.length);
};
```

Complexity: states = $m \cdot n$, $O(1)$ work each $\rightarrow O(mn)$ time and space. The tabulated version fills an $(m+1) \times (n+1)$ grid in the same asymptotics with $O(\min(m, n))$ space if you keep only two rows.

Problem 6. 0/1 knapsack. Already shown in Ch. 37 as pure recursion; the DP upgrade is one decorator plus memoizing on (i, W). States = $n \cdot W$, $O(1)$ each $\rightarrow O(nW)$ time and space. State this "pseudo-polynomial" caveat on exams: it's polynomial in the value W, not in the number of bits of W.

Chapter 43. Type C — "Analyze this recursion" problems

These give you code and ask for time/space. Apply the 30-second method from Chapter 35.

Problem 7. What is the complexity of the following?

```
def f(n):
    if n <= 1: return
    f(n // 2)
    f(n // 2)
    for i in range(n): pass # O(n) work
```

JavaScript:

```
function f(n) {
    if (n <= 1) return;
    f(Math.floor(n / 2));
    f(Math.floor(n / 2));
    for (let i = 0; i < n; i++) { /* O(n) work */ }
}
```

Reasoning: two calls on $n/2$, plus $O(n)$ work $\rightarrow T(n) = 2T(n/2) + O(n)$. Master Theorem: $a=2$, $b=2$, $c=\log_2 2=1$, $f(n)=n^1$, Case 2 $\rightarrow \Theta(n \log n)$.

Problem 8. And this one?

```
def g(n):
    if n <= 1: return
    g(n - 1)
    g(n - 1)
```

JavaScript:

```
function g(n) {
    if (n <= 1) return;
    g(n - 1);
    g(n - 1);
}
```

Reasoning: two calls on $n-1$, $O(1)$ work $\rightarrow T(n) = 2T(n-1) + O(1)$. Decreasing form with $a=2 \rightarrow \Theta(2^n)$. Contrast Problem 7: same "two calls," but subtracting instead of dividing flips $n \log n$ into exponential. This is *the* comparison examiners use to test whether you understand recurrences rather than memorized answers.

Problem 9. Space of an inorder traversal of a binary search tree? *Reasoning:* the traversal visits all n nodes ($\Theta(n)$ time), but the stack only ever holds one root-to-leaf path at a time, so the stack space equals the tree's **height**. This is where the wording matters: if the tree is *guaranteed balanced*, its height is $\Theta(\log n)$, so the stack is $\Theta(\log n)$ in the worst case. If the tree is an *arbitrary or degenerate* BST (values inserted in sorted order produce a linked-list shape), the height is $\Theta(n)$, so the stack is $\Theta(n)$. **Answer: $\Theta(\text{height})$ — that's $\Theta(\log n)$ for a balanced tree, $\Theta(n)$ for a degenerate one.** Stating "balanced" changes the worst case; don't conflate the two scenarios.

Chapter 44. MCQ traps and how to defeat them

Multiple-choice recursion questions are built around a small set of predictable misconceptions. Knowing the traps is worth several marks.

Trap: "number of calls = space." As covered, stack space is the *depth*, not the count. Naive fib: exponential time, but only $\Theta(n)$ space. Pick the depth-based answer.

Trap: hidden slicing cost. If code slices `xs[1:]` or concatenates strings inside the recursion, the per-level work is $O(n)$, not $O(1)$; a "linear-looking" recursion is secretly $O(n^2)$. Scan for copies before answering.

Trap: base case placement. A question like "how many times does the base case execute in `fib(n)`?" — the number of *leaves* in the call tree — equals `fib(n+1)` for the standard definition, not `n`. Draw two levels and count if unsure.

Trap: off-by-one in the recurrence. For $T(n)=T(n-1)+n$ with $T(0)=1$, the sum is $1+2+\dots+n$, giving $\Theta(n^2)$, *not* $\Theta(n)$. Distinguish "adds constant per level" (linear total) from "adds n per level" (quadratic total).

Trap: dividing vs. decreasing look-alikes. $T(n)=T(n/2)+1$ is $\Theta(\log n)$; $T(n)=T(n-1)+1$ is $\Theta(n)$; $T(n)=2T(n/2)+n$ is $\Theta(n \log n)$; $T(n)=2T(n-1)+1$ is $\Theta(2^n)$. Examiners place these four next to each other. The tell is *always* the shrink operation.

Trap: "does this terminate?" A recursion terminates iff every path strictly reduces toward a reachable base case. Watch for calls like `f(n)` (no shrink), `f(n/2)` when n never reaches the base (e.g. missing the `n==0` case for even inputs), or a base case that some inputs skip over (recurring `n-2` from an odd start toward an even base).

Chapter 45. A ten-minute exam protocol

When a recursion problem appears on a timed exam, run this fixed sequence rather than improvising:

1. **Classify (30s).** Match the problem to a row in the Chapter 36 table. Say the pattern name.
2. **Define the state (1 min).** Write the parameters that vary. This is your memo key and your function signature.
3. **Write base cases (1 min).** The smallest inputs and their direct answers. Make sure every path reaches one.
4. **Write the recurrence (2 min).** One line: how the answer combines the smaller answers. For "choose" problems, that's take-vs-skip; for "two strings," match-vs-mismatch; for divide-and-conquer, split-and-combine.
5. **Code it (3 min).** Translate the recurrence directly. Trust the recursive call.
6. **Optimize if asked (1 min).** Add `@lru_cache` or convert to a table using the mechanical mapping (Ch. 39).
7. **State complexity (1 min).** States $\times$ work for time; table size + depth for space. Check for the four hidden-cost traps.

Notice that only step 4 requires genuine insight; the rest is procedure. That's the goal of this whole book: **reduce**

recursion from a leap of inspiration to a short, reliable checklist.

Checkpoint — end of Part X. *You should now be able to: work an unfamiliar problem end to end (name the pattern, define the state, write the recurrence, code it, analyze it); recognize and defeat common MCQ traps; and follow the ten-minute exam protocol under time pressure.*

Part XI — Practice & Reference

Chapter 46. Language notes

The ideas are universal, but a few practical facts vary by language, and they matter when your recursion is deep.

Python (CPython) commonly starts with a recursion limit around 1000, which you can read with `sys.getrecursionlimit()`; exceeding it raises `RecursionError`. This is implementation- and environment-dependent — it is *not* a guaranteed count of safe native stack frames, and it's not fixed at exactly 1000 everywhere. You can raise it with `sys.setrecursionlimit(n)`, but setting it too high risks a hard interpreter crash rather than a catchable error, because you can exhaust the underlying C stack. Python performs **no** tail-call optimization by design, so prefer converting deep linear recursions to loops. Use `functools.lru_cache` for instant memoization.

JavaScript, C, C++, Java each have their own stack limits and no reliable guaranteed TCO in mainstream engines/compilers today (the ECMAScript spec includes proper tail calls, but most engines don't implement them). Treat deep recursion as a liability and reach for explicit stacks or iteration when input size is unbounded.

Functional languages (Scheme, Haskell, Erlang, Elixir, many Lisps) guarantee or strongly favor tail-call optimization, so tail recursion is the *idiomatic* loop; deep tail-recursive functions run in constant stack space. Haskell's laziness also makes corecursion and infinite structures natural.

The portable lesson: write the recursion for clarity first; if depth could be large, know your language's stack behavior and convert accordingly.

Chapter 47. A decision guide — should this be recursive, and which kind?

Use this as a quick flowchart in prose. Ask, in order:

Is the data self-similar (tree, nested list, JSON, filesystem, DOM)? → Yes: use **structural recursion** mirroring the data (Ch. 15–16). This is the strongest signal; recursion will be cleaner than any loop.

Do I need to *explore choices* — subsets, permutations, placements, paths? → Yes: use **combinatorial recursion / backtracking** (Ch. 18–19). Add pruning to stay fast.

Can I split the input into halves and merge? → Yes: use **divide and conquer** (Ch. 21). Expect fast ($\log$ or $n \log n$) performance.

Does the recursion revisit the same subproblem repeatedly (overlapping subproblems)? → Yes: **memoize**, i.e. dynamic programming (Ch. 22–23).

Is it a simple linear march (sum a list, count down) and possibly very deep? → Consider a **loop** instead, or an accumulator/tail style, to avoid stack cost (Ch. 24, 30).

Are two concepts defined in terms of each other (grammar rules, even/odd)? → **Mutual recursion** (Ch. 25).

If none of these fire, the problem may simply not be recursive — and forcing recursion where a loop is natural is its own kind of mistake.

Chapter 48. The problem gym

Practice is what converts understanding into instinct. Work these roughly in order; each maps to a chapter. Try to (a) state the contract, (b) name the base case, and (c) identify the "smaller" *before* you write code.

Warm-ups (numbers & lists, Ch. 13–14): sum of a list; product of a list; count occurrences of a value; find the maximum; reverse a string; check a palindrome; n th triangular number; count digits of an integer; power of two check.

Structures (Ch. 15–16): height of a tree; count leaves; mirror/invert a binary tree; sum a nested list; count files in a directory tree; validate a binary search tree; find the deepest value; serialize a tree to a string.

Branching & backtracking (Ch. 17–19): print all subsets; all permutations; all combinations of k from n ; generate balanced parentheses; letter combinations of a phone number; N-Queens; Sudoku solver; word search in a grid; subset-sum; combination-sum; rat-in-a-maze.

Divide & conquer (Ch. 21): binary search; merge sort; quicksort; count inversions; maximum subarray (divide-and-conquer version); fast exponentiation; majority element.

DP via recursion (Ch. 22-23): Fibonacci memoized; climbing stairs; coin change (min coins and number of ways); longest common subsequence; edit distance; 0/1 knapsack; grid unique paths; house robber; longest increasing subsequence; partition equal subset sum.

Advanced (Ch. 24-30): tail-recursive factorial with accumulator; mutual-recursion even/odd; Ackermann for small inputs; iterative inorder traversal with an explicit stack; flatten deeply nested structure; recursive-descent calculator.

For each solved problem, do one extra thing: draw its call tree for a small input. That habit, more than anything, welds the intuition in place.

How to practice: the seven-rung ladder

Don't jump straight to writing solutions from scratch — that's the hardest rung and it's demoralizing to start there. Climb these seven rungs in order for each new pattern. Each rung isolates one sub-skill.

1. **Trace.** Given working code and an input, predict the output *by hand* before running it.
2. **Fill the base case.** Given a recursion with the base case blanked out, supply it.
3. **Fill the recursive call.** Given the base case and combine step, supply the recursive call(s).
4. **Fix broken recursion.** Given code with one bug (no shrink, wrong base, missing return), find and fix it.
5. **Derive from a value recurrence.** Given a *value* recurrence — such as $f(n) = n + f(n-1)$, or a plain-English rule like "the answer for n is the answer for $n-1$ plus n " — write the recursive function. (Note: a *running-time* recurrence $T(n)=\dots$ does *not* uniquely determine code — many different algorithms share the same $T(n)$ — so we reserve $T(n)$ for complexity analysis in Part VIII and use value recurrences here.)
6. **Write from blank.** Given only the problem statement, write the whole thing.
7. **Optimize & convert.** Add memoization, then convert to iteration.

A mastery check that beats "what does `f(4)` print": ask yourself, "Without running it, what does `f(4)` trust `f(3)` to return?" If you can answer that for any recursion, you have internalized the leap of faith.

Rung 1 — Trace (predict the output)

For each, write the return value before reading on.

```
def mystery(n):
    if n == 0: return 0
    return n + mystery(n - 1)
# mystery(4) = ?      (trusts mystery(3) to return 6, so 4 + 6 = 10)

def guess(s):
    if len(s) <= 1: return s
    return guess(s[1:]) + s[0]
# guess("abc") = ?    (trusts guess("bc")="cb", so "cb"+"a" = "cba" — it reverses)

def what(n):
    if n < 2: return n
    return what(n - 1) + what(n - 2)
# what(4) = ?         (the Fibonacci recurrence: 0,1,1,2,3 → what(4) = 3)
```

JavaScript:

```
function mystery(n) {
    if (n === 0) return 0;
    return n + mystery(n - 1);
}
// mystery(4) = ?    (trusts mystery(3) to return 6, so 4 + 6 = 10)

function guess(s) {
    if (s.length <= 1) return s;
    return guess(s.slice(1)) + s[0];
}
// guess("abc") = ?  (trusts guess("bc")="cb", so "cb"+"a" = "cba" — it reverses)

function what(n) {
    if (n < 2) return n;
    return what(n - 1) + what(n - 2);
}
// what(4) = ?      (the Fibonacci recurrence: 0,1,1,2,3 → what(4) = 3)
```

Rung 4 — Fix broken recursion (find the one bug)

```
# BUG 1 – infinite recursion. Why? Fix it.
def countdown(n):
    if n == 0: return
    print(n)
    countdown(n)          # ← never shrinks; should be countdown(n - 1)

# BUG 2 – wrong answer for empty list. Why? Fix it.
def total(xs):
    if len(xs) == 1: return xs[0]    # ← base case can't handle []; use `if not xs: return 0`
    return xs[0] + total(xs[1:])

# BUG 3 – returns None. Why? Fix it.
def find_max(xs):
    if len(xs) == 1: return xs[0]
    max(xs[0], find_max(xs[1:]))    # ← missing `return` in front of max(...)
```

JavaScript:

```
// BUG 1 – infinite recursion. Why? Fix it.
function countdown(n) {
    if (n === 0) return;
    console.log(n);
    countdown(n);          // ← never shrinks; should be countdown(n - 1)
}

// BUG 2 – wrong answer for empty array. Why? Fix it.
function total(xs) {
    if (xs.length === 1) return xs[0]; // ← base can't handle []; use `if (!xs.length) return 0`
    return xs[0] + total(xs.slice(1));
}

// BUG 3 – returns undefined. Why? Fix it.
function findMax(xs) {
    if (xs.length === 1) return xs[0];
    Math.max(xs[0], findMax(xs.slice(1))); // ← missing `return` in front of Math.max(...)
}
```

Training yourself to spot these three failure modes — no shrink, base case that misses an input, and a dropped `return` — catches the large majority of real recursion bugs.

Complete ladders: one full climb per family

The rungs above are illustrated with a few examples each; here is a *complete* seven-rung climb for each major family, so you can practice the whole progression on one problem before moving to the next family. Do them with pen and paper first.

Family 1 — Linear recursion (worked ladder: sum of a list)

Rung 1 (trace). `s([4,1,7])` — what does it return, and what does it trust `s([1,7])` to return? (Trusts `s([1,7])=8`, so `4+8=12`.)

Rung 2 (fill the base case).

```
def s(xs, i=0):
    if ____:          # ← fill: i == len(xs)
        return ____  # ← fill: 0
    return xs[i] + s(xs, i+1)
```

JavaScript:

```
function s(xs, i = 0) {
    if (____) {        // ← fill: i === xs.length
        return ____;   // ← fill: 0
    }
    return xs[i] + s(xs, i + 1);
}
```

Rung 3 (fill the recursive call).

```
def s(xs, i=0):
    if i == len(xs): return 0
    return xs[i] + ____ # ← fill: s(xs, i+1)
```

JavaScript:

```
function s(xs, i = 0) {
    if (i === xs.length) return 0;
    return xs[i] + ____; // ← fill: s(xs, i + 1)
}
```

Rung 4 (debug). This never terminates — why?

```
def s(xs, i=0):
    if i == len(xs): return 0
    return xs[i] + s(xs, i) # BUG: index never advances → use i+1
```

JavaScript:

```
function s(xs, i = 0) {
    if (i === xs.length) return 0;
    return xs[i] + s(xs, i); // BUG: index never advances → use i + 1
}
```

Rung 5 (derive from a value recurrence). Given `s(xs, i) = xs[i] + s(xs, i+1)` with `s(xs, len)=0`, write the function. (That's the Rung 3 answer.)

Rung 6 (write from blank). "Return the sum of a list of numbers, recursively, without slicing." Write it from scratch.

Rung 7 (optimize & convert). It's already O(n) and single-branch; convert to a loop: `total=0; for x in xs: total+=x; return total`. No memoization needed (no overlapping subproblems).

Family 2 — Trees (worked ladder: max value in a binary tree)

Precondition note: the public call assumes a **non-empty** tree (the maximum of nothing is undefined). We handle empty *subtrees* internally by returning `float('-inf')` — a sentinel that is the identity for `max`, so a missing child never wins. If the whole input could be empty, check for that in a wrapper and raise or return `None` rather than handing back `-inf`.

Rung 1 (trace). For a tree with root 4, left subtree max 9, right subtree max 6: `tmax(root)` trusts `tmax(left)=9` and `tmax(right)=6`, returns `max(4,9,6)=9`.

Rung 2 (base case).

```
def tmax(node):
    if node is None:
        return ____ # ← fill: float('-inf') (sentinel for an empty subtree)
    return max(node.value, tmax(node.left), tmax(node.right))
```

JavaScript:

```
function tmax(node) {
    if (node === null) {
        return ____; // ← fill: -Infinity (sentinel for an empty subtree)
    }
    return Math.max(node.value, tmax(node.left), tmax(node.right));
}
```

Rung 3 (recursive call).

```
def tmax(node):
    if node is None: return float('-inf')
    return max(node.value, ____, ____ ) # ← tmax(node.left), tmax(node.right)
```

JavaScript:

```
function tmax(node) {
  if (node === null) return -Infinity;
  return Math.max(node.value, ____, ____); // <- tmax(node.left), tmax(node.right)
}
```

Rung 4 (debug). Why does this crash on a leaf?

```
def tmax(node):
  if node.left is None: return node.value # BUG: assumes left exists;
  return max(node.value, tmax(node.left), tmax(node.right))
# Fix: base case must test `node is None`, not `node.left is None`.
```

JavaScript:

```
function tmax(node) {
  if (node.left === null) return node.value; // BUG: assumes left exists;
  return Math.max(node.value, tmax(node.left), tmax(node.right));
}
// Fix: base case must test `node === null`, not `node.left === null`.
```

Rung 5 (derive). From "the max of a tree is the largest of its value and the maxes of its two subtrees; the max of an empty tree is $-\infty$," write it. (Rung 3 answer.)

Rung 6 (write from blank). "Return the largest value in a binary tree." From scratch.

Rung 7 (optimize & convert). Already $O(n)$, depth $O(h)$ stack. To avoid recursion on a very deep tree, convert to an explicit-stack DFS (Chapter 30). No memoization (each node is distinct).

Family 3 — Backtracking (worked ladder: all subsets)

Rung 1 (trace). `subsets([1,2])` — at index 0 you branch include-1 / exclude-1; each branches again on 2. Predict the four leaves: `[1,2], [1], [2], []`.

Rung 2 (base case).

```
def subsets(xs, i=0, cur=None, out=None):
  if cur is None: cur, out = [], []
  if ____: # <- fill: i == len(xs)
    out.append(cur[:]); return out
  ...
```

JavaScript:

```
function subsets(xs, i = 0, cur = [], out = []) {
  if (____) { // <- fill: i == xs.length
    out.push([...cur]); return out;
  }
  // ...
}
```

Rung 3 (recursive calls).

```
cur.append(xs[i]); ____ # <- subsets(xs, i+1, cur, out) (include)
cur.pop(); ____ # <- subsets(xs, i+1, cur, out) (exclude)
return out
```

JavaScript:

```
cur.push(xs[i]); ____; // <- subsets(xs, i+1, cur, out) (include)
cur.pop(); ____; // <- subsets(xs, i+1, cur, out) (exclude)
return out;
```

Rung 4 (debug). Why do all subsets come out identical (all equal to the full set)?

```
out.append(cur) # BUG: appends a reference; later mutations change it.
# Fix: out.append(cur[:]) — append a COPY (the classic aliasing bug of Ch.10c).
```

JavaScript:

```
out.push(cur);           // BUG: pushes a reference; later mutations change it.
// Fix: out.push([...cur]) - push a COPY (the classic aliasing bug of Ch.10c).
```

Rung 5 (derive). From "at each element, branch on take vs. skip; record cur at the end," write it.

Rung 6 (write from blank). "Print all subsets of a list." From scratch.

Rung 7 (optimize & convert). Output is inherently 2^n , so no asymptotic speedup; the point of practice is the choose/explore/undo shape and remembering to append a copy.

Family 4 — Divide and conquer (worked ladder: binary search)

Rung 1 (trace). `bsearch([1,3,5,7,9], 7)` — mid is 5 (index 2), $7 > 5$, so it trusts the recursion on the right half `[7,9]` to find 7. Predict: index 3.

Rung 2 (base case).

```
def bsearch(xs, t, lo, hi):
    if ____:                # ← fill: lo > hi    (empty range → not found)
        return -1
    ...
```

JavaScript:

```
function bsearch(a, t, lo, hi) {
    if (____) {              // ← fill: lo > hi    (empty range → not found)
        return -1;
    }
    // ...
}
```

Rung 3 (recursive calls).

```
mid = (lo + hi) // 2
if xs[mid] == t: return mid
if t < xs[mid]: return ____    # ← bsearch(xs, t, lo, mid-1)
return ____                  # ← bsearch(xs, t, mid+1, hi)
```

JavaScript:

```
const mid = Math.floor((lo + hi) / 2);
if (a[mid] === t) return mid;
if (t < a[mid]) return ____;    // ← bsearch(a, t, lo, mid-1)
return ____;                  // ← bsearch(a, t, mid+1, hi)
```

Rung 4 (debug). Why does this loop forever on a miss?

```
if t < xs[mid]: return bsearch(xs, t, lo, mid)    # BUG: should be mid-1
# Using `mid` (not mid-1) can leave the range unchanged → no progress.
```

JavaScript:

```
if (t < a[mid]) return bsearch(a, t, lo, mid);    // BUG: should be mid-1
// Using `mid` (not mid-1) can leave the range unchanged → no progress.
```

Rung 5 (derive). From $T(n) = T(n/2) + O(1)$ plus the value rule "compare the middle; recurse on the half that could contain t," write it. (Note: the value rule, not $T(n)$, is what determines the code — Rung 5's lesson.)

Rung 6 (write from blank). "Find a target's index in a sorted array in $O(\log n)$." From scratch.

Rung 7 (optimize & convert). Convert to an iterative loop with `lo / hi` — the tail-recursive shape maps directly to a `while lo <= hi` loop, giving $O(1)$ stack.

Family 5 — Dynamic programming (worked ladder: climbing stairs)

Rung 1 (trace). `ways(4)` trusts `ways(3)` and `ways(2)`; with `ways(0)=ways(1)=1` you get 1,1,2,3,5 → `ways(4)=5`.

Rung 2 (base case).

```
def ways(n):  
    if ____:          # ← fill: n <= 1  
        return 1  
    return ways(n-1) + ways(n-2)
```

JavaScript:

```
function ways(n) {  
    if (____) {          // ← fill: n <= 1  
        return 1;  
    }  
    return ways(n - 1) + ways(n - 2);  
}
```

Rung 3 (recursive call).

```
def ways(n):  
    if n <= 1: return 1  
    return ____ + ____ # ← ways(n-1), ways(n-2)
```

JavaScript:

```
function ways(n) {  
    if (n <= 1) return 1;  
    return ____ + ____; // ← ways(n-1), ways(n-2)  
}
```

Rung 4 (debug). Why is this exponential, and what one line fixes it?

```
# Plain version recomputes ways(n-2) etc. many times.  
# Fix: add @lru_cache(maxsize=None) above the function → each n computed once.
```

JavaScript:

```
// Plain version recomputes ways(n-2) etc. many times.  
// Fix: wrap in a Map-based memo so each n is computed once (JS has no @lru_cache).
```

Rung 5 (derive). From the value recurrence `ways(n)=ways(n-1)+ways(n-2)`, write the function (Rung 3), then memoize (Rung 4).

Rung 6 (write from blank). "Count the ways to climb n stairs taking 1 or 2 at a time." From scratch, then memoize.

Rung 7 (optimize & convert). Tabulate bottom-up with two rolling variables for O(n) time, O(1) space: `a,b=1,1; for _ in range(2,n+1): a,b=b,a+b; return b`. This is the full recursion → memoization → tabulation pipeline of Chapter 39.

Practicing these five complete ladders — rather than only writing final solutions — is the single most effective way to convert reading-comprehension into the ability to *produce* recursion under exam conditions. When a new problem appears, you'll find your hand reaching for the right rung automatically.

Fully worked problems (the complete format)

Each problem below is worked the way you should work every gym problem: statement, example, the two design prompts, hints, a skeleton, the solution, a common wrong answer, and complexity.

Worked problem A — Sum of digits until a single digit (digital root).

Statement. Repeatedly sum the digits of a non-negative integer until one digit remains. `dr(9875) → 9+8+7+5=29 → 2+9=11 → 1+1=2`.

Example. `dr(9875) = 2`, `dr(0) = 0`, `dr(7) = 7`.

Name the smaller problem. Summing the digits once produces a strictly smaller number; the digital root of *that* is the answer — a smaller instance of the same problem.

State the contract. `dr(n)` returns the single-digit digital root of non-negative integer `n`.

Hint 1. You need a helper that sums digits once (that itself is Chapter 13's `digit_sum`). *Hint 2.* Base case: if `n` is already a single digit (`n < 10`), return it.

Skeleton.

```
def dr(n):
    if n < 10:
        return ____ # base
    return dr(____) # recurse on the once-summed value
```

JavaScript:

```
function dr(n) { // Precondition: n >= 0
    if (n < 10) return n;
    let s = 0, m = n;
    while (m) { s += m % 10; m = Math.floor(m / 10); }
    return dr(s);
}
```

Solution.

```
def dr(n):
    """Precondition: n >= 0. Returns the digital root (a single digit)."""
    if n < 10:
        return n
    s = 0
    while n: # sum digits once
        s += n % 10; n //= 10
    return dr(s) # recurse on the smaller number
```

JavaScript:

```
function dr(n) {
    // Precondition: n >= 0. Returns the digital root (a single digit).
    if (n < 10) return n;
    let s = 0, m = n;
    while (m) { s += m % 10; m = Math.floor(m / 10); } // sum digits once
    return dr(s); // recurse on the smaller number
}
```

Common wrong answer. Returning `n % 9` — the closed-form trick — fails for multiples of 9 (it gives 0 instead of 9) unless special-cased. The recursion is safer and clearer.

Complexity. Summing the digits once is $O(\log n)$ work (proportional to the digit count). The recursion depth — how many times you must re-sum before reaching one digit — grows like the iterated logarithm $\log^*(n)$, which is at most about 5 or 6 for any integer you could actually store, but is *not* literally $O(1)$ for unbounded integers. So: $O(\log n)$ work at the top level, an extraordinarily shallow $O(\log^* n)$ call chain, and $O(\log^* n)$ stack space.

Worked problem B — Count paths in a grid with obstacles.

Statement. A robot at the top-left of an `m×n` grid moves only right or down to the bottom-right. Some cells are blocked (`1`). Count the distinct clear paths.

Example. For `[[0,0,0],[0,1,0],[0,0,0]]` the answer is `2`.

Name the smaller problem. The number of paths from cell `(r,c)` is paths-from-below plus paths-from-the-right — each a smaller cell nearer the goal.

State the contract. `paths(r,c)` returns the number of clear right/down paths from `(r,c)` to the bottom-right corner.

Hint 1. Base cases: off-grid or blocked $\rightarrow 0$; the goal cell $\rightarrow 1$. *Hint 2.* Overlapping subproblems — memoize on `(r,c)`.

Skeleton.

```

from functools import lru_cache
def unique_paths(grid):
    m, n = len(grid), len(grid[0])
    @lru_cache(maxsize=None)
    def paths(r, c):
        if r >= m or c >= n or grid[r][c] == 1:
            return 0
        if r == m - 1 and c == n - 1:
            return 1
        return paths(r + 1, c) + paths(r, c + 1)
    return paths(0, 0)

```

JavaScript:

```

function uniquePaths(grid) {
    const m = grid.length, n = grid[0].length;
    const memo = new Map();
    function paths(r, c) {
        if (r >= m || c >= n || grid[r][c] === 1) return 0; // off-grid or blocked
        if (r === m - 1 && c === n - 1) return 1; // reached the goal
        const key = r + "," + c;
        if (memo.has(key)) return memo.get(key);
        const v = paths(r + 1, c) + paths(r, c + 1); // down + right
        memo.set(key, v);
        return v;
    }
    return paths(0, 0);
}

```

Solution.

```

from functools import lru_cache
def unique_paths(grid):
    """Precondition: grid is non-empty; start/goal are clear. Returns path count."""
    m, n = len(grid), len(grid[0])
    @lru_cache(maxsize=None)
    def paths(r, c):
        if r >= m or c >= n or grid[r][c] == 1:
            return 0 # base: off-grid or blocked
        if r == m - 1 and c == n - 1:
            return 1 # base: reached the goal
        return paths(r + 1, c) + paths(r, c + 1) # down + right
    return paths(0, 0)

```

JavaScript:

```

function uniquePaths(grid) {
    // Precondition: grid is non-empty; start/goal are clear.
    const m = grid.length, n = grid[0].length;
    const memo = new Map();
    function paths(r, c) {
        if (r >= m || c >= n || grid[r][c] === 1) return 0; // base: off-grid or blocked
        if (r === m - 1 && c === n - 1) return 1; // base: reached the goal
        const key = r + "," + c;
        if (memo.has(key)) return memo.get(key);
        const v = paths(r + 1, c) + paths(r, c + 1); // down + right
        memo.set(key, v);
        return v;
    }
    return paths(0, 0);
}

```

Common wrong answer. Forgetting the blocked-cell base case, which counts paths through walls; or omitting memoization, which is correct but exponential.

Complexity. States = $m \cdot n$, $O(1)$ each $\rightarrow$ **$O(mn)$** time and space (Chapter 35's states $\times$ work rule).

Worked problem C — Generate all valid IP address segmentations.

Statement. Given a digit string, insert three dots to form a valid IPv4 address (four parts, each 0-255, no leading

zeros). Return all possibilities.

Example. "25525511135" → ["255.255.11.135", "255.255.111.35"] .

Name the smaller problem. After committing one valid segment (1-3 digits) at the front, the task is to split the remaining string into one fewer part — a smaller instance.

State the contract. `place(start, parts_left)` builds all valid completions from index `start` with `parts_left` segments still to place.

Hint 1. Base case: `parts_left == 0` is a success only if `start` reached the end. Hint 2. Try segment lengths 1, 2, 3; validate each (`<= 255`, no leading zero) before recursing — that validation is the pruning.

Solution.

```
def restore_ips(s):
    """Precondition: s is a string of digits. Returns all valid IPv4 strings."""
    out = []
    def valid(seg):
        return len(seg) == 1 or (seg[0] != '0' and int(seg) <= 255)
    def place(start, parts, acc):
        if parts == 0:
            if start == len(s):
                out.append('.'.join(acc))    # a complete valid address
            return
        for length in (1, 2, 3):           # try each segment length
            seg = s[start:start + length]
            if len(seg) == length and valid(seg):
                place(start + length, parts - 1, acc + [seg]) # choose + recurse
    place(0, 4, [])
    return out
```

JavaScript:

```
function restoreIps(s) {
    // Precondition: s is a string of digits. Returns all valid IPv4 strings.
    const out = [];
    const valid = seg => seg.length === 1 || (seg[0] !== "0" && parseInt(seg, 10) <= 255);
    function place(start, parts, acc) {
        if (parts === 0) {
            if (start === s.length) out.push(acc.join(".")); // a complete valid address
            return;
        }
        for (let len = 1; len <= 3; len++) { // try each segment length
            const seg = s.slice(start, start + len);
            if (seg.length === len && valid(seg)) {
                place(start + len, parts - 1, [...acc, seg]); // choose + recurse
            }
        }
    }
    place(0, 4, []);
    return out;
}
```

Common wrong answer. Allowing leading zeros ("01"), or not checking that the string is fully consumed when four parts are placed.

Complexity. The IPv4 specification fixes the structure: exactly four segments, each at most three digits. So the branching factor is at most 3 and the recursion depth is exactly 4 — the explored search space, the recursion depth, and the output size are all bounded by a small constant, independent of any notion of growing `n`. There is no asymptotic growth to speak of; the work is $O(1)$ under the IPv4 spec.

These three show the format you should impose on *every* problem you practice: never just write code, but first name the smaller problem and state the contract, and afterward name a plausible wrong answer and the complexity. That discipline is what turns a solved problem into transferable intuition.

Answer key for the per-chapter "Active practice" boxes

These are the answers to the small predict/complete/diagnose/derive boxes placed at the ends of Chapters 14, 15, 19, 21, and 23. Try each before reading here.

§14 — List recursion. (1) It trusts `count([2,3], 2)` to return `1` (one 2 in the tail), then adds 1 for the head, returning `2`. (2) `(1 if xs[0] > t else 0)`. (3) It errors on the empty list `[]` because there's no base case — `xs[0]` fails; add `if not xs: return 1` (1 is the identity for multiplication). (4) Second-largest distinct: track the two largest distinct values as you recurse (or collect, dedupe, and take the second); adjacent-duplicate removal: `dedup(xs) = xs if len(xs)<2 else ([xs[0]] + dedup(xs[1:])) if xs[0]!=xs[1] else dedup(xs[1:])`.

§15 — Trees. (1) The root trusts each child call to return the *sum of that entire subtree*. (2) base case returns `0`. (3) The off-by-one is almost always the base case: returning `1` for `None` instead of `0` (or `0` for a leaf instead of `1`, depending on whether you count nodes or edges). (4) `max_depth_value` must return *both* the deepest value and its depth so the parent can compare — a direct application of Chapter 16b's "return exactly what the parent needs": return a `(depth, value)` pair and let the parent keep the deeper one.

§19 — Backtracking. (1) `current` must be back exactly as it was before the include branch — the `current.pop()` must have run, restoring the pre-choice state (the undo invariant). (2) add `)` only when `close < open`. (3) The line that records the solution is appending a *reference* instead of a copy — use `out.append(current[:])` (or `[...current]`). (4) Place one rook per row, tracking used columns in a set; after exploring, remove the column (undo). Invariant: after each `undo`, the used-columns set and the partial placement are identical to their state before that choice.

§21 — Divide and conquer. (1) `9 > 6` (the midpoint value at index 2), so it explores the *right* half `[9, 11]`. (2) `mid = len(xs) // 2`. (3) `mid` is misused where `mid-1/mid+1` belongs — recursing on a range that still includes `mid` (e.g. `lo..mid` instead of `lo..mid-1`) can leave the range unchanged, so no progress is made. (4) `maxOf(a) = a[0] if len(a)==1 else max(maxOf(left half), maxOf(right half))`; recurrence $T(n)=2T(n/2)+O(1)$, which by the Master Theorem ($a=2, b=2, c=1$, case 1) is $\Theta(n)$.

§23 — Dynamic programming. (1) Exactly 21 distinct states (`climb(0)` through `climb(20)`), computed once each. (2) `@lru_cache(maxsize=None)`. (3) The base case (or guard) for a *blocked* cell was omitted — `if grid[r][c] == 1: return 0` — so paths are counted through walls. (4) State is `(r, c)`; recurrence `best(r,c) = grid[r][c] + min(best(r+1,c), best(r,c+1))` with the edge/last-cell base cases; states = $m \cdot n$, $O(1)$ work each $\rightarrow O(mn)$ time and space (states $\times$ work, Chapter 35).

Solutions bank (worked answers to selected gym problems)

Count leaves in a binary tree. Contract: number of nodes with no children. Base: `null` $\rightarrow$ 0; a leaf $\rightarrow$ 1. Combine: `leaves(left) + leaves(right)`.

```
def count_leaves(nd):
    if nd is None: return 0
    if nd.left is None and nd.right is None: return 1
    return count_leaves(nd.left) + count_leaves(nd.right)
```

JavaScript:

```
function countLeaves(nd) {
    if (nd === null) return 0;
    if (nd.left === null && nd.right === null) return 1;
    return countLeaves(nd.left) + countLeaves(nd.right);
}
```

Validate a BST. State: node plus the (lo, hi) range it must lie in. Each left child tightens the upper bound; each right child tightens the lower.

```
def is_bst(nd, lo=float('-inf'), hi=float('inf')):
    if nd is None: return True
    if not (lo < nd.value < hi): return False
    return is_bst(nd.left, lo, nd.value) and is_bst(nd.right, nd.value, hi)
```

JavaScript:

```
function isBst(nd, lo = -Infinity, hi = Infinity) {
    if (nd === null) return true;
    if (!(lo < nd.value && nd.value < hi)) return false;
    return isBst(nd.left, lo, nd.value) && isBst(nd.right, nd.value, hi);
}
```

Number of ways to make change (unbounded coins). Use a `start` index so `{1,2}` and `{2,1}` aren't double-counted: either use `coin[i]` again (stay at `i`) or move past it (`i+1`).

```

from functools import lru_cache
def change(coins, amount):
    """Precondition: every coin is a POSITIVE integer, amount >= 0.
    A zero coin makes `ways(i, rem - 0)` recurse on the same state forever."""
    @lru_cache(maxsize=None)
    def ways(i, rem):
        if rem == 0: return 1
        if i == len(coins) or rem < 0: return 0
        return ways(i, rem - coins[i]) + ways(i + 1, rem) # reuse coin[i] OR skip it
    return ways(0, amount)

```

JavaScript:

```

function change(coins, amount) {
    // Precondition: every coin is a POSITIVE integer, amount >= 0.
    const memo = new Map();
    function ways(i, rem) {
        if (rem === 0) return 1;
        if (i === coins.length || rem < 0) return 0;
        const key = i + "," + rem;
        if (memo.has(key)) return memo.get(key);
        const v = ways(i, rem - coins[i]) + ways(i + 1, rem); // reuse coin[i] OR skip it
        memo.set(key, v);
        return v;
    }
    return ways(0, amount);
}

```

Complexity: states = len(coins)·amount, O(1) each → O(coins·amount).

House robber (no two adjacent). State: index. Choice: rob i (skip i+1) or skip i. Optimize with max.

```

from functools import lru_cache
def rob(nums):
    @lru_cache(maxsize=None)
    def best(i):
        if i >= len(nums): return 0
        return max(nums[i] + best(i + 2), best(i + 1)) # rob i vs skip i
    return best(0)

```

JavaScript:

```

function rob(nums) {
    const memo = new Map();
    function best(i) {
        if (i >= nums.length) return 0;
        if (memo.has(i)) return memo.get(i);
        const v = Math.max(nums[i] + best(i + 2), best(i + 1)); // rob i vs skip i
        memo.set(i, v);
        return v;
    }
    return best(0);
}

```

Longest increasing subsequence (O(n²) DP). State: index of the last element kept plus the previous index (or its value). A compact top-down form:

```

from functools import lru_cache
def lis(nums):
    @lru_cache(maxsize=None)
    def best(i, prev): # prev = index of previously chosen element, -1 if none
        if i == len(nums): return 0
        skip = best(i + 1, prev)
        take = 0
        if prev == -1 or nums[i] > nums[prev]:
            take = 1 + best(i + 1, i)
        return max(take, skip)
    return best(0, -1)

```

JavaScript:

```
function lis(nums) {
  const memo = new Map();
  function best(i, prev) { // prev = index of previously chosen element, -1 if none
    if (i === nums.length) return 0;
    const key = i + "," + prev;
    if (memo.has(key)) return memo.get(key);
    let skip = best(i + 1, prev), take = 0;
    if (prev === -1 || nums[i] > nums[prev]) take = 1 + best(i + 1, i);
    const v = Math.max(take, skip);
    memo.set(key, v);
    return v;
  }
  return best(0, -1);
}
```

Word search in a grid (backtracking + flood fill). From each start cell, try to match the word by walking to neighbors, marking cells used, and unmarking on the way out.

```
def exist(board, word):
    R, C = len(board), len(board[0])
    def dfs(r, c, k):
        if k == len(word): return True
        if not (0 <= r < R and 0 <= c < C) or board[r][c] != word[k]:
            return False
        tmp, board[r][c] = board[r][c], '#' # mark visited
        found = (dfs(r+1,c,k+1) or dfs(r-1,c,k+1) or
                 dfs(r,c+1,k+1) or dfs(r,c-1,k+1))
        board[r][c] = tmp # undo
        return found
    return any(dfs(r, c, 0) for r in range(R) for c in range(C))
```

JavaScript:

```
function exist(board, word) {
  const R = board.length, C = board[0].length;
  function dfs(r, c, k) {
    if (k === word.length) return true;
    if (r < 0 || c < 0 || r >= R || c >= C || board[r][c] !== word[k]) return false;
    const tmp = board[r][c];
    board[r][c] = "#"; // mark visited
    const found = dfs(r+1,c,k+1) || dfs(r-1,c,k+1) ||
                  dfs(r,c+1,k+1) || dfs(r,c-1,k+1);
    board[r][c] = tmp; // undo
    return found;
  }
  for (let r = 0; r < R; r++)
    for (let c = 0; c < C; c++)
      if (dfs(r, c, 0)) return true;
  return false;
}
```

Count inversions (divide & conquer, piggybacks on merge sort). While merging, every time you take from the right half before the left half is exhausted, all remaining left elements form inversions.

```
def count_inversions(xs):
    def sort_count(a):
        if len(a) <= 1: return a, 0
        mid = len(a)//2
        left, cl = sort_count(a[:mid])
        right, cr = sort_count(a[mid:])
        merged, cm, i, j = [], 0, 0, 0
        while i < len(left) and j < len(right):
            if left[i] <= right[j]:
                merged.append(left[i]); i += 1
            else:
                merged.append(right[j]); j += 1
                cm += len(left) - i # left[i:] all exceed right[j]
        merged += left[i:]; merged += right[j:]
        return merged, cl + cr + cm
    return sort_count(xs)[1]
```

JavaScript:

```
function countInversions(xs) {
  function sortCount(a) {
    if (a.length <= 1) return [a, 0];
    const mid = Math.floor(a.length / 2);
    const [left, cl] = sortCount(a.slice(0, mid));
    const [right, cr] = sortCount(a.slice(mid));
    const merged = []; let cm = 0, i = 0, j = 0;
    while (i < left.length && j < right.length) {
      if (left[i] <= right[j]) merged.push(left[i++]);
      else { merged.push(right[j++]); cm += left.length - i; } // left[i:] all exceed right[j]
    }
    return [merged.concat(left.slice(i)).concat(right.slice(j)), cl + cr + cm];
  }
  return sortCount(xs)[1];
}
```

Complexity: same recurrence as merge sort, $T(n)=2T(n/2)+O(n) \rightarrow \Theta(n \log n)$.

Work these until you can reproduce each from just its pattern name and state definition — that recall is precisely what a timed exam rewards.

Chapter 49. Cheat sheet

The core idea. Solve a problem by trusting a solution to a smaller version of the same problem, plus a little of your own work.

Every recursion needs. (1) at least one reachable base case; (2) a recursive case that calls itself on strictly smaller input; (3) a way to combine.

The five-step recipe. State the contract $\rightarrow$ find the base case $\rightarrow$ assume the recursive call works $\rightarrow$ do one step and combine $\rightarrow$ verify you shrink toward the base case.

The list template. Index-based (production form — $O(n)$, no copying):

```
def f(xs, i=0):
    if i == len(xs): return BASE
    return COMBINE(xs[i], f(xs, i + 1))
# Conceptual form f(xs): `return COMBINE(xs[0], f(xs[1:]))` is clearer to read
# but  $O(n^2)$  in Python because xs[1:] copies the tail each call. Prefer the index form.
```

JavaScript:

```
function f(xs, i = 0) {
  if (i === xs.length) return BASE;
  return COMBINE(xs[i], f(xs, i + 1));
}
// Conceptual form f(xs): `return COMBINE(xs[0], f(xs.slice(1)))` is clearer to read
// but  $O(n^2)$  because slice copies the tail each call. Prefer the index form.
```

The tree template.

```
def f(node):
    if node is None: return BASE
    return COMBINE(node.value, f(node.left), f(node.right))
```

JavaScript:

```
function f(node) {
  if (node === null) return BASE;
  return COMBINE(node.value, f(node.left), f(node.right));
}
```

The backtracking template.

```
def bt(state):
    if complete(state): record(state); return
    for choice in choices(state):
        if valid(state, choice):
            make(choice); bt(state); undo(choice)
```

JavaScript:

```
function bt(state) {
    if (complete(state)) { record(state); return; }
    for (const choice of choices(state)) {
        if (valid(state, choice)) {
            make(choice); bt(state); undo(choice);
        }
    }
}
```

The divide-and-conquer template.

```
def f(data):
    if small(data): return base(data)
    a, b = split(data)
    return combine(f(a), f(b))
```

JavaScript:

```
function f(data) {
    if (small(data)) return base(data);
    const [a, b] = split(data);
    return combine(f(a), f(b));
}
```

The memoization move. If the same subproblem recurs, cache it (`@lru_cache` in Python) — that's dynamic programming.

The pick / don't-pick template (solves most "choose a subset" problems).

```
def solve(i, state):
    if i == n: return leaf(state)
    return combine(solve(i+1, take(state)), solve(i+1, state))
# combine = concat (enumerate) | or (feasibility) | + (count) | max/min (optimize)
```

JavaScript:

```
function solve(i, state) {
    if (i === n) return leaf(state);
    return combine(solve(i + 1, take(state)), solve(i + 1, state));
}
// combine = concat (enumerate) | || (feasibility) | + (count) | Math.max/min (optimize)
```

The recursion → DP pipeline. Write plain recursion → add `@lru_cache` (top-down) → convert to a table (bottom-up). The recurrence never changes; only the evaluation order does. Preconditions: a **well-defined, path-independent state** and **overlapping subproblems** (the same state recurs); *optimization* problems additionally need **optimal substructure** (the best answer is built from best sub-answers). Counting and feasibility recurrences need only the first two.

Recurrence → complexity quick reference.

Recurrence	Solved by	Result
$T(n) = T(n-1) + O(1)$	substitution	$\Theta(n)$
$T(n) = T(n-1) + O(n)$	substitution	$\Theta(n^2)$
$T(n) = T(n/2) + O(1)$	master (c=0, case 2)	$\Theta(\log n)$
$T(n) = 2T(n/2) + O(1)$	master (case 1)	$\Theta(n)$
$T(n) = 2T(n/2) + O(n)$	master (case 2)	$\Theta(n \log n)$
$T(n) = 2T(n/2) + O(n^2)$	master (case 3)	$\Theta(n^2)$
$T(n) = 2T(n-1) + O(1)$	decreasing form ($a > 1 \rightarrow \Theta(a^{n/b})$)	$\Theta(2^n)$
$T(n) = T(n-1) + T(n-2) + O(1)$	recursion tree / characteristic eq.	$\Theta(\varphi^n)$, $\varphi \approx 1.618$

Master Theorem in one line. For $T(n)=a \cdot T(n/b)+f(n)$: compute $n^{\log_b a}$; whichever of it or $f(n)$ grows faster wins; if they tie, multiply by $\log n$. *Applies only when the recurrence matches this form and the polynomial-gap and regularity conditions hold* — it does **not** apply to subtractive shrink (see the decreasing-form rows above) or to gap cases like $f(n)=n^c/\log n$. Recurrences with uneven splits (e.g. $T(n)=T(n/3)+T(2n/3)+n$), cyclic state dependencies, or non-polynomial f need other tools: the Akra-Bazzi method, the recursion-tree method, or the characteristic-equation method for linear recurrences. Not every recurrence fits a formula.

DP complexity rule. Time = (#distinct states) $\times$ (work per state). Space = (memo table size) + (max recursion depth).

Pattern signal words are heuristics, not proofs. The phrases below *suggest* a candidate technique; they don't confirm it. Always verify the state, the recurrence, the constraints, and whether subproblems actually overlap before committing — "minimum" or "how many ways" often means DP, but not always.

Top bugs. Missing/unreachable base case; recursing on input that isn't smaller; forgetting to return the recursive result; recomputing overlapping subproblems.

Hidden-cost traps. Slicing `xs[1:]` is $O(n)$ (turns linear into quadratic); string `+` in recursion copies each level; number of *calls* $\neq$ stack space (depth does); a base case that scans input isn't $O(1)$.

The runtime truth. Recursion uses the call stack; depth = memory cost. Python (CPython) has no tail-call optimization and a recursion limit that commonly starts near 1000 (read it via `sys.getrecursionlimit()`; implementation-dependent). Very deep recursion $\rightarrow$ convert to a loop or explicit stack.

The one habit. When stuck, don't trace inward — take the leap of faith: assume the smaller call already works, and ask only "what do I do with its answer?"

Chapter 49b. The exercise bank (practice without scaffolding)

The worked ladders in Chapter 48 show every answer immediately, which is good for *learning* but poor for *mastery* — you can't tell whether you could have produced the answer yourself. Real mastery comes from **retrieval practice**: attempting problems cold, with hints available only when you're stuck, and solutions kept out of sight until you've committed to an answer.

This chapter is organized so you can practice honestly. Each problem gives the statement and, separately, progressive hints and a solution sketch you should not read until you've tried. **Cover the hints and solutions.** Attempt each problem on paper first; consult a hint only after a genuine stall; check the solution only after you have written something complete.

Unsolved ladders (one per family)

Climb all seven rungs (trace $\rightarrow$ base case $\rightarrow$ recursive call $\rightarrow$ debug $\rightarrow$ derive $\rightarrow$ write $\rightarrow$ optimize) for each. The rungs are defined in Chapter 48.

L1 — Linear: product of a list. Return the product of all numbers in a list (empty list $\rightarrow$ 1). **L2 — Trees: count nodes with exactly one child.** In a binary tree, count nodes that have exactly one non-null child. **L3 — Backtracking: all combinations that sum to a target.** Given positive integers and a target, print every combination (each number usable once) that sums to the target. **L4 — Divide & conquer: find the peak.** In an array that increases then decreases, find the peak element in $O(\log n)$. **L5 — DP: minimum path sum in a grid.** From top-left to bottom-right moving only right/down, find the minimum sum of visited cells.

Level A — Trace (predict output, no running)

A1. `f([2,4,6])` where `f(xs)= 0 if not xs else xs[0]+f(xs[1:])` . What does `f` trust `f([4,6])` to return, and what is the final result? **A2.** `g("abcd")` where `g(s)= s if len(s)<=1 else g(s[1:])+s[0]` . What does it produce? **A3.** `h(3)` where `h(n)= 1 if n==0 else h(n-1)+h(n-1)` . How many times is the base case reached?

Level B — Fill the blank

B1 (base case). Count down and print; supply the base case.

```
def countdown(n):
    if ____:          # ?
        return
    print(n); countdown(n-1)
```

JavaScript:

```
function countdown(n) {
    if (____) return;    // ?
    console.log(n); countdown(n - 1);
}
```

B2 (recursive call). Sum a binary tree; supply the two calls.

```
def tree_sum(node):
    if node is None: return 0
    return node.value + ____ + ____    # ?
```

JavaScript:

```
function treeSum(node) {
    if (node === null) return 0;
    return node.value + ____ + ____;    // ?
}
```

B3 (combine). Check if a target exists in a sorted array via binary search; supply the combine branches.

```
def bsearch(a, t, lo, hi):
    if lo > hi: return False
    mid = (lo+hi)//2
    if a[mid] == t: return True
    if t < a[mid]: return ____    # ?
    return ____    # ?
```

JavaScript:

```
function bsearch(a, t, lo, hi) {
    if (lo > hi) return false;
    const mid = Math.floor((lo + hi) / 2);
    if (a[mid] === t) return true;
    if (t < a[mid]) return ____;    // ?
    return ____;    // ?
}
```

Level C — Debug (one bug each)

C1. Reverse a string; it returns the string unchanged. Why?

```
def rev(s):
    if len(s) <= 1: return s
    return s[0] + rev(s[1:])
```

JavaScript:


```
function rev(s) {
  if (s.length <= 1) return s;
  return s[0] + rev(s.slice(1));
}
```

C2. Count list length; it raises an error on `[]`. Why?

```
def length(xs):
  return 1 + length(xs[1:])
```

JavaScript:

```
function length(xs) {
  return 1 + length(xs.slice(1));
}
```

C3. Fibonacci; it's correct but unusably slow for `n=45`. What single change fixes it?

Level D — Mixed problems (pattern NOT given)

For each, first *name the pattern* (which Chapter 36 row?), then define the state and write the recursion. Deciding the pattern is the point.

D1. Given a string, return all ways to split it into palindromic substrings. **D2.** Given a staircase and a list of allowed step sizes, count the ways to reach the top. **D3.** Given a binary tree, return its maximum root-to-leaf path sum. **D4.** Given two strings, return the length of their longest common substring (contiguous, unlike subsequence). **D5.** Given `n`, return all structurally distinct binary search trees storing values `1..n` (count only).

Level E — End-of-part mastery tests

Test 1 (after Part IV). (a) Write `flatten` for arbitrarily nested lists. (b) Reverse a linked list recursively and state which phase does the rewiring. (c) Explain why walking a directory tree that contains a symlink cycle can loop forever, and name the fix.

Test 2 (after Part VI). (a) Write `subsets` two ways: return-value style and side-effect style. (b) Solve "combination sum" (numbers reusable) with backtracking. (c) Convert naive `fib` to memoized then to $O(1)$ -space tabulation.

Test 3 (after Part VIII). Give the tight Θ bound and the stack space for each: (a) $T(n)=2T(n/2)+O(n)$; (b) $T(n)=T(n-1)+O(1)$; (c) $T(n)=2T(n-1)+O(1)$; (d) $T(n)=T(n/2)+O(1)$; (e) LCS of two length-`n` strings, memoized.

Level F — Cumulative final test

Combine everything. Give yourself 45 minutes, no references.

F1. A robot on an `m×n` grid with some blocked cells starts top-left. (a) Count clear paths to the bottom-right (right/down only) with memoization; state the complexity. (b) Now the robot may move in all four directions and must not revisit a cell — count *all* simple paths, and explain why (a)'s memoization no longer applies. **F2.** Given a binary tree, (a) return its height, (b) determine whether it is height-balanced, and (c) state the time complexity of your balanced-check and whether it is $O(n)$ or $O(n^2)$, explaining why. **F3.** Given coins and an amount, (a) return the number of *combinations* that make the amount and (b) the *minimum* number of coins, then (c) state which coin-change row (Chapter 37) each corresponds to and the transition each uses. **F4.** Prove, by the induction correspondence of Chapter 4b, that your `F2(a)` height function is correct.

Hints (consult only when stuck)

L1-L5: L1 combine with `*`, base returns 1. L2 a node counts if exactly one of its children is None but not both. L3 use a `start` index and subtract from the target; prune when `target < 0`. L4 compare `a[mid]` with `a[mid+1]` to decide which side the peak is on. L5 `best(r,c)=grid[r][c]+min(best(r+1,c),best(r,c+1))`, memoize on `(r,c)`.

A1: trusts `f([4,6])=10`, result `12`. **A2:** reverses `→ "dcba"`. **A3:** base reached `23 = 8` times.

B1: `n < 0` — or better, `n == 0` if you stop at zero; match the intended range. **B2:** `tree_sum(node.left)`, `tree_sum(node.right)`. **B3:** `bsearch(a,t,lo,mid-1)`, then `bsearch(a,t,mid+1,hi)`.

C1: it's actually correct as a reversal only if you swap to `rev(s[1:]) + s[0]` — as written it returns the string unchanged because `s[0]+rev(s[1:])` reassembles the original order; the bug is the concatenation order. **C2:** no base

case — add `if not xs: return 0`. C3: add `@lru_cache(maxsize=None)` (or a manual memo dict).

D1: backtracking; at each position, try every prefix that is a palindrome, recurse on the rest. D2: DP/counting; `ways(n)=Σ ways(n-step)`. D3: tree return-value. Distinguish two different "max path" problems. For **root-to-leaf** path sum, you *cannot* clamp with `max(0, ...)` — a genuine root-to-leaf path must go all the way to a leaf even through negative values: `bestLeaf(node) = node.value + max(bestLeaf(left), bestLeaf(right))`, ignoring null children (use $-\infty$ for a missing child so it never wins). For the **any-node-to-any-node** maximum path (which may start and end anywhere), *there* you use `gain(node) = node.value + max(0, gain(left), gain(right))` — the `max(0, ...)` is valid only because you're allowed to drop a negative branch entirely, and you update a global best with `node.value + max(0, gain(left)) + max(0, gain(right))`. Don't mix the two. D4: 2-D DP on `(i, j)` but reset to 0 on mismatch (unlike LCS). D5: Catalan-number recursion; pick each value as root, multiply counts of left/right subtrees.

E/F: these integrate earlier chapters directly; if a part stalls, revisit the cited chapter (14b for linked lists and symlinks, 37 for coin-change rows, 4b for the induction proof, 33 for stack space). For F2(b), the naive balanced-check is $O(n^2)$ because it recomputes height at every node; the $O(n)$ version returns height and a balanced-flag together in one pass.

Solution sketches (consult last)

Full runnable solutions to L1-L5, D1-D5, and the tests are intentionally left for you to complete — that is the retrieval practice. Every one of them is a direct instance of a template already in this book: L-problems map to Chapters 13/15/19/21/23; D1 and F1(b) are backtracking (Ch. 19); D2, D5, F1(a), F3 are DP (Ch. 23, 37, 39); D3, F2 are tree return-value recursion (Ch. 15); Test 3 and F-complexities are Chapter 32's recurrences. If you can state the pattern, define the state, and write the recurrence for each, you have reached the goal of this book. Check your code by running it against the examples, and by the induction checklist of Chapter 4b.

The difference between reading this book and *mastering* recursion is precisely the work in this chapter. Do these cold.

Chapter 50. Glossary

Accumulator. An extra parameter that carries a partial result *downward* through the recursion, so the answer is assembled by the time the base case is reached, rather than combined on the way up.

Backtracking. A search that walks a tree of decisions depth-first: choose an option, explore, and if it leads to a dead end, *undo* the choice and try the next. See Chapter 19.

Base case. The smallest, simplest input, answered directly with no further recursion. It is what stops the recursion and begins the return journey.

Branch / branching recursion. A recursive case that makes more than one recursive call (e.g. Fibonacci, tree traversal), producing a branching call tree. Contrast *linear recursion*.

Call stack. The runtime structure that holds one *frame* per unfinished call. Its maximum depth is the recursion's space cost.

Combine step. The work that merges the recursive call's result(s) with the current call's own contribution to produce this call's answer.

Contract. A one-sentence statement of what a function returns for a general input — written before the implementation. The anchor for designing any recursion.

Corecursion. A dual of recursion that *builds output up* indefinitely (e.g. an infinite stream), defined by how it produces rather than how it terminates. See Chapter 28.

Decreasing recurrence. A recurrence where the subproblem shrinks by subtraction, `T(n)=a·T(n-b)+f(n)`. Solved by the decreasing-form rule, not the Master Theorem.

Depth (recursion depth). The length of the longest chain of nested calls — equivalently, the maximum height of the call stack. Determines stack-space cost.

Divide and conquer. A strategy that splits a problem into independent parts (usually halves), solves each recursively, and combines. Yields recurrences of the form `T(n)=a·T(n/b)+f(n)`.

Dynamic programming (DP). Memoized recursion (top-down) or its bottom-up table equivalent (tabulation), applicable when a problem has *optimal substructure* and *overlapping subproblems*.

Frame (stack frame). The workspace for a single call: its parameters, local variables, and the bookmark for where to resume when the call it made returns.

Leap of faith. The practice of assuming the recursive call already works correctly for smaller inputs, instead of tracing it — the core mental move of writing recursion.

Linear recursion. A recursive case making exactly one recursive call (e.g. `factorial`, `sum_list`). Contrast *branching recursion*.

Master Theorem. A formula that solves dividing recurrences $T(n)=a \cdot T(n/b)+f(n)$ by comparing $f(n)$ to $n^{(\log_b a)}$. See Chapter 32.

Memoization. Caching each subproblem's result the first time it is computed, so repeated subproblems are looked up rather than recomputed. Top-down DP.

Mutual recursion. Two or more functions that call one another (e.g. `is_even` / `is_odd`, or a recursive-descent parser's rule functions).

Optimal substructure. The property that a problem's optimal solution is built from optimal solutions of its subproblems — a precondition for DP.

Overlapping subproblems. The property that the same subproblem recurs multiple times in the recursion — the precondition that makes memoization pay off.

Precondition. The set of inputs a function is defined for. Recursion design begins here, because the precondition guarantees the "smaller" input reaches a base case.

Recurrence relation. An equation expressing $T(n)$ (time, or a sequence term) in terms of smaller inputs, e.g. $T(n)=2T(n/2)+O(n)$.

Recursion tree. A drawing of all recursive calls as a tree, used to sum work per level and derive complexity.

State. The minimal set of parameters that fully describes a subproblem, independent of how it was reached. The memo key and, in DP, the table's dimensions.

Structural recursion. Recursion whose smaller input is a piece of the input's own structure (list tail, subtree, substring). Termination follows whenever every recursive call operates on a *proper substructure of a finite, acyclic input* — which is why trees are safe but graphs with cycles are not (Chapter 15b). Contrast *generative recursion*, which computes a new smaller problem (e.g. `a mod b`) and requires a separate termination argument.

Subproblem. A smaller instance of the same problem that a recursive call solves.

Tabulation. Bottom-up DP: filling a table from base cases upward in dependency order, with no recursion. Same recurrence as the memoized version, different evaluation order.

Tail recursion. A recursion whose recursive call is the very last action, allowing frame reuse (tail-call optimization) in languages that support it. CPython does not.

Turnaround point. The moment the base case returns the first concrete value, ending the downward phase and starting the upward (returning) phase.

You should now be able to...

Use this as a final self-check. If any item feels shaky, the chapter in brackets is where to review.

- Recognize, in seconds, whether a problem is recursive and which family it belongs to. [Ch. 4, 36]
- Explain *why* the leap of faith is valid by mapping recursion onto mathematical induction. [Ch. 4b]
- Write a correct recursion by contract $\rightarrow$ base case $\rightarrow$ leap of faith $\rightarrow$ combine $\rightarrow$ shrink-check. [Ch. 6]
- Tell whether a piece of work belongs on the way *down* or the way *up*, and place it correctly. [Ch. 10b]
- Identify which of the three styles (return-value, accumulator, side-effect) a recursion uses, and debug it accordingly. [Ch. 10c]
- Handle numbers, lists, strings, trees, and nested data with the standard templates. [Ch. 13–16]
- Work recursively with linked lists, including reversal and merging by rewiring on the upward phase. [Ch. 14b]
- Distinguish tree recursion from graph traversal, and decide whether visited state should be retained or undone. [Ch. 15b, 19]
- Enumerate subsets, permutations, and combinations, and solve constraint puzzles by backtracking with pruning. [Ch. 18–19]
- Apply divide-and-conquer and recognize when halving beats peeling. [Ch. 21]
- Turn any overlapping-subproblem recursion into memoized then tabulated DP, mechanically. [Ch. 39]

- Write a recurrence from code and solve it by substitution, recursion tree, or the Master Theorem. [Ch. 31–32]
- State time *and* stack-space complexity, and spot the hidden-cost traps (slicing, concatenation, missing memoization). [Ch. 33–34]
- Name the pattern from a problem statement and reach for the matching template under time pressure. [Ch. 36–37, 45]

If you can do these, you have what the book set out to give you: not memorized solutions, but the intuition to derive them.

End of book.